

“Smart Restaurant Ordering and Kitchen Display System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

MYDAM ASWITHA PATEL 2520030001

GUTHA SUSHMA SREE 2520030098

NAGABHYRAVA YISHNAANI 2520030414

Under the Esteemed Guidance of

Dr S. Spandana

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



Declaration

The Project Report entitled **“Smart Restaurant Ordering and Kitchen Display System”** is a record of Bonafide work of **MYDAM ASWITHA PATEL - 2520030001, GUTHA SUSHMA SREE - 2520030098, NAGABHYRAVA YISHNAANI – 2520030414** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

MYDAM ASWITHA PATEL – 2520030001

GUTHA SUSHMA SREE – 2520030098

NAGABHYRAVA YISHNAANI – 2520030414

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **“Smart Restaurant Ordering and Kitchen Display System”** is a bonafide work done and submitted by **MYDAM ASWITHA PATEL - 2520030001, GUTHA SUSHMA SREE - 2520030098, NAGABHYRAVA YISHNAANI – 2520030414** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2025-2026**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. Ramatenki Sateesh Kumar**, and my guide **Dr S. Spandana**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)** Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

MYDAM ASWITHA PATEL – 2520030001

GUTHA SUSHMA SREE – 2520030098

NAGABHYRAVA YISHNAANI – 2520030414

Abstract

The restaurant industry is increasingly adopting digital technologies, but traditional food ordering systems still rely heavily on manual communication between customers and kitchen staff. This can cause order errors, delays, and difficulty in tracking order progress. To address this, our project focuses on the Design and Development of a Smart Restaurant Ordering and Kitchen Display System. Our core aim was to develop a unified digital platform that simplifies the food ordering process and improves coordination between customers, kitchen staff, and administrators. The system is structured around three primary interconnected modules: Customer, Kitchen, and Admin. The Customer Module allows users to register and log in, browse the menu, add food items to the cart, place orders, select a table, and track their order status. The Kitchen Display System (KDS) provides kitchen staff with a centralized view of incoming orders and allows them to update the order status from New → Accepted → Preparing → Ready → Served. The Admin Module enables administrators to manage menu items, tables, users, and orders through a dedicated dashboard. The system is developed using modern web technologies including React.js, Node.js, Express.js, REST APIs, and MySQL. The platform provides a simple, responsive, and user-friendly interface while maintaining centralized data management. In summary, this project provides a complete digital solution for restaurant ordering and kitchen management, helping reduce manual errors, improve order visibility, and make restaurant operations more efficient.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	5-15
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	16-20
3	Technology Stack	3.1 Front-end (e.g., React.js, Angular, HTML/CSS) 3.2 Back-end (e.g., Node.js, Django, Spring Boot) 3.3 Database (e.g., MongoDB, MySQL, PostgreSQL) 3.4 Version Control (e.g., Git, GitHub) 3.5 APIs or External services (e.g., Firebase, Stripe)	21-31
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)	32-36
5	Design	5.2 ER Diagram / Database schema	37
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	38-48
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	49-53
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	54-62
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	63-69
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	70-76
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	77-81
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	82-89
13	References	- Books, tutorials, APIs, documentation sites used	90-91

14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide -Geo Tag photos with guide -Review forms with guide signatures	92-98
----	------------	---	-------

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

The restaurant industry is rapidly adopting digital technologies to improve food ordering, customer service, and daily restaurant operations. Traditionally, customers depend on waiters or manual ordering methods to communicate their food choices to restaurant staff. Kitchen staff then receive and process these orders manually. This process can sometimes lead to incorrect orders, delays, difficulty in tracking order status, and communication gaps between customers and the kitchen. With the growth of web technologies, restaurants can replace these manual processes with centralized digital ordering systems. Customers can browse menus, select food items, manage their cart, place orders, select a table, and track the progress of their orders through a web application. At the same time, kitchen staff can receive orders through a dedicated Kitchen Display System (KDS) instead of relying on handwritten notes or verbal communication. The idea for this project emerged from identifying the need for a simple and integrated restaurant management solution that connects customers, kitchen staff, and administrators through a single system. The proposed “Smart Restaurant Ordering and Kitchen Display System” provides a unified platform for managing the complete food ordering process. The system connects the customer interface with the backend services and database, allowing information to flow efficiently between different modules. The proposed system is divided into three major modules: Customer Module, Kitchen Display System, and Admin Module. The Customer Module allows users to register and log in, browse available food items, add items to the cart, place orders, select tables, and view their order status.

The Kitchen Display System provides kitchen staff with a centralized view of incoming orders and allows them to update orders through different stages such as New, Accepted, Preparing, Ready, and Served. The Admin Module allows administrators to manage menu items, tables, users, and orders. The system is developed using modern web technologies. The frontend is developed using React.js, providing an interactive interface for customers and staff. The backend uses Node.js and Express.js to handle requests and provide REST APIs. MySQL is used as the database for storing user information, menu items, restaurant tables, orders, and order items. This architecture allows the different components of the application to communicate in an organized manner. From a database management perspective, the system maintains structured information about users, menu items, tables, and orders. Relationships between these entities help maintain consistency and make it easier to retrieve and manage restaurant information.

The system also supports authentication and role-based access so that customers, kitchen staff, and administrators can access their respective functionalities. The project also focuses on improving the overall restaurant workflow. Instead of depending completely on manual communication, orders are digitally transferred to the kitchen. Kitchen staff can view the ordered items and update their progress, while customers can track the current status of their orders. This helps improve order visibility and reduces unnecessary communication between the customer and kitchen. From an academic and engineering perspective, the project combines several important areas of computer science:

- 1.Web Development: Development of an interactive frontend using React.js, HTML, CSS, and JavaScript.
- 2.Backend Development: Implementation of server-side functionality using Node.js and Express.js.
- 3.Database Management: Storage and retrieval of users, menu items, tables, orders, and order items using MySQL.

4. REST API Integration: Communication between the frontend, backend, and database through REST APIs. administrators.

6. Software Engineering: Development of separate modules to improve organization, maintainability, and scalability.

The system is designed to provide a simple and efficient experience for both customers and restaurant staff. Customers can complete the ordering process digitally, while kitchen staff can manage orders through a dedicated display. Administrators can monitor and manage important restaurant information from a centralized dashboard. In a broader context, digital restaurant systems can help improve operational efficiency and provide better visibility into the ordering process. The proposed system focuses on reducing manual work, minimizing communication gaps, and providing a centralized platform for restaurant operations. To summarize, the background of this project is based on three main observations: *Traditional restaurant ordering can involve manual processes and communication gaps. *Modern web technologies make digital ordering and centralized order management practical. *A unified system can connect customers, kitchen staff, and administrators in a single platform. By addressing these requirements, the Smart Restaurant Ordering and Kitchen Display System provides a structured digital solution that simplifies food ordering, improves kitchen order management, and enables customers to track their orders efficiently.

1.2 Problem Statement

In today's digital era, restaurants are increasingly adopting online ordering systems, yet many restaurants still depend on manual communication between customers, waiters, and kitchen staff. Traditional ordering methods often involve handwritten orders or verbal communication, which can result in incorrect orders, delays, difficulty in tracking order progress, and communication gaps between the restaurant staff and customers. The core problem addressed by this project is the lack of an integrated, user-friendly restaurant ordering and kitchen management system that connects customers, kitchen staff, and administrators through a single platform.

In many traditional systems, customers have limited visibility into the status of their orders and kitchen staff may have to depend on manual order slips or verbal instructions. Another major issue is the difficulty of managing orders efficiently during busy restaurant hours. When multiple customers place orders simultaneously, manually recording and communicating each order can increase the possibility of errors and delays. Kitchen staff need a centralized system where they can clearly view incoming orders, ordered food items, quantities, and special instructions. Customers also need a convenient way to browse the available menu, select food items, manage their cart, place orders, and choose their table. Without a digital ordering system, these activities require direct interaction with restaurant staff, which can increase waiting time and reduce the efficiency of the ordering process. Order tracking is another important challenge.

In a traditional restaurant environment, customers may not know whether their order has been accepted, is being prepared, or is ready to be served. The absence of a proper order-status mechanism reduces visibility and can lead to unnecessary communication between customers and staff. From the restaurant management perspective, administrators require an organized way to manage menu items, restaurant tables, users, and orders. Maintaining these details manually can become difficult as the number of customers and orders increases. A centralized database can help store and retrieve this information efficiently.

There is also a need for proper communication between the frontend, backend, and database. Without an integrated architecture, customer orders, menu information, table information, and order statuses cannot be managed effectively. Therefore, the system needs a reliable backend and database that can process requests and maintain consistent information. The problem also extends to user access and security. Different users should have access to different functionalities. Customers should be able to manage their orders, kitchen staff should manage order statuses, and administrators should manage restaurant operations. Therefore, authentication and role-based authorization are important parts of the system. The proposed project aims to solve these challenges by developing a Smart Restaurant Ordering and Kitchen Display System. The system provides a single digital platform where customers can browse the menu, add items to the cart, place orders, select tables, and track order status. At the same time, kitchen staff can view and process incoming orders through the Kitchen Display System, while administrators can manage restaurant information through the Admin Dashboard. By addressing these challenges, the project aims to achieve the following problem resolutions:

- 1.Reduce manual ordering by providing a digital food ordering process.
- 2.Improve kitchen communication through a centralized Kitchen Display System.
- 3.Reduce order errors and delays by digitally transferring orders to the kitchen.
- 4.Enable order tracking through different stages such as New, Accepted, Preparing, Ready, and Served.
- 5.Simplify restaurant management through centralized menu, table, user, and order management.

6. Provide role-based access for customers, kitchen staff, and administrators.

7. Maintain centralized data using a MySQL database.

In summary, the problem statement identifies the gap between the need for an efficient digital restaurant ordering system and the limitations of traditional manual ordering processes. The proposed solution addresses these issues by integrating customer ordering, kitchen order management, table selection, order tracking, and administration into one unified web-based platform. This helps improve order visibility, reduce manual errors, and make restaurant operations more organized and efficient.

1.3 Scope of the Project

The scope of this project, “Smart Restaurant Ordering and Kitchen Display System,” covers the development of a web-based platform that integrates customer food ordering, kitchen order management, table selection, order tracking, and administrative management into a single system. The main objective is to provide a simple and efficient digital solution that reduces manual work and improves communication between customers, kitchen staff, and administrators. The technical scope involves developing a responsive web application using React.js, HTML, CSS, and JavaScript for the frontend. The backend is developed using Node.js and Express.js, which handles application requests and provides REST APIs for communication between the frontend and database. MySQL is used for storing and managing user details, menu items, restaurant tables, orders, and order items. The functional scope of the Customer Module includes user registration and login, browsing the available menu, viewing food details, adding items to the cart, modifying cart quantities, placing orders, selecting a restaurant table, and viewing order history and status. This provides customers with a complete digital ordering experience without depending entirely on manual communication with restaurant staff.

The Kitchen Display System forms an important part of the project. Kitchen staff can view incoming customer orders along with the required food items, quantities, customer information, table details, and instructions. Orders can be processed through different stages such as New, Accepted, Preparing, Ready, and Served. This provides better visibility of active orders and helps kitchen staff organize their work. The Admin Module provides centralized management of important restaurant operations. Administrators can manage menu items, restaurant tables, users, and orders through the Admin Dashboard. This helps maintain restaurant information in a structured manner and provides better control over the system. The project also includes authentication and role-based access. Different users are provided with access to appropriate modules based on their roles, such as Customer, Kitchen Staff, and Admin. This helps separate customer activities from kitchen and administrative operations. From a database perspective, the scope includes maintaining structured information using MySQL. The major entities include Users, Menu Items, Restaurant Tables, Orders, and Order Items. Relationships between these entities allow order information and related details to be stored and retrieved efficiently. The project also focuses on user interface and usability. The interfaces are designed to be simple and easy to navigate so that customers can browse food items and place orders with minimal effort. Separate interfaces are provided for customers, kitchen staff, and administrators according to their requirements. From an operational perspective, the system aims to reduce manual order handling, minimize communication gaps, improve order visibility, and provide centralized restaurant management. The system can be expanded in the future to support additional restaurant services and features. The current scope does not include features such as online payment gateway integration, mobile application development, AI-based food recommendations, or real-time push notifications. These features can be considered as future enhancements. To summarize, the scope of the project includes the following key points:

1. Development of a unified web-based restaurant ordering platform.
2. Digital menu browsing, cart management, and food ordering.
3. Table selection and order management.
4. Kitchen Display System for processing customer orders.
5. Order status tracking from New to Served.
6. Admin management of menu, tables, users, and orders.

- 7.MySQL-based centralized data management.
- 8.Role-based authentication for customers, kitchen staff, and administrators.
- 9.REST API-based communication between frontend and backend.
- 10.A scalable structure that can support additional features in the future.

In conclusion, the scope of this project extends beyond a basic food-ordering website. It provides an integrated system connecting customers, kitchen staff, and administrators while improving the overall restaurant ordering and management process. The system focuses on convenience, efficient communication, centralized data management, and better visibility of order progress.

1.4 Objectives of the Project

The main objective of this project is to design and develop a Smart Restaurant Ordering and Kitchen Display System that simplifies the food ordering process and improves coordination between customers, kitchen staff, and administrators through a unified web-based platform. In traditional restaurant environments, customers often depend on waiters for menu information and order placement, while kitchen staff receive orders through handwritten notes or verbal communication. This project aims to reduce these manual processes by providing a digital platform where customers can browse the menu, place orders, select tables, and track their order status. The primary technical objective is to develop a web-based system using React.js, Node.js, Express.js, REST APIs, and MySQL.

The system should provide smooth communication between the frontend, backend, and database while maintaining structured information about users, menu items, tables, and orders. Another important objective is to provide a simple and user-friendly interface. Customers should be able to easily navigate through the menu, select food items, manage their cart, place orders, and view their order status. The kitchen interface should clearly display incoming orders and provide the required information for efficient order preparation.

The Kitchen Display System (KDS) is a major objective of this project. It is designed to provide kitchen staff with a centralized view of customer orders. Staff can process orders through different stages such as New, Accepted, Preparing, Ready, and Served. This improves order visibility and helps organize kitchen operations. The project also aims to provide an Admin Dashboard for centralized restaurant management. Administrators should be able to manage menu items, restaurant tables, users, and orders from a single interface. This reduces the need for separate manual records and makes restaurant information easier to manage. From a database perspective, the objective is to maintain reliable and organized data using MySQL. The system stores information related to users, menu items, tables, orders, and order items and establishes relationships between these entities for efficient data retrieval and management. Security and controlled access are also important objectives. The system implements authentication and role-based access so that customers, kitchen staff, and administrators can access only the functionalities relevant to their roles. The project also aims to develop a modular and extendable architecture.

The system can be expanded in the future with features such as online payments, real-time notifications, mobile application support, and AI-based food recommendations without completely redesigning the existing system. From an educational and technical perspective, the project demonstrates the practical application of web development, database management, REST APIs, authentication, user interface design, and software engineering principles to solve a real-world restaurant management problem.

- 1.To summarize, the major objectives of the project are:
- 2.To design and develop a unified digital restaurant ordering system.
- 3.To provide customers with digital menu browsing, cart, ordering, and table selection.
- 4.To develop a Kitchen Display System for efficient order processing.
- 5.To enable order tracking through New, Accepted, Preparing, Ready, and Served stages.
- 6.To provide an Admin Dashboard for managing menu, tables, users, and orders.
- 7.To maintain centralized restaurant data using MySQL.
- 8.To implement authentication and role-based access.
- 9.To reduce manual errors and improve communication between customers and kitchen staff.
- 10.To develop a responsive and user-friendly interface.

11.To demonstrate the practical application of modern web development and database technologies. In essence, the objective of this project goes beyond simply creating an online food-ordering website. It aims to create an integrated restaurant management ecosystem that connects customers, kitchen staff, and administrators while improving ordering

1.5 Importance of the Application

The restaurant industry is increasingly adopting digital technologies to improve customer service and operational efficiency. However, many restaurants still depend on manual ordering and communication between customers, waiters, and kitchen staff. This can result in order errors, delays, difficulty in tracking orders, and inefficient management. Hence, the importance of this application lies in its ability to bring the major restaurant operations into one unified digital platform. The proposed Smart Restaurant Ordering and Kitchen Display System provides a complete digital solution for customers, kitchen staff, and administrators. By integrating menu browsing, cart management, food ordering, table selection, kitchen order processing, and order tracking, the application reduces the need for manual communication and provides a more organized restaurant workflow. One of the major benefits of the application is improved customer convenience. Customers can browse the available menu, select food items, add them to their cart, place orders, select a table, and track their order status through the application. This reduces waiting time and allows customers to have better visibility of their orders.

Another important aspect is the Kitchen Display System (KDS). Kitchen staff can view incoming orders in a centralized interface along with food items, quantities, table details, and instructions. Orders can be processed through different stages such as New, Accepted, Preparing, Ready, and Served. This improves communication between the ordering system and kitchen and helps staff manage multiple orders more effectively. The application is also important from a restaurant management perspective. The Admin Dashboard provides centralized management of menu items, tables, users, and orders. Instead of maintaining separate manual records, administrators can manage important restaurant information through a single system. From a technological perspective, the project demonstrates the practical use of modern web technologies. React.js is used for the frontend, Node.js and Express.js are used for backend development and REST APIs, and MySQL is used for database management. These technologies allow the application to maintain structured data and provide communication between different modules. The application also provides importance in terms of data organization and accessibility. User information, menu details, table information, and order records are stored centrally in the database.

This makes it easier to retrieve and manage information when required. Security and controlled access are also important aspects of the application. Different roles such as Customer, Kitchen Staff, and Admin have access to different functionalities. Authentication and role-based access help maintain separation between different types of users. From an operational perspective, the system helps reduce manual work and communication gaps. Digital order transmission can reduce the possibility of mistakes caused by handwritten or verbally communicated orders. The centralized KDS also provides better visibility of active orders and their current status.

The project also has academic and technical importance because it demonstrates the practical application of web development, database management, REST APIs, authentication, user interface design, and software engineering principles in solving a real-world problem. To summarize, the importance of this application can be understood through the following key points: It provides a unified platform for restaurant ordering and management. It improves customer convenience through digital menu and ordering. It reduces manual communication and potential order errors. It improves kitchen efficiency through the Kitchen Display System. It provides real-time visibility of order progress.

It simplifies management of menu items, tables, users, and orders. It provides centralized data storage using MySQL. It supports role-based access for different users. It demonstrates the practical application of modern web

technologies. It provides a foundation for future features such as online payments, notifications, and mobile applications.

In conclusion, the importance of this application extends beyond simply providing online food ordering. It creates an integrated digital environment that connects customers, kitchen staff, and administrators, improving communication, order visibility, and overall restaurant management efficiency.

1.6 Target Users / Audience

The target users of the Smart Restaurant Ordering and Kitchen Display System include customers, kitchen staff, and restaurant administrators. The application is designed to serve different users involved in the restaurant ordering and management process by providing them with specific functionalities according to their roles. The primary users are restaurant customers who want a simple and convenient way to browse the menu and place food orders. Customers can register and log in to the system, view available food items, add items to their cart, place orders, select a restaurant table, and track their order status. The secondary users are kitchen staff who are responsible for preparing customer orders. Through the Kitchen Display System, kitchen staff can view incoming orders, check ordered food items and quantities, view table information and instructions, and update the order status as it moves through different preparation stages. Restaurant administrators are another important group of users. Administrators can use the Admin Dashboard to manage menu items, restaurant tables, users, and orders. This provides centralized control over important restaurant operations. The system can also be useful for restaurant owners and managers who need better visibility into orders and restaurant activities. The centralized system helps them maintain organized records and manage daily operations more efficiently. In essence, the application caters to:

1. Customers – For browsing menus, placing orders, selecting tables, and tracking orders.
2. Kitchen Staff – For viewing and processing incoming orders through the KDS.
3. Administrators – For managing users, menu items, tables, and orders.
4. Restaurant Owners/Managers – For monitoring and managing overall restaurant operations.

Overall, the target audience represents the main participants in the restaurant ordering process. The system connects these users through a single digital platform, helping improve communication, order visibility, and operational efficiency.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Hardware requirements form the foundation of the software development process. They define the physical resources necessary to design, develop, test, and deploy the Smart Restaurant Ordering and Kitchen Display System efficiently. Since this project is a full-stack web-based application integrating multiple modules such as Customer Ordering, Kitchen Display System, Table Management, Menu Management, Order Management, and Admin Dashboard, the hardware should support multitasking, database operations, web development, and network communication. The hardware requirements are divided into two categories: those required by developers during development and testing, and those required by end users to access the deployed application. The development environment requires a moderately configured computer because developers need to run multiple applications such as Visual Studio Code, React development server, Node.js, Express.js, MySQL server, web browsers, and API testing tools such as Postman simultaneously. The processor should ideally be an Intel Core i5 or higher or an equivalent AMD Ryzen 5, with a minimum clock speed of around 2.4 GHz.

A multi-core processor helps in handling frontend compilation, backend execution, database operations, and multiple development processes simultaneously. A minimum of 8 GB RAM is recommended for smooth development, while 16 GB RAM provides better performance when running the frontend, backend, MySQL server, browser, and development tools at the same time. Since the project involves installing Node.js packages, React dependencies, database files, and project repositories, an SSD is preferred over a traditional hard disk because of its faster read and write speeds. A minimum storage capacity of 250 GB is sufficient for the development environment, with additional storage or cloud backup recommended for maintaining project files and versions. A stable internet connection is required for downloading dependencies, accessing documentation, synchronizing project files with GitHub, testing APIs, and installing required packages.

A minimum internet speed of around 10 Mbps is recommended for comfortable development and testing. Reliable connectivity is also useful when multiple components of the web application are being tested together. The operating system can be Windows 10/11, Ubuntu Linux, or macOS, as the technologies used in the project are cross-platform. An updated web browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge is required for testing the React frontend, user interface, responsiveness, and application functionality. For UI development and testing, a Full HD monitor with a resolution of 1920×1080 pixels or higher is recommended. This provides sufficient screen space for working with the code editor, browser, database tools, and application interface. Standard peripherals such as a keyboard, mouse, and headphones are also useful during development and communication. The server-side hardware depends on the number of users and expected workload. For development and internal testing, a system with a quad-core processor, 8–16 GB RAM, and SSD storage is sufficient to run the Node.js/Express backend and MySQL database. For larger real-world deployments, cloud-based infrastructure can be used to provide better scalability and availability.

From the end user's perspective, the hardware requirements are minimal because the application is accessed through a web browser. Customers can access the system using a desktop, laptop, tablet, or smartphone. A basic device with a dual-core processor, 4 GB RAM, and a modern browser is sufficient for accessing the menu, cart, ordering, table selection, and order tracking features.

The client-side storage requirement is also minimal because the application mainly operates through the browser. The system does not require users to install large software packages on their devices. A stable internet connection is required for communicating with the backend server and retrieving updated menu, table, and order information.

The Kitchen Display System can be accessed using a computer, laptop, or suitable display device with a modern web browser. Since kitchen staff need to continuously view incoming orders and update their status, a device with at least 4 GB RAM and a stable network connection is recommended. Similarly, administrators can access the Admin Dashboard using a standard computer or laptop.

In summary, the hardware requirements for the Smart Restaurant Ordering and Kitchen Display System are designed to support efficient development, testing, deployment, and daily usage. Developers require moderately powerful systems for running React, Node.js, Express.js, MySQL, and development tools simultaneously, while customers, kitchen staff, and administrators can access the system using standard computing devices with a modern web browser and stable internet connection.

2.2 Software Requirements

Software requirements define the digital environment necessary to design, develop, test, and deploy the Smart Restaurant Ordering and Kitchen Display System efficiently. Since the system is developed as a full-stack web application, it requires software technologies that support responsive user interfaces, backend processing, database management, API communication, authentication, and order management. A clear understanding of the software stack helps ensure smooth development, testing, and future maintenance of the application. The operating system is a fundamental component of the development environment. The project can be developed and executed on Windows 10/11, Ubuntu Linux, or macOS. Windows provides a user-friendly environment for development using tools such as Visual Studio Code, Node.js, MySQL, and web browsers, while Linux can also be used for server-side deployment because of its stability and flexibility. The web browser is required to access and test the frontend application. Modern browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge can be used to run the React-based interface. Browser developer tools are also useful for checking page responsiveness, debugging JavaScript errors, and monitoring API requests. At the frontend, the system is developed using React.js, HTML5, CSS3, and JavaScript. React.js is used to create reusable and interactive user interface components. HTML provides the basic structure of the application, CSS is used for styling and layout, and JavaScript provides dynamic functionality. The frontend includes customer pages such as the menu, cart, checkout, orders, and order tracking, along with kitchen and admin interfaces. The backend of the system is developed using Node.js and Express.js. Node.js provides the runtime environment for executing server-side JavaScript, while Express.js is used to create the backend server and RESTful API routes. The backend handles important operations such as user authentication, menu management, order processing, table management, and communication between the frontend and database. The database management system used for the project is MySQL.

MySQL stores and manages structured information required by the application. The database contains tables such as Users, Menu Items, Restaurant Tables, Orders, and Order Items. It provides centralized storage and allows the backend to perform operations such as inserting, updating, retrieving, and deleting application data. The system uses REST APIs to establish communication between the React frontend, Node.js/Express backend, and MySQL database. Data is exchanged mainly in JSON format, providing a simple and standardized method of communication between different application modules. APIs are used for operations such as login, menu retrieval, order creation,

table management, and updating kitchen order status. For development and coding, Visual Studio Code (VS Code) is used as the primary Integrated Development Environment (IDE). It provides features such as syntax highlighting, code completion, debugging, terminal access, and Git integration. The development environment also uses the Node Package Manager (npm) for installing and managing required React and Node.js packages. For version control and collaboration, Git and GitHub are used to maintain project files and track changes during development. Git allows developers to manage different versions of the source code, while GitHub can be used to store and share the project repository. Postman is used for API testing. It helps test backend endpoints by sending HTTP requests such as GET, POST, PUT, and DELETE and checking the returned responses. This helps verify the communication between the frontend and backend before integrating the complete application. For database management, MySQL tools can be used to create databases, tables, relationships, and SQL queries. The MySQL environment allows developers to insert, update, retrieve, and maintain restaurant data during development and testing. The system also requires software support for authentication and authorization.

User authentication allows customers, kitchen staff, and administrators to log in to the system, while role-based authorization restricts access to the appropriate modules. Passwords are stored securely using bcrypt hashing in the backend. The application follows a modular architecture in which the Customer, Kitchen, and Admin modules operate as separate functional sections while communicating through the same backend APIs and database. This structure makes the application easier to maintain, debug, and extend with additional features in the future. The deployment environment can consist of a Node.js server for the backend, MySQL for database storage, and a web server or cloud platform for hosting the frontend and backend. The exact deployment platform can be selected according to the requirements and scale of the application. In conclusion, the software requirements of the Smart Restaurant Ordering and Kitchen Display System provide the necessary environment for developing a reliable full-stack web application. By combining React.js, Node.js, Express.js, MySQL, REST APIs, Git, GitHub, VS Code, and Postman, the system supports digital ordering, kitchen management, table management, authentication, and centralized data handling while providing a foundation for future enhancements.

2.3 Development Tools and Frameworks

Developing the Smart Restaurant Ordering and Kitchen Display System requires a combination of development tools and frameworks to support frontend development, backend processing, database management, API communication, testing, version control, and deployment. These tools provide the necessary structure for developing an efficient, maintainable, and user-friendly restaurant management platform. The following tools are used throughout different stages of the development process. At the front-end layer, the system is developed using React.js, HTML5, CSS3, and JavaScript. React.js is used to build reusable and interactive components for different sections of the application. The Customer Module includes components such as the menu, cart, checkout, orders, table selection, and order tracking. React also helps in managing changes in the user interface when data such as cart items or order status is updated. HTML5 provides the basic structure of the web pages, while CSS3 is used for styling, layouts, spacing, colors, buttons, cards, and responsive design. CSS techniques such as Flexbox, Grid, and media queries help the application adapt to different screen sizes including desktops, laptops, tablets, and smartphones. JavaScript (ES6+) is used to implement the interactive functionality of the application. It handles user interactions, form validation, cart operations, API communication, and dynamic updates. React Hooks such as `useState` and `useEffect` can be used to manage component state and application behavior. The backend development is implemented using Node.js and Express.js. Node.js provides the runtime environment for executing JavaScript on the server side, while Express.js provides a framework for creating REST API endpoints.

The backend handles important operations such as user authentication, menu management, order creation, table management, and communication with the MySQL database. For database management, MySQL is used as the relational database management system. It stores structured information related to users, menu items, restaurant tables, orders, and order items. MySQL allows the application to perform operations such as creating, retrieving, updating, and deleting records. Database management tools can also be used to execute SQL queries and inspect the stored data during development. The system uses RESTful APIs to establish communication between the React frontend and Node.js/Express backend. API endpoints are used for operations such as user login, retrieving menu items, creating orders, retrieving customer orders, managing tables, and updating kitchen order statuses. JSON is used as the primary data exchange format between the frontend and backend. During development, Visual Studio Code (VS Code) is used as the primary Integrated Development Environment (IDE). It provides features such as syntax highlighting, code completion, debugging, integrated terminal support, and Git integration. It allows developers to work with React, JavaScript, Node.js, Express.js, and other project files within a single environment. For package management, npm (Node Package Manager) is used to install and manage the dependencies required by the frontend and backend. It simplifies the installation, updating, and maintenance of libraries and frameworks used in the project. For version control and collaboration, Git and GitHub are used. Git helps developers track changes made to the source code and maintain different versions of the project. GitHub provides an online repository for storing the project, maintaining backups, and supporting collaboration between team members.

The testing phase uses Postman for testing REST API endpoints. It allows developers to send requests such as GET, POST, PUT, and DELETE and examine the responses returned by the backend. This helps verify authentication, menu operations, order processing, table management, and other API functionalities before integrating them with the frontend. For authentication and security, the system uses password hashing techniques such as `bcrypt` to protect user passwords.

Authentication and role-based authorization are used to distinguish between different users such as customers, kitchen staff, and administrators. This ensures that users can access only the modules appropriate to their roles. The Kitchen Display System (KDS) is implemented as part of the web application using React.js and backend APIs. It provides kitchen staff with incoming order information, including food items, quantities, table details, and instructions.

The order status can be updated through the sequence New → Accepted → Preparing → Ready → Served, allowing the progress of an order to be monitored. The Admin Dashboard is also developed using React.js and communicates with the backend through REST APIs. It provides administrative functionality for managing menu items, tables, users, and orders. This centralized interface helps administrators monitor and manage restaurant operations. For deployment, the frontend, backend, and database can be hosted using suitable web and cloud infrastructure depending on the requirements of the final implementation. The React frontend requires a web hosting environment, while the Node.js/Express backend requires a server environment and MySQL requires a database server. In summary, the combination of React.js, HTML5, CSS3, JavaScript, Node.js, Express.js, MySQL, REST APIs, VS Code, npm, Git, GitHub, and Postman provides a complete development environment for the Smart Restaurant Ordering and Kitchen Display System. Each tool contributes to a specific stage of development, from creating the user interface and processing orders to managing databases, testing APIs, maintaining source code, and supporting future deployment.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design

The front-end of the Smart Restaurant Ordering and Kitchen Display System represents the user interface layer through which customers, kitchen staff, and administrators interact with the application. It is the visual and interactive part of the system and is responsible for providing a simple, responsive, and user-friendly experience. The front-end is developed using React.js, HTML5, CSS3, and JavaScript, allowing the application to work efficiently across different devices and screen sizes.

The main purpose of the front-end design is to allow customers to easily perform restaurant-related activities such as browsing the menu, selecting food items, adding items to the cart, placing orders, selecting a table, and tracking order status. The interface also provides dedicated screens for kitchen staff and administrators. The design focuses on simple navigation, clear information presentation, consistent styling, and easy interaction so that users can operate the system without technical difficulty.

HTML5 provides the basic structure of the application. It is used to organize elements such as navigation bars, menu sections, food cards, forms, buttons, cart details, order information, and dashboard sections. Semantic HTML elements help maintain a structured and readable interface.

CSS3 is used for the visual presentation of the application. It controls layouts, colors, fonts, spacing, buttons, cards, tables, and other interface elements. CSS techniques such as Flexbox, Grid, and media queries are used to create responsive layouts that adapt to desktops, laptops, tablets, and smartphones. Hover effects, transitions, and appropriate spacing are also used to improve the overall user experience.

JavaScript (ES6+) provides the interactive functionality of the frontend. It handles user actions such as adding food items to the cart, changing quantities, removing items, submitting forms, and displaying updated order information. JavaScript also enables communication with backend APIs without requiring the entire webpage to reload.

The project uses React.js to create a component-based frontend architecture. The application is divided into reusable components and pages such as the Navbar, Menu, Cart, Checkout, Orders, Kitchen Dashboard, Admin Dashboard, and Table Management. This approach makes the application easier to develop, maintain, and update because individual components can be modified without affecting the entire interface.

React Router is used to manage navigation between different pages of the application. It allows users to move between pages such as the Home page, Menu, Login, Register, Cart, Checkout, Orders, Kitchen Dashboard, and Admin Dashboard without requiring traditional page navigation. Different routes are also used to provide access to appropriate modules based on the user's role.

The frontend communicates with the backend through REST APIs. API requests are used to retrieve menu information, authenticate users, create orders, manage tables, retrieve customer orders, and update kitchen order statuses. Data is exchanged between the frontend and backend mainly in JSON format. This separation between frontend and backend makes the application more modular and easier to maintain.

The Customer Module provides the main ordering interface. Customers can browse available food items, view prices and descriptions, select quantities, add items to the cart, and proceed to checkout. After placing an order, customers can select a restaurant table and view the progress of their order.

The Kitchen Display System (KDS) provides a dedicated interface for kitchen staff. Orders received from customers are displayed along with relevant details such as food items, quantities, customer information, table number, and special instructions.

Kitchen staff can update orders through the sequence New → Accepted → Preparing → Ready → Served.

The Admin Module provides an administrative interface for managing restaurant operations. The Admin Dashboard displays information related to users, menu items, orders, active orders, revenue, and tables.

Additional interfaces allow administrators to manage menu items and restaurant tables. The frontend also implements form validation to prevent incomplete or invalid information from being submitted. Required fields, input validation, and appropriate error messages help improve reliability and user experience. Authentication and role-based access are incorporated into the frontend to provide different interfaces for customers, kitchen staff, and administrators. After login, users are directed to the appropriate module according to their assigned role. This prevents unauthorized access to restricted sections of the application. Responsive design is an important part of the frontend implementation. The interface is designed to maintain usability across different screen sizes. This allows customers to access the restaurant system using laptops, tablets, or smartphones, while kitchen and administrative users can use larger displays for better visibility. Performance and usability are also considered during frontend development. Reusable React components, organized CSS, efficient API communication, and appropriate state management help maintain a responsive application. The interface is designed to keep important information such as food items, cart contents, order details, and order status clearly visible to the user. In conclusion, the front-end design of the Smart Restaurant Ordering and Kitchen Display System combines React.js, HTML5, CSS3, and JavaScript to provide a responsive and interactive interface. The component-based architecture makes the application modular and maintainable, while separate Customer, Kitchen, and Admin interfaces ensure that each user group can perform its required activities efficiently.

3.2 Back-End Design

3.2 Back-End Design The back-end of the Smart Restaurant Ordering and Kitchen Display System acts as the main processing layer of the application. It manages user requests, business logic, database operations, authentication, order processing, and communication between the front-end and database. The back-end is developed using Node.js and Express.js, providing a structured and efficient environment for handling multiple requests from customers, kitchen staff, and administrators. Node.js is used as the primary server-side runtime environment. Its event-driven and asynchronous architecture allows the server to handle multiple requests efficiently. This is useful for a restaurant system where several customers may browse the menu, place orders, or check their order status while kitchen staff and administrators are simultaneously accessing the system. Express.js is used as the backend framework for developing the RESTful APIs. It provides a simple structure for creating API routes and handling HTTP requests. The APIs act as a communication layer between the React frontend and the MySQL database. Operations such as user login, menu retrieval, order creation, table management, and order-status updates are handled through these APIs. The back-end follows a modular structure in which different functionalities are organized into separate routes, controllers, models, and configuration files. The main modules include User Management, Menu Management, Order Management, Table Management, Kitchen Management, and Admin Management. This structure makes the application easier to maintain, debug, and extend.

When a request is received from the frontend, it is directed to the appropriate Express API endpoint. The backend validates the request, performs the required processing, communicates with the MySQL database when necessary, and returns the result to the frontend in JSON format. For example, when a customer places an order, the backend receives the selected food items, customer information, table information, and order details before storing the information in the database. The MySQL database is the main data storage layer of the backend. The backend communicates with MySQL to store and retrieve information related to users, menu items, restaurant tables, orders, and order items. Relationships between these tables help maintain data consistency and allow information to be retrieved efficiently. The User Management module handles registration and login operations. When a user registers, the backend validates the provided information and stores the account details in the database. Passwords are protected using bcrypt hashing instead of storing them directly as plain text. During login, the backend verifies the entered credentials and returns the appropriate user information and role.

The system implements role-based authorization for different types of users. The main roles are Customer, Kitchen Staff, and Admin. Customers can access customer-related functions, kitchen staff can access the Kitchen Display System, and administrators can access administrative features. This prevents users from accessing modules that are not intended for their roles. The Menu Management module handles operations related to restaurant food items. The backend provides APIs to retrieve menu items and allows authorized administrators to add, update, or delete menu information. Menu records contain information such as food name, category, price, description, rating, and other relevant details. The Order Management module handles the complete order-processing process. When a customer confirms an order, the backend creates an order record and stores the associated food items in the order items table. The backend calculates and stores the order information and maintains its current status. The Table Management module manages restaurant table information. It provides information about available tables and allows table details to be associated with customer orders. This supports the requirement that customers select their table during the ordering process.

The Kitchen Display System module retrieves active orders from the backend and displays them to kitchen staff. It provides information such as the order number, customer details, table number, food items, quantities, and special instructions. Kitchen staff can update the order status through the defined sequence: New → Accepted → Preparing → Ready → Served. Whenever the kitchen updates an order status, the backend stores the updated status in the MySQL database. The customer-facing frontend can then retrieve the latest status and display the progress of the order. The Admin module provides APIs for administrative operations. Administrators can manage users, menu items, tables, and orders. The backend also provides dashboard information such as the number of users, menu items, orders, active orders, revenue, and tables, which can be displayed through the Admin Dashboard. The backend uses REST API communication between the frontend and server. Standard HTTP methods such as GET, POST, PUT, and DELETE are used according to the operation being performed. JSON is used for request and response data, providing lightweight communication between the React application and Node.js server. Error handling and input validation are also important parts of the backend design. The server validates incoming data before performing database operations and returns suitable error responses when invalid information is provided. This helps prevent incorrect data from being stored and allows the frontend to display meaningful messages to users. The backend configuration contains the database connection and other required settings. Sensitive configuration information can be maintained using environment variables rather than directly placing it in application source code. This improves the security and maintainability of the application.

The backend can also be tested independently using Postman. API endpoints for login, menu management, orders,

tables, kitchen operations, and administration can be tested by sending different requests and checking the returned JSON responses. This helps identify API and database-related problems before complete frontend integration.

The communication architecture can be represented as: React Frontend → REST API → Node.js + Express.js → MySQL Database. The back-end design is therefore responsible for connecting all major modules of the Smart Restaurant Ordering and Kitchen Display System. It manages authentication, menu data, customer orders, table information, kitchen operations, and administrative functions through a centralized server and database. In conclusion, the back-end provides the core processing and data-management layer of the Smart Restaurant Ordering and Kitchen Display System. The combination of Node.js, Express.js, REST APIs, MySQL, bcrypt, and role-based authorization provides a structured foundation for handling restaurant operations. Its modular design makes the system easier to maintain and provides a foundation for future enhancements such as online payments, real-time notifications, mobile applications, and AI-based recommendations.

3.3 Database Design

The database design of the Smart Restaurant Ordering and Kitchen Display System forms the foundation upon which all the system's dynamic features and operations are built. It serves as the central repository where user information, menu details, restaurant table information, order details, and ordered food items are securely stored and efficiently managed. The database is designed to provide data consistency, reliability, and efficient access, ensuring that every operation performed by a customer, kitchen staff member, or administrator is captured accurately and processed correctly. For this project, MySQL is used as the primary relational database management system (RDBMS), providing a structured and reliable environment for storing and managing restaurant-related information. MySQL is suitable for this project because the system mainly handles structured data with clearly defined relationships between different entities. It uses predefined schemas and relationships that help maintain data consistency through the use of primary keys, foreign keys, constraints, and relational operations. This structured approach is particularly suitable for storing transactional information such as customer orders, order items, table assignments, and user accounts. The database also supports reliable transaction processing, ensuring that important operations are completed correctly without causing inconsistent or incomplete records.

The database is organized into several logical modules—Users, Menu Items, Restaurant Tables, Orders, and Order Items. Each module is carefully designed to store specific information required by the application while reducing unnecessary data duplication. For instance, the Users table contains attributes such as User ID, Name, Email, Password, Role, and account creation information, ensuring that each user's identity and access level are clearly defined. The Menu Items table stores details such as Menu Item ID, Name, Category, Price, Rating, Description, Vegetarian status, Popular status, and image information. Similarly, the Restaurant Tables table contains information such as Table ID, Table Number, Seating Capacity, and Table Status.

The Orders table acts as one of the most important tables in the system because it stores the details of customer orders. It contains attributes such as Order ID, Order Number, User ID, Table ID, Customer Name, Customer Phone, Total Amount, Payment Method, Order Status, Instructions, and Order Creation Time. The Order Items table stores the individual food items included in each order. It contains references to the Order ID and Menu Item ID along with the quantity and price of each item. This structure allows one order to contain multiple food items while avoiding the need to repeatedly store complete menu information.

The relationship between the major database entities can be represented through primary and foreign keys. The Users table is connected to the Orders table through the User ID. The Restaurant Tables table is connected to the Orders table through the Table ID. The Orders table is connected to the Order Items table through the Order ID, while the Menu Items table is connected to Order Items through the Menu Item ID. These relationships ensure that the system can retrieve complete information about a customer's order, the food items included in it, and the table associated with the order.

To improve performance and ensure efficient data access, indexes can be created on frequently searched fields such as user email, order number, order status, and table information. MySQL queries and relational operations allow the system to retrieve information from multiple tables efficiently. For example, a query can combine the Orders and Order Items tables with Menu Items to retrieve the complete list of food items belonging to a particular customer order. Similarly, customer-specific order information can be retrieved using the User ID.

Data security is treated as an important part of the database design. User passwords stored in MySQL are protected using bcrypt hashing, ensuring that plain-text passwords are not directly stored in the database. Database access is controlled through the backend server, preventing the frontend from directly accessing MySQL. Role-based permissions are also implemented at the application level to ensure that customers, kitchen staff, and administrators can perform only the operations associated with their respective roles. For example, menu and table management operations are restricted to authorized administrative users. The database plays an important role in the Kitchen Display System (KDS). When a customer places an order, the order information and corresponding order items are stored in the database. Kitchen staff can retrieve active orders from the backend and view details such as customer name, table number, food items, quantities, and special instructions.

The order status is maintained in the Orders table and follows the defined sequence of New → Accepted → Preparing → Ready → Served. Whenever kitchen staff update the order status, the new status is stored in the database and can subsequently be retrieved by the customer interface for order tracking. The database also supports the Admin Dashboard, where administrators can monitor and manage important restaurant information. The backend retrieves information from the database to display details such as the total number of users, menu items, orders, active orders, revenue, and restaurant tables. Administrators can also perform CRUD operations on menu items and manage restaurant tables through the application. To maintain data consistency, the database design follows basic normalization principles. Information is divided into separate tables according to its purpose instead of storing all information in a single table. For example, menu item information is maintained separately from order information, while the Order Items table connects individual food items with their corresponding orders. This reduces redundancy and makes updating and maintaining the database easier.

The database communicates with the back-end layer through Node.js and Express.js. REST API endpoints are used to perform database operations such as retrieving menu items, registering users, creating orders, retrieving customer orders, managing tables, and updating order statuses. The backend processes requests received from the React frontend and communicates with MySQL to retrieve or modify the required information. The resulting data is then returned to the frontend in JSON format. For database development and maintenance, MySQL tools and SQL queries are used to create tables, define relationships, insert records, update information, and retrieve data. Developers can also perform database testing by executing different SQL queries and verifying whether the expected results are returned. Regular database backups can be maintained to reduce the risk of data loss and ensure that important restaurant information can be recovered when required. The database design also supports the complete order-processing workflow of the system. When a customer selects food items and places an order, the order information is inserted into the Orders table and the selected food items are stored in the Order Items table. The associated table information is maintained through the Restaurant Tables table.

The kitchen then retrieves the order and updates its status as preparation progresses. Finally, the customer can view the updated order status through the frontend. In terms of architecture, the database works as the central data storage layer of the application. The overall communication can be represented as React Frontend → Node.js + Express Backend → MySQL Database. The frontend interacts only with the backend APIs, while the backend is responsible for validating requests, applying business logic, and performing database operations. This separation improves security, maintainability, and organization of the application.

In summary, the database design of the Smart Restaurant Ordering and Kitchen Display System provides a structured, secure, and reliable architecture for managing restaurant information. By using MySQL for structured transactional data, the system maintains relationships between users, menu items, restaurant tables, orders, and order

items. The use of primary keys, foreign keys, normalization, indexing, password hashing, and backend-controlled database access ensures data integrity and security. Overall, the database acts as the central backbone of the application, supporting customer ordering, kitchen order processing, table selection, order tracking, and administrative management while providing a strong foundation for future enhancements.

3.4 Version Control

Version control plays a crucial role in the development and maintenance of the Smart Restaurant Ordering and Kitchen Display System, serving as the backbone of the project's collaborative workflow and ensuring that every stage of development from initial design to testing and deployment is tracked, synchronized, and recoverable. The system uses Git as the primary version control system and GitHub as the cloud-based repository hosting service to facilitate distributed development, efficient code management, collaboration, and backup. Version control is essential for a multi-module project like this one, where different components such as Customer Ordering, Kitchen Display System, Menu Management, Table Management, and Admin Dashboard are developed simultaneously. By implementing Git, each team member can work independently on their assigned modules while still being able to merge their changes safely into the main codebase without overwriting or conflicting with other work. Git's distributed nature ensures that every team member can maintain a local copy of the repository along with its commit history, allowing developers to track changes and recover previous versions when required. The workflow begins with the creation of a central remote repository on GitHub, which serves as the main repository for the project. Developers clone the repository into their local machines and can create separate branches to isolate their work. For example, a developer working on the customer interface can create a branch such as `feature/customer-ui`, while another developer working on the Kitchen Display System can work on a branch such as `feature/kitchen-dashboard`. This branching approach allows multiple modules to be developed simultaneously while keeping the main branch stable. Each new feature, modification, or bug fix can be developed in its respective branch and tested before being merged into the main branch. Pull requests can be used to review changes before merging them into the official project repository. This process allows team members to examine the code, identify possible problems, and provide feedback before changes become part of the main application. It also promotes better coordination and code quality among the project members.

To maintain consistency and make the project history easier to understand, the team can follow standardized commit messages such as "feat:" for new features, "fix:" for bug fixes, and "refactor:" for code restructuring. Each commit represents a snapshot of the project at a particular point in time. This allows developers to identify when a particular change was introduced and revert to an earlier stable version if an unexpected problem occurs. GitHub provides a graphical interface for managing the project repository, allowing developers to view commit history, branches, files, and changes. The repository can also be used to manage project tasks and issues. GitHub Issues can be used to report bugs, record development tasks, or suggest improvements. Tasks can be assigned to individual team members so that responsibilities are clearly distributed among the development team. The GitHub Projects feature can also be used to organize development activities through task boards. Tasks can be categorized as pending, in progress, or completed.

For example, tasks related to frontend development, backend API implementation, database development, KDS implementation, and testing can be organized into separate activities. This provides a clear overview of project progress and helps the team manage development activities systematically. For continuous collaboration, GitHub

can also support automated workflows through GitHub Actions. Whenever new code is pushed to the repository, automated checks or tests can be executed to verify that the changes do not introduce problems into the existing application. Such workflows can help improve the reliability of the development process and reduce manual testing effort. Another important aspect of version control in this project is documentation management.

Project documentation, database diagrams, architecture diagrams, API information, installation instructions, and other supporting documents can be maintained within the project repository. This ensures that important technical information is stored together with the source code and can be updated as the project develops. Branch protection can also be applied to important branches such as the main branch. This helps prevent accidental changes or direct modifications to the stable version of the project. Pull requests and code reviews can be used before important changes are merged into the main branch, providing an additional level of control over the source code. For data security, sensitive information such as database passwords, API keys, and environment variables should not be uploaded to the GitHub repository. These files can be excluded using the gitignore file. Environment configuration files such as .env can be maintained locally while ensuring that confidential information is not accidentally committed to the repository. Version tags can also be used to identify important stages of the project. For example, releases such as v1.0 can represent a stable version of the Smart Restaurant Ordering and Kitchen Display System. Tags make it easier to identify and access specific versions of the application during evaluation, demonstration, or future development. Backup and redundancy are additional benefits of using Git and GitHub. Since the project repository is stored remotely and can also exist as local copies on the developers' computers, the risk of losing the complete source code is reduced. Previous commits provide additional recovery points if files are accidentally modified or deleted. From an academic and software engineering perspective, the use of version control demonstrates the team's adoption of modern development practices. It supports collaboration, code tracking, project organization, backup, testing, and documentation.

It also provides a clear record of the project's development, showing how the Customer Module, Kitchen Display System, Admin Module, database, and other components were developed and improved over time. In the context of the Smart Restaurant Ordering and Kitchen Display System, Git and GitHub help coordinate the development of the frontend, backend, database, customer module, kitchen module, and administrative features. The repository acts as a central location where the complete project source code and supporting files can be maintained and accessed by authorized team members. In conclusion, Git and GitHub provide a reliable and collaborative environment for version control in the Smart Restaurant Ordering and Kitchen Display System. They ensure that project changes are systematically organized, securely stored, and easily traceable. The use of branches, commits, pull requests, issues, and repository management helps reduce development conflicts and improves project organization. By integrating version control throughout the development lifecycle, the project achieves better collaboration, maintainability, backup, and systematic management of the complete restaurant management application.

3.5 APIs or External Services

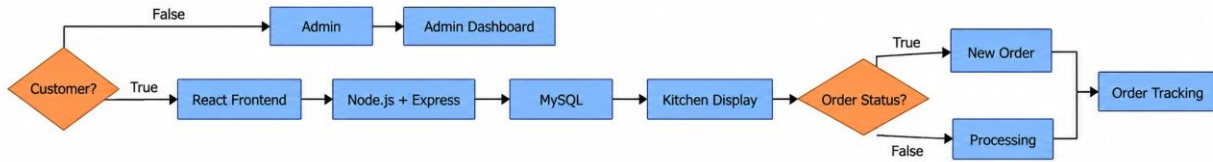
APIs (Application Programming Interfaces) and external services play an important role in the Smart Restaurant Ordering and Kitchen Display System, enabling communication between the front-end, back-end, and database layers. In this project, APIs act as bridges that allow the React.js frontend to communicate with the Node.js and Express.js backend and retrieve or modify information stored in the MySQL database. The use of APIs makes the application dynamic, interactive, and capable of processing customer orders and managing restaurant operations efficiently. The API architecture of the system follows RESTful principles, providing standardized endpoints for different operations such as user authentication, menu management, order processing, table management, kitchen operations, and administrative functions. These APIs use HTTP methods such as GET for retrieving information, POST for creating new records, PUT for updating existing records, and DELETE for removing records. The data exchanged between the frontend and backend is primarily formatted using JSON (JavaScript Object Notation) because of its lightweight structure and compatibility with JavaScript and React.js applications. The core of the system consists of internal REST APIs developed using Node.js and Express.js. These APIs provide communication between the different modules of the application. For example, user-related endpoints handle registration and login, menu endpoints retrieve and manage food items, order endpoints create and retrieve customer orders, table endpoints provide table information, and kitchen endpoints retrieve active orders and update their status. Admin APIs provide access to administrative information such as users, menu items, orders, tables, and dashboard statistics. The User API manages customer, kitchen, and administrator information. It handles operations such as user registration, login, retrieving user information, and role-based access. During login, the backend verifies the user's credentials and returns the appropriate user information and role to the frontend. This allows the application to provide different interfaces according to whether the logged-in user is a customer, kitchen staff member, or administrator. The Menu API manages the restaurant's food items. The frontend uses API requests to retrieve available menu items and display them to customers. Authorized administrators can use the API to add new food items, update existing items, or remove items from the menu. The API communicates with the MySQL database to perform the required operations and returns the results in JSON format. The Order API is responsible for handling customer orders. When a customer adds food items to the cart and confirms the order, the frontend sends the order information to the backend through a REST API. The backend validates the information and stores the order and its corresponding order items in the MySQL database. The API can also retrieve previous and active orders for customers. The Table API manages information related to restaurant tables. It allows the frontend to retrieve available tables and provides the required table information during the ordering process. The selected table can then be associated with the customer's order through the backend and database. One of the most important API integrations in the project is the Kitchen Display System API. Kitchen staff can retrieve active orders through the kitchen API and view details such as customer name, table number, food items, quantities, and special instructions. Kitchen staff can also update the order status through the API. The order follows the sequence New → Accepted → Preparing → Ready → Served, and each status update is stored in the database. The Admin API provides data required by the Admin Dashboard. It retrieves information such as the number of users, menu items, total orders, active orders, revenue, and restaurant tables. It also supports administrative operations related to menu items, tables, users, and orders. This allows the administrator to manage important

restaurant information through a centralized interface. The integration between the frontend and backend follows a simple communication architecture: React.js Frontend → REST API → Node.js + Express.js → MySQL Database. When a user performs an action on the frontend, such as placing an order, the React application sends an HTTP request to the appropriate backend endpoint. Express.js receives and processes the request, performs the required database operation, and returns a JSON response. The frontend then uses this response to update the user interface. Security is an important part of API communication. Authentication is used to verify user identity, while role-based authorization restricts access to specific API operations. Passwords are protected using bcrypt hashing before being stored in the database. Sensitive configuration information such as database credentials can be maintained using environment variables rather than being directly included in the source code. Postman is used as an external development and testing service for validating the REST APIs. Developers can use Postman to send GET, POST, PUT, and DELETE requests to the backend and verify the returned responses. API testing helps identify problems in authentication, menu operations, order processing, table management, kitchen operations, and administrative functions before complete frontend integration. The application does not currently depend on external flight, hotel, payment, mapping, or third-party restaurant APIs because the core restaurant functionality is handled by the project's own Node.js/Express backend and MySQL database. The system therefore focuses mainly on internal REST API communication between its frontend, backend, and database layers. For performance and reliability, the API layer handles requests asynchronously through Node.js. This allows the server to process multiple requests from customers, kitchen staff, and administrators without unnecessarily blocking other operations. Proper error handling is also implemented so that invalid requests, unavailable resources, or database errors can be returned to the frontend with appropriate responses. The API structure is designed to be modular so that additional external services can be integrated in the future. For example, online payment services, real-time notifications, mobile applications, or AI-based food recommendation services can be connected through additional APIs without significantly changing the existing architecture. From a development perspective, APIs provide a clear separation between the frontend and backend. The React application does not directly communicate with the MySQL database. Instead, all database operations are performed through the Node.js and Express.js API layer. This improves organization, security, maintainability, and the possibility of extending the application to other client platforms in the future. In terms of scalability, the modular API architecture allows new endpoints and services to be added as the restaurant system grows. For example, future versions can introduce APIs for online payments, real-time kitchen notifications, customer feedback, loyalty programs, delivery management, or mobile applications. These services can communicate with the existing backend without requiring major changes to the core database structure. From an academic and practical perspective, the use of APIs demonstrates the application of modern web development principles. It shows how different layers of a full-stack application communicate through standardized interfaces. In the context of the Smart Restaurant Ordering and Kitchen Display System, REST APIs provide the connectivity required between customers, kitchen staff, administrators, the backend server, and the database. In conclusion, APIs form an important part of the connectivity and functionality of the Smart Restaurant Ordering and Kitchen Display System. Through RESTful APIs developed using Node.js and Express.js, the system provides seamless communication between the React frontend and MySQL database.

The APIs support authentication, menu management, table management, order processing, kitchen operations, and administrative functions. Postman further supports API testing and validation, while the modular architecture provides a foundation for future integration with online payments, real-time notifications, mobile applications, and other external services.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram



4.2 Description of Each Layer (Frontend, Backend, Database)

The architecture of the Smart Restaurant Ordering and Kitchen Display System follows a well-defined layered approach that separates the user interface, business logic, and data management components. This structured layering enhances modularity, scalability, and maintainability, ensuring that each component performs its specific responsibility while maintaining smooth communication across the system. The three primary layers of the system are the Frontend, Backend, and Database layers. The first layer, the Frontend, is the visual and interactive face of the system, developed using React.js, HTML5, CSS3, and JavaScript. This layer provides the user interface through which customers interact with the restaurant system to browse food items, add items to the cart, place orders, select tables, and track order status. React.js enables a component-based structure that supports reusable and maintainable interface elements. The frontend contains different pages and components such as the Menu, Cart, Checkout, Orders, Login, Register, Kitchen Dashboard, Admin Dashboard, and Table Management.

The interface is designed to be responsive so that it can be accessed through desktops, laptops, tablets, and smartphones. The Customer Module in the frontend allows customers to register and log in, browse available food items, view food details and prices, add items to their cart, modify quantities, place orders, select a restaurant table, and view the progress of their orders. React state management is used to handle dynamic information such as cart contents, user information, and application interactions. React Router is used to navigate between different pages without requiring complete page reloads. The frontend communicates with the backend through REST APIs and receives data in JSON format.

The Backend layer is the central processing layer of the system where business logic, request processing, authentication, and database communication are handled. It is developed using Node.js and Express.js and acts as an intermediary between the React frontend and MySQL database. The backend provides RESTful API endpoints for different operations such as user authentication, menu management, order processing, table management, kitchen operations, and administration. The backend is organized into different functional modules such as User Management, Menu Management, Order Management, Table Management, Kitchen Management, and Admin Management. Each request received from the frontend is routed to the appropriate backend endpoint. The backend

validates the request, performs the required operation, communicates with the database, and returns the result to the frontend in JSON format. For example, when a customer places an order, the backend receives the selected food items and customer information, processes the request, stores the order in the database, and returns the order information to the frontend. The backend also manages authentication and authorization. When users log in, their credentials are verified against the stored information in the database. Passwords are protected using bcrypt hashing. Role-based authorization is used to distinguish between customers, kitchen staff, and administrators, ensuring that each user can access the appropriate functionality. The backend also handles validation and error responses to prevent invalid data from being stored in the system. The Kitchen Display System is connected to the backend layer through REST APIs. The backend retrieves active orders from the database and provides them to the kitchen interface.

Kitchen staff can view information such as customer name, table number, ordered food items, quantities, and instructions. They can update the order status according to its progress: New → Accepted → Preparing → Ready → Served. These status changes are stored in the database and can be retrieved by the customer interface for order tracking. The Admin Module also communicates with the backend through REST APIs. The backend provides the information required by the Admin Dashboard, including users, menu items, orders, active orders, revenue, and restaurant tables. Administrators can perform management operations such as adding, updating, and deleting menu items and managing restaurant table information. The Database layer forms the foundation of the system by storing, managing, and retrieving all information required for the application's operation. The project uses MySQL as the relational database management system.

The database contains structured tables such as Users, Menu Items, Restaurant Tables, Orders, and Order Items. These tables are connected using primary keys and foreign keys to maintain relationships and data integrity. The Users table stores customer, kitchen staff, and administrator information. The Menu Items table stores food-related information such as name, category, price, rating, description, and other menu details. The Restaurant Tables table stores table numbers, seating capacity, and table status. The Orders table stores customer order information and order status, while the Order Items table stores the individual food items and quantities associated with each order. The database follows relational principles to reduce data duplication and maintain consistency. Primary keys uniquely identify records, while foreign keys establish relationships between tables. For example, the Orders table is connected to the Users table through the User ID and to the Restaurant Tables table through the Table ID. The Order Items table connects orders with menu items, allowing the system to retrieve complete order information efficiently. The interaction between these three layers follows a systematic data flow process. When a customer performs an action through the frontend, such as placing an order, the request is sent to the backend through a REST API.

The backend processes the request and communicates with the MySQL database to store or retrieve the required information. The database returns the result to the backend, which sends a JSON response back to the frontend. The frontend then updates the user interface according to the received information. The overall data flow can be represented as: Frontend → REST API → Backend → MySQL Database → Backend → Frontend. This layered architecture provides separation of concerns, meaning that each layer has a specific responsibility. The frontend handles user interaction and presentation, the backend handles business logic and API communication, and the database handles data storage and retrieval.

Therefore, changes to one layer can be made with minimal impact on the other layers. For example, the frontend design can be modified without changing the database structure, while database operations can be updated without directly changing the user interface. Security and performance are also considered across the different layers.

The frontend performs input validation and provides controlled access to different pages. The backend implements authentication, authorization, password hashing, request validation, and error handling. The database maintains structured relationships and protects stored user information through secure password storage. This combined approach improves the reliability and security of the complete system. In conclusion, the integration of the Frontend, Backend, and Database layers forms a cohesive and organized architecture for the Smart Restaurant Ordering and Kitchen Display System. The frontend provides an interactive interface for customers, kitchen staff, and administrators; the backend handles application logic, authentication, APIs, and processing; and the MySQL database provides reliable data storage and retrieval. This layered architecture simplifies development and debugging while providing a strong foundation for future enhancements such as online payments, real-time notifications, mobile applications, and AI-based food recommendations.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

The deployment architecture of the Smart Restaurant Ordering and Kitchen Display System is designed to provide a reliable environment for developing, testing, and running the application. The system follows a local deployment approach during development, where the React frontend, Node.js and Express.js backend, and MySQL database are executed on the development machine. This approach allows the development team to build, test, and debug all modules efficiently before moving the application to a production environment. During development, the React.js frontend runs on a local development server, while the Node.js and Express.js backend runs on a separate local server. The MySQL database is hosted locally and stores information related to users, menu items, restaurant tables, orders, and order items. The frontend communicates with the backend through REST APIs, while the backend communicates with MySQL for storing and retrieving application data. This local architecture provides a controlled environment for testing the complete application before deployment.

The local deployment architecture can be represented as: React Frontend → Node.js + Express Backend → MySQL Database. The frontend can be accessed through a local browser using the development server, while the backend provides API endpoints for authentication, menu management, order processing, table management, kitchen operations, and administration. The MySQL server runs locally and provides persistent storage for the application. This setup is suitable for project development, demonstrations, debugging, and academic evaluation. For version control and project collaboration, Git and GitHub are used to maintain the source code. The project repository contains the frontend and backend source code along with configuration files and project documentation. Developers can commit changes, create branches, and push updates to GitHub. This provides a backup of the project and allows team members to coordinate their development work efficiently. The deployment process can also be extended to a cloud environment when the system is ready for real-world use. The React frontend can be deployed using cloud-based hosting services, while the Node.js and Express.js backend can be hosted on a cloud server.

The MySQL database can similarly be hosted using a managed cloud database service. Cloud deployment allows the system to be accessed remotely by customers, kitchen staff, and administrators instead of being restricted to a local machine. The project can use services such as Vercel, Netlify, Render, AWS, or other suitable cloud platforms depending on deployment requirements. The frontend and backend can be deployed separately while maintaining communication through the configured REST API URL. The database remains connected to the backend so that all users access the same centralized restaurant information. The deployment pipeline follows the Continuous Integration and Continuous Deployment (CI/CD) approach. GitHub can be integrated with CI/CD services such as GitHub Actions to automate important development tasks.

Whenever developers push updated code to the repository, the pipeline can automatically install dependencies, perform basic checks, build the application, and verify that the project is ready for deployment. The basic CI/CD workflow can be represented as: Developer → Git → GitHub → Build & Test → Deployment. CI/CD reduces the need for repeated manual deployment steps and helps maintain consistency between different versions of the application. It also makes it easier to identify errors before updated code is deployed to the production environment. If the project is expanded in the future, automated testing can be added to verify frontend components, backend APIs, authentication, database operations, and order processing. Environment configuration is also an important part of the deployment architecture.

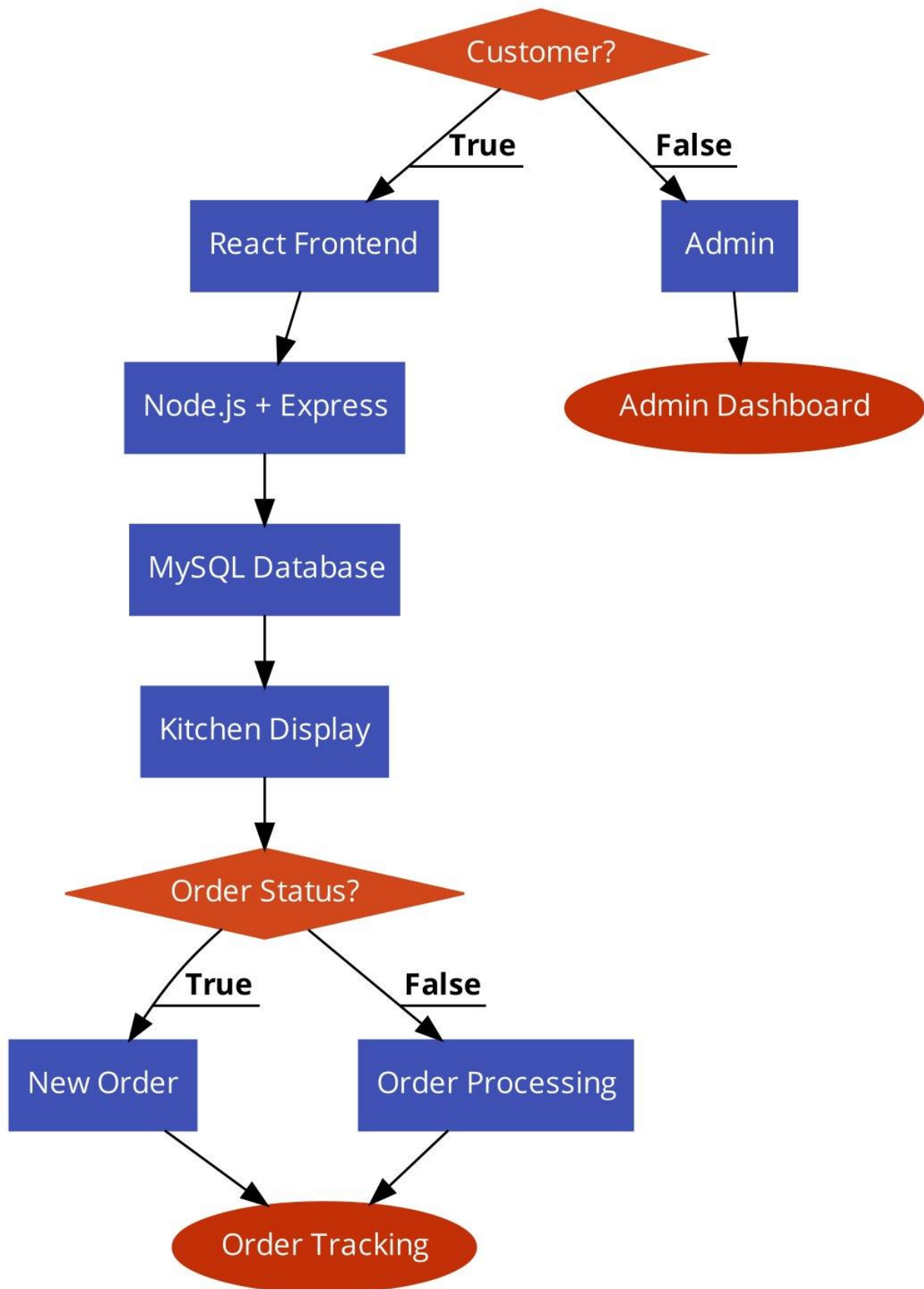
Configuration values such as the MySQL database connection, backend port, API URL, and other sensitive settings should be maintained using environment variables instead of directly placing them in the source code. The .env file can be used during local development, while secure environment-variable management can be provided by the selected cloud hosting platform during production deployment. For database deployment, the MySQL database can initially run on the local development machine. For production deployment, it can be moved to a managed MySQL hosting service or a cloud database server. Regular database backups can be maintained to protect important information such as user accounts, menu items, restaurant tables, orders, and order items. Database access should be restricted to the backend application rather than being directly exposed to end users. Security is an important consideration in the deployment architecture. Communication between the frontend and backend in production should use HTTPS to protect data transmitted over the network. Authentication and role-based authorization ensure that customers, kitchen staff, and administrators can access only their respective functionalities.

Database credentials and other sensitive configuration values should not be stored directly in the GitHub repository and should be excluded using the .gitignore file.

Monitoring and maintenance can also be introduced when the application is deployed to a cloud environment. Server logs can be used to identify backend errors, API failures, and database connection problems. Cloud hosting platforms generally provide basic application logs and deployment information, which can help developers identify and resolve issues. Additional monitoring tools can be integrated in the future if the system handles a larger number of users. Scalability is another advantage of cloud deployment. During periods of high restaurant activity, the backend server can be provided with additional computing resources when required. The database can also be upgraded according to the amount of stored data and number of concurrent users. This allows the application to grow from an academic project into a larger restaurant management platform without completely changing its architecture. The deployment process concludes with testing and verification after each major update. The frontend is checked for correct page navigation and user interactions, while the backend APIs are tested using Postman. Database operations are verified to ensure that users, menu items, tables, orders, and order items are stored and retrieved correctly. The Kitchen Display System is also tested to confirm that order status updates are reflected correctly.

In the context of the Smart Restaurant Ordering and Kitchen Display System, the deployment architecture provides a clear path from local development to cloud-based production. The local environment supports development and testing, GitHub provides version control and collaboration, and CI/CD pipelines can automate the build and deployment process. The architecture can therefore support the current academic implementation while providing a foundation for future real-world deployment. In conclusion, the deployment architecture combines local development, GitHub-based version control, cloud deployment, and CI/CD practices to provide a structured and maintainable environment for the Smart Restaurant Ordering and Kitchen Display System. The use of React.js, Node.js, Express.js, and MySQL allows the system to be developed and tested locally, while cloud services can be introduced when production deployment is required. This approach reduces deployment errors, improves project maintainability, and provides a foundation for future expansion of the restaurant system.

CHAPTER -5 DESIGN



CHAPTER -6 IMPLEMENTATION

Section 6.1 – Module-Wise Implementation

The implementation of the Smart Restaurant Ordering and Kitchen Display System was structured into interconnected modules that together form a complete digital restaurant management platform. Each module was designed with a specific functional responsibility while maintaining smooth communication through the shared backend APIs and MySQL database. This modular structure improves scalability, maintainability, and usability by separating customer ordering, kitchen operations, table management, and administrative functions. The implementation focuses on reducing manual ordering processes and providing a unified platform for customers, kitchen staff, and administrators. The first major module is the User Authentication and Account Management Module, which manages customer, kitchen staff, and administrator accounts. The module supports user registration and login while maintaining different roles within the system. Customer accounts provide access to menu browsing, cart management, ordering, table selection, and order tracking. Kitchen and administrator accounts are provided with access to their respective dashboards. Passwords are securely stored using hashing, and role-based access ensures that users can access only the features relevant to their responsibilities. The Menu Management Module is responsible for displaying and managing the restaurant's available food items. Customers can browse food items according to categories and view information such as food name, price, rating, description, and availability. The module provides an organized interface that allows customers to select food items easily. Administrators can add new menu items, update existing information, and remove items when required.

The menu information is stored in the MySQL database and retrieved through REST APIs, ensuring that the displayed information remains synchronized with the backend. The Cart and Checkout Module handles the process of preparing a customer's order before final submission. Customers can add multiple food items to their cart, increase or decrease quantities, and remove items when required. The system calculates the total order amount based on the selected items and quantities. During checkout, the customer can review the selected food items and provide the required order information before confirming the order. This module provides a simple and organized process that reduces errors during manual ordering. The Order Management Module forms one of the most important parts of the system. When a customer confirms an order, the order information is sent from the React frontend to the Node.js and Express.js backend through REST APIs. The backend stores the order information in the MySQL database along with the corresponding order items. Each order contains information such as the customer, selected items, quantities, total amount, table information, and current order status. This centralized order structure allows the customer, kitchen staff, and administrator to access the required order information.

The Table Management Module manages the restaurant's table information. The system maintains information about available restaurant tables, table numbers, seating capacity, and table status. Customers can select a suitable table during the ordering process. The selected table is associated with the customer's order and stored in the database. Administrators can manage table information through the Admin Module. This digital approach simplifies table allocation and provides better visibility of restaurant seating information. The Kitchen Display System (KDS) Module is the central operational module for kitchen staff.

It provides a dedicated interface where kitchen staff can view incoming customer orders without depending on handwritten notes or verbal communication. Each order displays important information such as customer name,

table number, food items, quantities, and special instructions. Kitchen staff can update the order according to its preparation progress using the sequence New → Accepted → Preparing → Ready → Served.

These status changes are stored in the backend and database, allowing the order information to remain synchronized throughout the system. The Order Tracking Module allows customers to monitor the progress of their orders after they have been placed. The module retrieves the latest order status from the backend and displays it to the customer. The status sequence provides clear information about whether the order is new, accepted, being prepared, ready, or served. This feature improves order visibility and reduces uncertainty for customers waiting for their food. The Admin Management Module provides centralized administrative control over the restaurant system. Administrators can access the Admin Dashboard to monitor important information such as users, menu items, orders, active orders, revenue, and restaurant tables. The module also provides management operations for menu items and tables. By bringing important restaurant information into a single dashboard, the module helps administrators monitor and manage daily operations more efficiently. The Database Management Module provides centralized storage for all information used by the application.

The system uses MySQL with tables such as Users, Menu Items, Restaurant Tables, Orders, and Order Items. Relationships between these tables are maintained using primary keys and foreign keys. The database receives requests from the backend and returns the required information to the application. This centralized data management ensures consistency between the customer, kitchen, and admin modules. The REST API Module provides communication between the frontend, backend, and database. The APIs are developed using Node.js and Express.js and handle operations such as authentication, menu retrieval, order creation, table management, kitchen order processing, and administrative operations. The APIs use standard HTTP methods such as GET, POST, PUT, and DELETE, while JSON is used for exchanging information between the frontend and backend. This modular API structure makes communication organized and allows additional functionality to be introduced in the future. The Admin Dashboard and Monitoring Module combines important restaurant information into a centralized interface. It provides summarized information about total orders, total revenue, active orders, customers, menu items, and tables. Recent orders can also be monitored from the dashboard.

This module gives administrators a better overview of restaurant activity and helps them manage the system from a single location. The implementation of the system also focuses on role-based functionality. Customers, kitchen staff, and administrators use different interfaces according to their responsibilities. Customers interact primarily with the menu, cart, checkout, orders, and tracking features. Kitchen staff work with the Kitchen Display System, while administrators access management and monitoring functions. This separation improves usability and prevents unauthorized access to management features. Collectively, the module-wise implementation of the Smart Restaurant Ordering and Kitchen Display System ensures that every component contributes to the overall objective of creating an efficient digital restaurant environment.

Each module is designed as part of a unified structure rather than as an isolated feature. The modular organization makes the system easier to develop, test, maintain, and extend. Future enhancements such as online payment integration, real-time notifications, mobile applications, customer feedback, loyalty programs, and AI-based food recommendations can be added without requiring major changes to the existing architecture.

Through these interconnected modules, the system transforms the traditional restaurant ordering process into a centralized digital workflow connecting customers, kitchen staff, and administrators. The implementation therefore provides improved order management, better communication, centralized data handling, and clear order-status visibility while maintaining a structured and scalable architecture.

Section 6.2 – Front-End Logic (UI Rendering and State Management)

The front-end of the Smart Restaurant Ordering and Kitchen Display System plays an important role in shaping the user experience of the application. Since customers, kitchen staff, and administrators interact with the system through the frontend, the main objective of this layer is to provide a simple, interactive, responsive, and easy-to-use interface. The frontend logic converts the different restaurant operations such as menu browsing, cart management, food ordering, table selection, order tracking, kitchen management, and administration into a unified digital experience. The implementation therefore focuses on maintaining a balance between visual presentation, functional accuracy, and smooth interaction. The foundation of the front-end design is built on principles of usability and responsiveness, ensuring that the interface can be accessed comfortably through desktops, laptops, tablets, and smartphones. The application provides different interfaces according to the user's role. Customers are provided with menu, cart, checkout, orders, and tracking features, while kitchen staff are provided with the Kitchen Display System and administrators are provided with management dashboards. The layout is organized to keep important information visible and reduce unnecessary navigation. Buttons, menus, cards, forms, and other interface elements are arranged logically so that users can complete their tasks without confusion.

Central to the front-end operation is its component-based architecture, implemented using React.js. The user interface is divided into reusable components such as navigation bars, food cards, menu sections, cart components, order cards, tables, dashboards, and status displays. Each component performs a specific function and can be maintained independently. This modular structure improves consistency throughout the application and makes it easier to update or extend individual sections without affecting the entire interface. React's component-based approach also allows the application to update only the required parts of the interface when the underlying data changes. Another crucial element of the front-end logic is state management, which controls how data is maintained and updated within the application. The system manages dynamic information such as logged-in user details, food items, cart contents, selected table information, order information, and order status.

When a customer adds or removes a food item from the cart, the corresponding state is updated and the interface immediately reflects the change. Similarly, when an order status changes, the order information displayed to the user can be updated based on the latest data received from the backend. This synchronization helps maintain consistency between user actions and application responses. The navigation system further improves the usability of the application. React Router is used to navigate between different pages and modules such as Home, Menu, Login, Register, Cart, Checkout, Orders, Kitchen Dashboard, Admin Dashboard, Admin Menu, and Tables. The navigation structure is designed according to the responsibilities of each user role. Customers can access customer-related pages, while kitchen staff and administrators are directed to their respective dashboards.

This organized navigation allows users to move between different sections without unnecessary page reloads and provides a smooth application experience. The front-end also relies on dynamic data binding to display information received from the backend. For example, menu information retrieved through REST APIs is displayed as food cards containing details such as food name, price, rating, category, description, and image information. Similarly,

customer orders are retrieved from the backend and displayed in the Orders section. The Kitchen Display System dynamically displays incoming orders along with customer details, table information, ordered items, quantities, and instructions. This ensures that the interface reflects the information stored in the system. An important part of the frontend is the menu and cart interaction. Customers can browse different food categories and select the items they want to order. When an item is added to the cart, the cart state is updated and the selected item becomes visible in the cart. Customers can change quantities or remove items before proceeding to checkout. The total amount is recalculated according to the selected items and quantities. This dynamic interaction provides immediate feedback and simplifies the ordering process compared with traditional manual ordering.

The checkout interface provides customers with a final review of their selected items before an order is submitted. The frontend displays the order summary and required information before sending the order request to the backend. After successful submission, the customer can access the order information through the Orders section. This flow helps prevent accidental or incomplete orders and provides a clear transition from cart selection to order confirmation. The Kitchen Display System interface uses front-end logic to present active restaurant orders in an organized manner. Each order is displayed as a separate order card containing relevant information such as customer name, table number, food items, quantities, and special instructions.

Kitchen staff can interact with the available controls to update the order status. The interface visually separates different order states such as New, Accepted, Preparing, Ready, and Served, making it easier for kitchen staff to understand the current workload and progress of each order. The Admin Dashboard also depends on front-end rendering and state management to display restaurant information. Data received from the backend is used to show statistics such as total users, menu items, orders, active orders, revenue, and tables. Recent orders and management information can also be displayed through tables and cards. The frontend updates these elements whenever fresh information is retrieved from the backend, allowing administrators to monitor restaurant operations through a centralized interface. Another important consideration in front-end logic is form validation and error handling. Forms such as registration, login, menu management, and order-related inputs are validated before information is submitted to the backend.

Required fields are checked to prevent incomplete submissions, while appropriate messages are displayed when invalid information is entered. API errors, login failures, and unsuccessful requests are also handled by displaying meaningful feedback to users rather than leaving the interface in an unclear state. These mechanisms improve reliability and user confidence. The frontend communicates with the backend using REST APIs and asynchronous requests. When a customer requests menu information, places an order, or retrieves order history, the React application sends the appropriate request to the Node.js and Express.js backend. The backend processes the request and returns data in JSON format. The frontend then uses the received response to update the relevant components. This separation allows the interface to remain independent from the database while maintaining continuous communication with the backend. Performance optimization also forms an important part of the front-end implementation.

The React component structure avoids unnecessary rendering by updating the parts of the interface that require changes. Images and other resources can be optimized to reduce loading time, while browser storage can be used where appropriate for temporary client-side information such as cart data. The responsive layout ensures that the application remains usable across different screen sizes without requiring separate interfaces for every device. The front-end also implements role-based navigation and access control. After successful login, the user's role is used to determine the appropriate interface and available routes. Customers are directed towards ordering and tracking

features, kitchen staff are provided with the Kitchen Display System, and administrators can access management dashboards. This separation improves the overall structure of the application and prevents users from unnecessarily accessing unrelated modules. The front-end logic also maintains synchronization between the customer, kitchen, and admin modules. When a customer places an order, the information is sent to the backend and becomes available to the kitchen interface.

When kitchen staff update the order status, the updated information can be retrieved by the customer interface. Similarly, administrators can access order and system information through the Admin Dashboard. This communication creates a connected workflow between the different users of the restaurant system. From an architectural perspective, the communication between the frontend and backend is managed through clearly defined REST API endpoints. Each user action triggers an appropriate asynchronous request without requiring direct communication between the frontend and MySQL database. For example, when a customer places an order, the frontend sends the order information to the backend, which stores it in the database and returns the result. This approach keeps the frontend organized and separates presentation logic from business and database operations. The final aspect of the front-end logic concerns security and controlled user interaction.

The frontend avoids directly exposing database credentials or backend configuration information. User inputs are validated before being submitted, while protected routes restrict access to specific modules based on the user's role. Login and registration interfaces are also designed to provide clear feedback during authentication. These measures, together with backend authentication and authorization, contribute to the overall security of the application. In conclusion, the front-end logic of the Smart Restaurant Ordering and Kitchen Display System combines React.js components, state management, responsive UI rendering, REST API communication, navigation, validation, and role-based access to create a smooth digital restaurant experience. The frontend provides customers with an easy ordering process, kitchen staff with an organized Kitchen Display System, and administrators with a centralized management interface. Its modular and responsive structure also provides a strong foundation for future enhancements such as real-time notifications, mobile applications, online payments, and AI-based food recommendations.

6.3 API Integration

The API Integration stage constitutes the backbone of communication within the Smart Restaurant Ordering and Kitchen Display System, connecting the front-end with the backend and database layers of the architecture. Through Application Programming Interfaces, the React front-end interacts seamlessly with the Node.js and Express.js backend, enabling data retrieval, order creation, updates, and synchronization between the different modules of the application. The design follows RESTful architectural principles, which provide structured communication, clear endpoint organization, and efficient data exchange across the Customer, Kitchen, and Admin modules. At its core, the restaurant system relies mainly on internal REST APIs developed using Node.js and Express.js. These APIs manage functions such as user registration, authentication, menu management, table management, order creation, order retrieval, kitchen order processing, and administrative operations. Each functional area is organized through dedicated routes and controllers. For instance, endpoints under `/api/users` handle registration and login, `/api/menu` manages food items, `/api/orders` manages customer orders, `/api/tables` handles restaurant table information, `/api/kitchen` manages kitchen orders and status updates, and `/api/admin` provides administrative operations. The communication between the React front-end and the backend is performed using HTTP requests and JSON data. When a customer opens the menu, the front-end sends a request to the menu API, which retrieves the required food information from the MySQL database.

Similarly, when a customer places an order, the front-end sends the order details to the backend through a POST request. The backend processes the request, stores the order and order-item information in MySQL, and returns a structured JSON response to the frontend. This allows the interface to update dynamically without requiring direct access to the database. The Menu API is responsible for transferring restaurant food information between the frontend and backend. It provides details such as food name, category, price, rating, description, availability-related information, and image information. Customers use this information while browsing the menu and selecting food items. Administrators can also use the API to add, update, or remove menu items through the Admin Menu interface. This ensures that menu information is centrally maintained in the database. The Order API is one of the most important API components of the system.

It manages the complete order creation and retrieval process. When the customer confirms the cart, the order information is sent to the backend along with the selected food items, quantities, customer information, table information, and total amount. The backend stores the order in the orders table and the individual food selections in the order_items table. The API can then be used by the customer interface, kitchen interface, and admin dashboard to retrieve the required order information. The Kitchen API connects the Kitchen Display System with the backend. When a new customer order is created, it becomes available to the kitchen through the kitchen API. Kitchen staff can retrieve active orders along with customer details, table number, food items, quantities, and instructions. The kitchen staff can then update the order status through the API.

The status follows the sequence New → Accepted → Preparing → Ready → Served. These changes are stored in the database and can subsequently be displayed to the customer through the order tracking interface. The Table API is responsible for managing restaurant table information. It retrieves available tables and their relevant information for the customer interface and administrative dashboard. When a customer selects a table during checkout, the selected table information is associated with the order through the backend.

Administrators can also manage table information through the corresponding API. This creates a connection

between table management and order processing. The User API handles user registration, login, and account-related operations. When a user provides login credentials, the frontend sends the information to the backend through the login endpoint. The backend verifies the credentials and returns the appropriate user information and role. The role determines whether the user accesses the Customer, Kitchen, or Admin interface. Passwords are stored securely using password hashing rather than storing plain-text passwords. The Admin API provides communication between the Admin Dashboard and the backend. It retrieves information such as total users, menu items, orders, active orders, revenue, and restaurant tables. It also supports administrative management operations for menu items and tables.

This allows administrators to access centralized information without directly interacting with the MySQL database. The API communication follows standard HTTP methods such as GET, POST, PUT, and DELETE. GET requests are mainly used for retrieving menu items, orders, tables, and dashboard information. POST requests are used for operations such as registration, login, and creating orders. PUT requests can be used for updating menu, table, or order information, while DELETE requests can be used when removing supported records. JSON is used as the primary data exchange format because it is lightweight and easily handled by JavaScript and React applications. Security and reliability are important aspects of API integration. Authentication and role-based authorization are implemented to restrict access to protected modules. Customer, kitchen, and admin users are provided with different levels of access according to their roles. Backend validation is used to check incoming data before it is processed or stored. CORS configuration allows communication between the React frontend and the Node.js backend running on different local development ports. Sensitive configuration information such as database credentials is maintained separately from the frontend code.

The API layer also includes error handling to manage unsuccessful requests. If an API request contains invalid information or the requested operation cannot be completed, the backend returns an appropriate response to the frontend. The frontend then displays a suitable message to the user. This approach prevents the application from failing silently and provides better feedback during operations such as login, menu management, order creation, or data retrieval. All important APIs are tested using Postman during development. API testing is used to verify whether requests are reaching the correct endpoints and whether the backend returns the expected responses. Different operations such as user login, menu retrieval, order creation, order retrieval, table retrieval, and kitchen status updates can be tested independently. Testing also helps identify problems in request parameters, response structures, database operations, and backend routes before integrating them with the frontend. The API architecture also supports automatic data synchronization between different modules. For example, when a customer places an order, the order is stored through the Order API and becomes available to the Kitchen Display System. When kitchen staff change the order status, the updated status is stored by the backend and can be retrieved by the customer Orders interface.

Administrators can also access the same order information through the Admin Dashboard. This creates a connected workflow between customers, kitchen staff, and administrators. From a performance perspective, the API structure separates frontend operations from database operations. The React application does not directly communicate with MySQL. Instead, all database operations are performed by the Node.js and Express.js backend. This separation improves security, simplifies maintenance, and allows the backend to control how data is validated, processed, and stored. It also makes the system easier to extend in the future if additional clients such as mobile applications are introduced.

The API routes are organized according to their respective modules, making the backend easier to maintain and

debug. Changes to one API module can generally be performed without significantly affecting unrelated modules. For example, modifications to the menu API can be handled independently from the kitchen API. This modular structure also allows additional features such as online payment, notifications, customer feedback, or AI-based recommendations to be integrated later through additional API endpoints. In summary, the API integration strategy forms the communication backbone of the Smart Restaurant Ordering and Kitchen Display System. It enables data to flow efficiently between the React frontend, Node.js and Express.js backend, and MySQL database. The REST-based design supports customer ordering, menu management, table selection, kitchen processing, order tracking, and administrative operations through structured endpoints. The combination of JSON communication, role-based access, validation, error handling, and Postman testing provides a reliable foundation for the complete restaurant management system.

6.4 Authentication and Authorisation

The Authentication and Authorisation module is an important part of the Smart Restaurant Ordering and Kitchen Display System, responsible for protecting user accounts, controlling access to different modules, and ensuring that customers, kitchen staff, and administrators can use only the features assigned to them. Authentication verifies the identity of a user, while authorisation determines what actions the authenticated user is allowed to perform within the system. Since the application manages customer information, orders, menu items, tables, and restaurant operations, proper access control is essential. The system implements a role-based authentication mechanism for managing different types of users. During login, the user provides their registered email and password through the login interface. The backend receives these credentials through the user API and verifies them against the information stored in the MySQL database. Passwords are stored using secure hashing rather than plain-text storage. The backend compares the entered password with the stored password hash using the bcrypt library. If the credentials are valid, the backend returns the authenticated user's information along with their assigned role. The system supports three primary user roles: Customer, Kitchen Staff, and Administrator. Each role has different responsibilities and permissions within the application. Customers can browse the menu, manage their cart, place orders, select tables, and track their orders. Kitchen staff can access the Kitchen Display System, view incoming orders, and update order statuses.

Administrators have access to management features such as menu management, table management, order monitoring, and dashboard information. This role-based structure ensures that users are provided with only the functionality required for their responsibilities. After successful authentication, the frontend stores the required login information and uses the user's role to determine the appropriate interface and navigation options. The React application uses conditional rendering and protected routes to control access to different pages. For example, customer users can access pages such as Menu, Cart, Checkout, and Orders, while kitchen users can access the Kitchen Dashboard and administrators can access the Admin Dashboard and management pages.

This prevents unnecessary access to unrelated sections of the application. The backend also performs authorisation checks to provide an additional layer of protection. Frontend restrictions alone are not sufficient because users could attempt to access backend endpoints directly. Therefore, protected API routes verify the authenticated user's role before performing sensitive operations. For example, menu management operations and administrative dashboard information are restricted to authorized administrators, while kitchen operations are limited to kitchen staff. This two-level approach provides better control over access to the system. During registration, the system collects the required customer information and validates the submitted data before creating a new account. The backend checks whether the email address already exists in the database and prevents duplicate accounts from being created. Passwords are hashed before being stored in the Users table.

This ensures that the database does not contain plain-text passwords and provides better protection for user credentials. The login process follows a structured sequence. First, the user enters their email and password through the frontend. The frontend sends the credentials to the backend login endpoint. The backend searches for the corresponding user account and verifies the password using bcrypt. If the credentials are correct, the backend returns the user information and role. The frontend then updates the login state and provides access to the appropriate modules. If the credentials are incorrect, an appropriate error message is displayed to the user. The system also implements input validation and error handling during authentication.

Required fields are checked before requests are submitted. Invalid email addresses, missing credentials, incorrect passwords, and unavailable accounts are handled appropriately. The backend also validates incoming requests before performing database operations. These mechanisms reduce invalid requests and provide clear feedback to users during registration and login. Role-based authorisation is particularly important because the application contains different levels of functionality. A customer should not be able to access administrative menu management or modify restaurant tables, while kitchen staff should not have access to administrative management operations. Similarly, administrative functions should remain separate from normal customer operations.

The role information stored with each user account provides the basis for these access decisions. The authentication system is also integrated with the navigation and application state of the React frontend. When a user logs in or logs out, the frontend updates the current user information and navigation options. The system can also refresh the displayed user information when the login state changes. This ensures that the interface remains consistent with the user's current authentication status. Security is further supported by keeping sensitive backend configuration information separate from the frontend application. Database credentials and other server-side configuration values are maintained within the backend environment rather than being exposed through React components.

The frontend communicates with the backend through REST APIs instead of directly accessing the MySQL database. This separation protects the database from direct client-side access. The authentication module also provides logout functionality, allowing users to end their current session from the application. When a user logs out, the stored login information is cleared and the frontend returns to the appropriate public interface. Protected pages and features are no longer available to the logged-out user. This ensures that another person using the same device cannot continue using the previous user's account. Error handling within the authentication module is designed to provide appropriate responses for different situations. Incorrect login credentials, missing authentication information, unauthorized access, and invalid requests are handled separately. The backend can return suitable HTTP status codes such as 401 Unauthorized for authentication failures and 403 Forbidden when an authenticated user does not have sufficient permissions.

The frontend then displays suitable messages based on the received response. The authentication and authorisation mechanism also works together with the Customer, Kitchen, and Admin modules. Customer authentication connects users to their orders and account information. Kitchen authentication provides access to the Kitchen Display System and order status management. Administrator authentication provides access to management and monitoring features. This integration ensures that each user interacts with the system according to their assigned responsibilities. From a database perspective, the users table maintains important account information such as user ID, name, email, password, role, and account creation information. The role field is used to distinguish between customer, kitchen, and administrator accounts. The user ID can also be associated with customer orders, allowing the system to identify which user placed a particular order. This relationship provides a connection between authentication and order management. The authentication system is designed to support future improvements as the application develops. Additional security features such as email verification, password reset, two-factor authentication, session expiry, and more detailed activity logging can be introduced in future versions.

Similarly, more granular permissions can be added if additional restaurant staff roles are introduced. In conclusion, the Authentication and Authorisation module provides an important security and access-control layer for the Smart Restaurant Ordering and Kitchen Display System.

By combining secure password hashing, user authentication, role-based authorisation, protected routes, backend access control, validation, and error handling, the system separates the responsibilities of customers, kitchen staff, and administrators. This ensures that users can securely access the features relevant to them while protecting restaurant data and management operations.

CHAPTER 7- FEATURES

7.1 List of Main Features

The Smart Restaurant Ordering and Kitchen Display System has been developed as a comprehensive digital platform designed to connect customers, kitchen staff, and administrators through a single integrated system. The system brings together the major activities involved in restaurant ordering and management, beginning with menu browsing and food selection and continuing through cart management, order placement, table selection, kitchen processing, order tracking, and administrative monitoring. Each feature has been designed as part of a connected workflow rather than as an isolated function, with the objective of reducing manual work, minimizing ordering errors, and improving communication between customers and restaurant staff. Foremost among these components is the User Registration and Login Feature, which provides controlled access to the system.

Customers can create accounts and log in using their registered credentials, while kitchen staff and administrators can access their respective interfaces. The system identifies users according to their assigned roles and provides access to the appropriate modules. This feature forms the basic authentication layer of the application and helps maintain separation between customer, kitchen, and administrative operations. The Digital Menu Feature provides customers with an organized interface for browsing the restaurant's available food items. Menu items are presented with information such as food name, category, price, rating, description, and image information. Customers can explore different food categories and select the items they want to order.

The menu information is retrieved from the backend and stored centrally in the MySQL database, allowing administrators to manage menu information when required. The Food Selection and Cart Feature allows customers to create their orders before submitting them to the restaurant. Customers can add food items to the cart, modify quantities, and remove unwanted items. The cart dynamically calculates the order amount based on the selected items and quantities. This feature provides customers with an opportunity to review their selections before proceeding to checkout and reduces mistakes that commonly occur during manual ordering. The Checkout and Order Placement Feature provides the final stage of the customer ordering process. Customers can review their selected food items, quantities, total amount, and required order information before confirming the order. Once the customer submits the order, the frontend communicates with the backend through REST APIs. The backend processes the request and stores the order information and corresponding order items in the MySQL database. This creates a centralized record of every order placed through the system.

The Table Selection Feature allows customers to select a restaurant table during the ordering process. The system maintains information about restaurant tables, including table numbers, seating capacity, and availability status. The selected table is associated with the customer's order and stored in the database. This feature provides a digital alternative to manual table assignment and helps connect customer orders with their corresponding restaurant tables. The Kitchen Display System (KDS) Feature represents the main operational component of the system for kitchen staff. Instead of depending on handwritten order slips or verbal communication, kitchen staff receive incoming orders through a centralized digital display. Each order provides important information such as customer name, table number, food items, quantities, and special instructions.

Kitchen staff can process orders according to the defined status sequence: New → Accepted → Preparing → Ready → Served. The Order Status Tracking Feature allows customers to monitor the progress of their orders after placing them. The current order status is retrieved from the backend and displayed through the customer interface. The status progression provides clear information about whether an order has been accepted, is currently being prepared, is ready, or has been served. This improves order visibility and reduces uncertainty while customers wait for their food.

The Order Management Feature provides centralized handling of customer orders throughout their lifecycle. The system maintains information about the customer, table, ordered food items, quantities, total amount, and current status. The backend connects the order information with the Kitchen Display System and customer order-tracking interface. This ensures that the same order information can be accessed by the appropriate modules without maintaining separate records. Supporting the restaurant's operational activities is the Admin Dashboard Feature, which provides administrators with a centralized view of important system information. The dashboard can display details such as total users, menu items, orders, active orders, revenue, and restaurant tables. Recent orders can also be monitored through the dashboard. This feature provides administrators with a convenient interface for observing restaurant activity and managing important operations.

The Menu Management Feature provides administrators with control over the restaurant's food catalogue. Administrators can manage menu items by adding new items, updating existing information, and removing items when required. Information such as food name, category, price, rating, description, and other menu details can be maintained through the administrative interface. Changes made through the admin module are stored in the MySQL database and can subsequently be reflected in the customer menu. The Table Management Feature allows administrators to maintain restaurant table information through the system. The administrator can view the available tables and manage relevant table information such as table number, seating capacity, and status.

This feature works together with the customer table-selection and order-management functions to provide centralized table information. The User Management Feature provides administrators with information about registered users and their roles. The system maintains customer, kitchen, and administrator accounts within the Users table. Role information determines the modules available to each account. This feature supports centralized account management and helps administrators maintain an organized user structure. The REST API Integration Feature provides communication between the different layers of the application. The React frontend communicates with the Node.js and Express.js backend through REST APIs, while the backend communicates with the MySQL database. APIs are used for operations such as user authentication, menu retrieval, order creation, table management, kitchen operations, and administrative functions.

JSON is used for exchanging information between the frontend and backend, providing a structured communication mechanism. The Security and Role-Based Access Feature is integrated throughout the application to protect system operations. Authentication verifies the identity of users, while role-based authorisation controls access to specific modules. Customers, kitchen staff, and administrators are provided with different permissions according to their responsibilities. Passwords are securely hashed, and protected backend routes help prevent unauthorized users from accessing restricted operations. The Database Management Feature provides centralized storage for the information required by the application. MySQL stores important entities such as users, menu items, restaurant tables, orders, and order items. Relationships between these entities are maintained using primary keys and foreign keys. This centralized structure ensures that customer, kitchen, and admin modules work with consistent information. The

Notification and Feedback Handling capability can be extended in future versions of the system to provide real-time notifications regarding order progress and customer feedback. Although advanced notification services and feedback mechanisms are not part of the current core implementation, the existing modular API and database architecture provides a foundation for integrating these features later. Finally, the System Integration Feature ensures that the Customer, Kitchen, and Admin modules operate as a unified application.

Customer actions such as placing an order are transferred through the backend to the database and made available to the Kitchen Display System. Kitchen status updates can then be reflected in the customer's order tracking interface, while administrators can monitor the same order through the Admin Dashboard. This integration transforms the individual modules into a connected restaurant management workflow. Collectively, these features demonstrate the main capabilities of the Smart Restaurant Ordering and Kitchen Display System. Each component contributes to a unified digital environment where customers can conveniently order food, kitchen staff can efficiently process orders, and administrators can monitor and manage restaurant operations. The modular structure also provides a foundation for future enhancements such as online payment integration, real-time notifications, mobile application support, customer feedback, loyalty programs, and AI-based food recommendations.

7.2 How Each Feature Works

The operation of the Smart Restaurant Ordering and Kitchen Display System is the result of a carefully designed interaction between the customer interface, kitchen operations, administrative controls, and underlying database. Each module has a specific purpose but works together through a common system architecture to provide a smooth digital restaurant experience. The following sections describe how each major feature works from both the user's perspective and the technical perspective governing its execution.

Pasted markdown

- 1. Customer Registration and Login Feature**
User Perspective: Upon accessing the restaurant website, customers can create an account by providing their name, email address, and password. Existing customers can log in using their registered credentials. After successful login, the customer is redirected to the main restaurant interface where they can browse the menu, add food items to the cart, place orders, select a table, and track their order status.
Technical Description: The registration and login functionality is handled through the React.js front-end and Node.js with Express.js backend. Customer details are stored in the MySQL users table. Passwords are securely hashed using bcrypt before being stored. During login, the backend verifies the submitted credentials against the stored user information and returns the authenticated user details and role. Role-based access ensures that customers can access only customer-related features.
- 2. Menu Browsing Feature**
User Perspective: Customers can access the menu page to view available food items organized into categories such as Tiffins, Starters, Main Course, Pizza & Burgers, Desserts, and Beverages. Each food item displays information such as name, price, rating, description, vegetarian status, and image. Customers can select food items and add them directly to the cart.
Technical Description: Menu information is maintained in the MySQL menu_items table. The React.js front-end retrieves menu data through REST API requests handled by Node.js and Express.js. The returned JSON data is displayed dynamically using reusable React components. Menu operations such as adding, updating, deleting, and retrieving items are handled through backend API endpoints connected to the database.

3. **Cart and Food Selection Feature User Perspective:** After browsing the menu, customers can add their preferred food items to the cart. The cart displays selected items, quantities, individual prices, and the total amount. Customers can increase or decrease quantities or remove items before proceeding to checkout. **Technical Description:** The React.js application manages the cart information and updates the interface whenever the customer changes item quantities. The selected menu item details are maintained with their corresponding identifiers and prices. Before order submission, the cart data is sent to the backend through a REST API. The backend validates the order information and prepares it for storage in the database.
4. **Order Placement Feature User Perspective:** After selecting the required food items, the customer proceeds to checkout and confirms the order. The customer provides the required information and can select or declare the restaurant table after ordering the food. Once the order is successfully placed, the customer receives an order confirmation and can view the order in the Orders section. **Technical Description:** When the customer confirms an order, the React.js front-end sends the order details to the Node.js and Express.js backend using a POST request. The backend creates a unique order number and stores the order information in the orders table. Individual food items are stored in the order_items table with their quantities and prices. Foreign-key relationships connect the order with the customer, selected table, and menu items.
5. **Table Selection and Management Feature User Perspective:** Customers can select an available restaurant table after placing their food order. The table interface displays table numbers and availability so that customers can choose a suitable table. Administrators can also view and manage the restaurant tables through the admin section. **Technical Description:** Restaurant table information is stored in the MySQL restaurant_tables table. The backend provides APIs for retrieving table information and updating table status. When an order is associated with a table, the selected table_id is stored in the corresponding order record. This allows the kitchen and admin modules to identify where the order needs to be served.
6. **Kitchen Display System Feature User Perspective:** The Kitchen Display System provides kitchen staff with a centralized screen containing incoming customer orders. Each order displays important information such as order number, customer name, table number, ordered food items, quantities, and special instructions. Kitchen staff can process the order by updating its status. **Technical Description:** The KDS retrieves active orders from the backend through REST APIs. Node.js and Express.js communicate with the MySQL database to retrieve orders and their associated order items. The kitchen interface displays orders in separate cards according to their current status. The system supports the order progression New → Accepted → Preparing → Ready → Served. Status updates are sent to the backend and stored in the orders table.
7. **Order Status Tracking Feature User Perspective:** Customers can open the Orders section to view their current and previous orders. The system displays the current status of each order, allowing customers to understand whether their food is new, accepted, being prepared, ready, or served. **Technical Description:** The order status is maintained in the status field of the orders table. Whenever kitchen staff update an order, the backend modifies this value in MySQL. The customer interface retrieves the latest order information through the order API and displays the corresponding status. This provides visibility into the complete order lifecycle without requiring direct communication with kitchen staff.
8. **Admin Dashboard and Management Feature User Perspective:** The Admin Dashboard provides administrators with an overview of restaurant operations.

Administrators can view total users, menu items, orders, active orders, revenue, and table information. Separate management interfaces allow administrators to manage menu items, restaurant tables, users, and orders. Technical Description: The admin module communicates with dedicated backend routes that retrieve summary information from MySQL. SQL queries are used to calculate order counts, revenue, active orders, users, menu items, and table information. The React.js dashboard displays this information using cards, tables, and management interfaces. Access to administrative features is restricted according to the user's admin role.

9. Authentication and Authorization Feature User Perspective: The system provides separate access for customers, kitchen staff, and administrators. Customers can access ordering features, kitchen staff can access the Kitchen Display System, and administrators can access management features. This prevents users from accessing functions that do not belong to their role. Technical Description: Authentication is implemented through the login system using credentials stored in the MySQL users table. Passwords are protected using bcrypt hashing. After successful login, the application identifies the user role and redirects the user to the appropriate section. React route protection and backend role checking help prevent unauthorized access to customer, kitchen, and admin functionality.

10. Order and Database Management Feature User Perspective: Customers can place orders and view their order history, while kitchen staff and administrators can access the information required for restaurant operations. The system keeps the order information organized so that food items, customers, tables, and order statuses remain connected. Technical Description: MySQL acts as the central database for storing users, menu items, restaurant tables, orders, and order items. The backend performs CRUD operations through Node.js and Express.js APIs. Relationships between tables are maintained using primary keys and foreign keys. The orders and order items tables connect customer orders with their respective food items, quantities, prices, and table information.

11. Integration and System Interoperability Feature User Perspective: To the customer, the system appears as one connected restaurant platform. A customer can log in, browse the menu, add items to the cart, place an order, select a table, and track the order without moving between separate applications. Similarly, kitchen and admin users access their respective modules through the same system. Technical Description: The system follows a layered web architecture in which React.js acts as the front-end, Node.js and Express.js handle application logic and REST APIs, and MySQL provides centralized data storage. JSON is used for communication between the front-end and backend. The API structure allows customer, kitchen, and admin modules to exchange information while maintaining separate responsibilities. This modular design also allows future features such as online payments, real-time notifications, mobile applications, and AI-based food recommendations to be added without redesigning the complete system.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

The process of verifying the reliability and functional accuracy of the Smart Restaurant Ordering and Kitchen Display System's application programming interfaces (APIs) was carried out using Postman, an API testing environment. This testing phase was important for ensuring that the frontend and backend layers communicated correctly and that data was transferred accurately between the React.js application, Node.js and Express.js server, and MySQL database. The testing methodology was designed to simulate common restaurant operations, validate API responses, and identify errors during different stages of system interaction. Postman served as an intermediary testing tool between the frontend and backend of the restaurant system. By sending HTTP requests directly to the backend endpoints, developers were able to test individual services without depending completely on the graphical user interface.

This helped in identifying API-related issues and verifying the behaviour of backend services. Environment variables such as the backend base URL and required request parameters were configured in Postman so that requests could be tested repeatedly during development. A comprehensive collection of test requests was created in Postman for the major functionalities of the system, including user registration, login authentication, menu retrieval, menu management, table retrieval, order creation, order retrieval, order-item management, kitchen order management, and admin dashboard operations. Each request was categorized according to HTTP methods such as GET for retrieving information, POST for creating records, PUT for updating records, and DELETE for removing records. This classification provided a systematic way to verify CRUD (Create, Read, Update, Delete) operations across the application. For frontend validation, the APIs used by the React.js application were tested separately in Postman before integrating them with the user interface. This helped verify that the frontend would receive the expected JSON response from the backend. For example, the menu API was tested to ensure that food item names, prices, categories, ratings, and other details were returned correctly. Similarly, the order API was tested to verify that newly created orders returned the correct order number, customer information, table information, total amount, and order status.

The backend testing involved testing each endpoint with both valid and invalid input data. User registration was tested with valid customer details as well as duplicate email addresses and incomplete information. Login APIs were tested using valid and invalid credentials to verify that incorrect login attempts were rejected appropriately. Menu APIs were tested for retrieving, adding, updating, and deleting menu items. Table APIs were tested to ensure that available restaurant tables were retrieved correctly and that table information was maintained properly. The order management APIs were tested extensively because ordering is one of the core functions of the system. A valid order request was submitted with customer information, selected table information, total amount, and ordered food items. The response was checked to confirm that the order was successfully created. The corresponding order information was then verified through the order retrieval API. This ensured that data was correctly stored and retrieved from the MySQL database.

The Kitchen Display System APIs were also tested using Postman. Requests were made to retrieve active restaurant orders and their associated order items. The returned information was checked for order number, customer name, table number, food items, quantities, instructions, and current order status. Status update requests were then tested to verify the movement of orders through the defined workflow: New → Accepted

→ Preparing → Ready → Served

This confirmed that kitchen staff could update order progress successfully through the backend. In addition to positive testing, negative testing was performed to identify how the backend responds to invalid requests. Incorrect user IDs, invalid order IDs, missing fields, invalid quantities, and unavailable records were used during testing.

The system was checked to ensure that appropriate HTTP response codes such as 200 (OK), 201 (Created), 400 (Bad Request), 401 (Unauthorized), 403 (Forbidden), and 404 (Not Found) were returned wherever applicable. These tests helped verify the reliability of the backend validation and error-handling mechanisms. Boundary condition testing was also applied to important restaurant operations.

For example, food quantities were tested with different values, empty cart submissions were tested, and invalid or missing table information was checked. The order total was also verified against the selected food items and quantities. These tests helped ensure that unusual or incomplete input did not result in incorrect order records. Postman's testing features were used to verify response status codes, response data, and execution times. Test scripts can also be used to automatically check whether the returned JSON response contains the expected fields and values. For example, an order response can be checked for fields such as order ID, order number, customer name, total amount, table ID, and status.

This makes API validation more consistent and reduces the need for repeated manual checking. The Admin APIs were tested separately to verify administrative functionality. Dashboard requests were used to retrieve information such as the total number of users, menu items, orders, active orders, revenue, and tables. Menu management APIs were tested for adding, updating, retrieving, and deleting food items. These tests ensured that the administrator receives accurate information from the database and can perform the required management operations. Authentication and authorization were also tested through Postman.

Login requests were verified using valid and invalid credentials, while protected admin and kitchen endpoints were tested with different user roles. The testing ensured that customer users could access customer functionality while kitchen and admin users could access their respective modules. Unauthorized requests were checked to ensure that restricted operations were not performed. Database integration was verified indirectly through API testing. After creating or modifying records through Postman, corresponding information was retrieved again through GET requests to confirm that the changes were successfully stored in MySQL. For example, after creating an order, the order retrieval API was used to confirm that the order and its associated order items were available correctly.

This helped verify the communication between the backend APIs and the database layer. For frontend integration, the tested APIs were connected to the React.js application. The frontend sends requests to the Node.js and Express.js backend and receives JSON responses. Postman testing helped confirm that the API endpoints used by pages such as Menu, Cart, Checkout, Orders, Kitchen Dashboard, and Admin Dashboard were functioning correctly before and during integration. Error handling was also verified through Postman. When an invalid request was submitted, the backend was checked to ensure that it returned an appropriate response rather than crashing.

This included testing missing request fields, invalid IDs, incorrect login details, and requests for records that do not exist. Such testing improves the stability of the system and provides meaningful responses to the frontend. Postman collections can also be maintained along with the project through GitHub, allowing the

development team to keep API requests organized and accessible. This provides a reusable testing environment whenever backend changes are introduced.

Developers can repeat the same requests after modifying controllers, routes, database queries, or frontend API calls to check whether existing functionality continues to work correctly. From a technical standpoint, Postman testing provided an effective method for validating the communication between the React.js frontend and Node.js/Express.js backend.

It also helped verify the connection between backend services and the MySQL database. Testing individual APIs before complete frontend integration made debugging easier and reduced the possibility of runtime errors. To conclude, Postman testing played an important role in establishing the functional reliability of the Smart Restaurant Ordering and Kitchen Display System. By testing authentication, menu management, tables, cart-related operations, orders, order items, kitchen status updates, and admin services, the system was verified across its major functional areas. The testing process helped ensure that data was exchanged correctly between the frontend, backend, and database, resulting in a more reliable and stable restaurant management application.

8.2 Integration Testing

Integration testing constituted an important phase in validating the overall coherence and interoperability of the Smart Restaurant Ordering and Kitchen Display System. Whereas unit testing ensures the internal accuracy of individual components, integration testing was designed to confirm that the Customer Module, Kitchen Display System, Admin Module, authentication, menu, cart, table management, and order management function together as a unified system. The primary objective was to ensure that data flows between the frontend, backend, and database occurred correctly without loss, duplication, or inconsistency. The testing environment was established to represent the complete application stack consisting of the React.js frontend, Node.js and Express.js backend, REST APIs, and MySQL database. Representative data was used for users, menu items, restaurant tables, orders, and order items. Different scenarios were considered, including valid user login, invalid credentials, food selection, cart operations, order placement, table selection, and kitchen order processing.

A modular incremental strategy was adopted, meaning that system components were integrated one at a time. Initially, the connection between the React frontend and Node.js backend was tested to ensure accurate request–response communication. Once verified, authentication was integrated, followed by menu management, cart operations, order processing, table management, and finally the Kitchen Display System and Admin Dashboard. This gradual approach helped identify integration problems at an early stage. At the heart of the testing procedure was the validation of API communication between the frontend and backend. For example, when a customer browses the menu, the frontend communicates with the menu API and receives the required food item information. Similarly, when a customer places an order, the order details are sent through the REST API to the backend, where the information is stored in the MySQL database. Integration testing ensured that the required information such as user ID, menu item ID, quantity, price, table ID, and order status was transferred correctly. The order processing workflow represented another important area of integration testing. When a customer places an order, the order is stored in the database with the initial status New.

The Kitchen Display System retrieves the order and allows kitchen staff to update its status through New → Accepted → Preparing → Ready → Served. Integration tests verified that each status update was correctly stored in the database and reflected in the customer order tracking interface. The testing also evaluated database consistency across different modules. When a customer adds food items to the cart and places an order, the corresponding records are created in the orders and order_items tables. The integration tests verified that the correct menu items, quantities, prices, customer details, and table information were associated with the correct order. This helped prevent incorrect item mapping and duplicate or incomplete order records. Session persistence and role-based access were also tested across modules. Customer accounts were verified to access customer-related features such as the menu, cart, checkout, and orders. Kitchen users were tested for access to the Kitchen Display System, while administrators were tested for access to the Admin Dashboard and management features.

Integration testing confirmed that users were directed to the appropriate modules based on their assigned roles. A major element of the integration tests involved communication between the Customer Module and Kitchen Display System. When a customer successfully submits an order, the order should become available to the kitchen staff without requiring manual entry.

The kitchen interface was tested to confirm that customer name, table number, ordered items, quantities, instructions, and order status were displayed correctly. Changes made by kitchen staff were also verified against the stored order information. The Admin Module was integrated with the database and tested for correct retrieval of system information. The Admin Dashboard was checked to ensure that information such as total orders, customers, menu items, active orders, revenue, and tables was retrieved correctly from the backend. Menu and table management operations were also tested to verify that changes made through the admin interface were properly reflected in the system. Frontend integration testing confirmed that the React.js components correctly rendered dynamic information received from backend APIs. When users performed operations such as login, browsing menu items, adding food to the cart, placing orders, or checking order status, the interface was tested to ensure that the displayed information was updated correctly. Error conditions such as invalid login details, unavailable data, or unsuccessful API requests were also tested to ensure that suitable messages were displayed without breaking the application flow.

Performance and reliability were also considered during integration testing. Multiple API requests were tested to verify that the frontend, backend, and database could communicate correctly without unexpected failures. The system was observed during operations involving menu retrieval, order creation, order status updates, and dashboard data retrieval. These tests helped identify communication issues and database query problems during development. Integration testing also supported security verification. Protected routes were tested to ensure that users without the required authentication or role could not access restricted modules. Customer, kitchen, and administrator access was checked separately. Database operations were also verified to ensure that the correct records were modified during CRUD operations. From a process perspective, integration test cases were documented with their input conditions, expected results, actual results, and status. Test cases covered successful operations as well as invalid and boundary scenarios. Any integration issues identified during testing were corrected and the affected test cases were executed again to verify the fixes. In its concluding phase, integration testing confirmed the coordinated operation of the major components of the Smart Restaurant Ordering and Kitchen Display System. The successful communication between the React frontend, Node.js and Express.js backend, REST APIs, and MySQL database demonstrated that the system could handle complete restaurant workflows.

The integration of customer ordering, kitchen processing, table management, order tracking, and administration provided a connected and reliable application environment. This testing stage therefore helped ensure that the system was ready for further deployment and practical use in a restaurant environment.

8.3 Tools Used for Testing

The success of the Smart Restaurant Ordering and Kitchen Display System testing phase was supported by the systematic use of suitable development and testing tools that ensured accuracy, reliability, and efficiency. Each tool was selected based on its role in validating different parts of the system, including REST API communication, frontend functionality, database operations, version control, and application performance. Together, these tools provided a structured testing environment for validating the Customer, Kitchen, and Admin modules.

1. Postman

Postman was the primary tool used for API testing and validation. It provided an easy-to-use interface for creating, sending, and analysing REST API requests and responses.

The Node.js and Express.js backend exposes APIs related to user authentication, menu items, orders, tables, order items, kitchen operations, and admin operations. Postman was used to test different HTTP methods such as GET, POST, PUT, and DELETE and to verify that the APIs returned the expected JSON responses and HTTP status codes. For example, login, menu retrieval, order creation, table retrieval, order status updates, and admin dashboard APIs were tested using Postman. Postman collections were used to organize related API requests and simplify repeated testing. Different valid and invalid inputs were also tested to verify backend validation and error handling. This helped confirm that the frontend could communicate correctly with the backend APIs.

2. Git and GitHub

Git and GitHub were used for version control and maintaining the project source code. The repository contained the React frontend, Node.js and Express.js backend, database-related scripts, and documentation. Git helped maintain different versions of the project and track changes made during development. Separate changes related to customer features, kitchen features, admin features, and bug fixes could be managed systematically. GitHub also provided a centralized location for storing and sharing the project code among team members. The use of version control was helpful during testing because previous working versions could be referred to when an issue was introduced after a code modification.

3. Web Browser Developer Tools

Browser Developer Tools were used for testing and debugging the React.js frontend. The tools available in browsers such as Chrome were useful for checking console errors, network requests, API responses, and page behaviour. The Network tab was used to verify whether requests were correctly sent from the frontend to the backend and whether the expected responses were received. The Console helped identify JavaScript and React errors during operations such as login, menu browsing, cart management, checkout, and order tracking. Developer Tools also helped verify that the user interface was correctly updated when order information or status changed.

4. MySQL

MySQL was used to validate the database operations performed by the application. The database contains important tables such as users, menu_items, restaurant_tables, orders, and order_items. During testing, database records were checked after operations such as user registration, login, menu management, order creation, table selection, and order status updates. This helped verify that the information received through the REST APIs was correctly stored and retrieved. Database verification was particularly important for checking the relationship between orders and their corresponding order items, users, menu items, and restaurant tables.

5. Visual Studio Code

Visual Studio Code was used as the primary development and debugging environment. The project frontend and backend code were maintained and tested through VS Code. The integrated terminal was used to run the React/Vite development server and Node.js/Express backend. VS Code also helped identify syntax errors, coding issues, and problems during application execution.

6. React Developer Tools

React Developer Tools were useful for inspecting React components and understanding the frontend application state. Components related to the menu, cart, checkout, orders, kitchen dashboard, and admin dashboard could be inspected during testing. This helped identify issues related to component rendering,

state updates, and data received from backend APIs.

7. GitHub

GitHub was also used as a centralized repository for maintaining the project and its testing-related changes. Source code updates, documentation, and project files were stored in the repository. The repository provided a backup of the project and helped the team maintain a history of changes made during development and testing.

8. Browser

Testing Different web browsers were used to verify the basic compatibility of the application. The React frontend was tested using commonly available browsers such as Google Chrome, Microsoft Edge, and Mozilla Firefox. The testing focused on menu display, navigation, login, cart operations, checkout, order tracking, Kitchen Display System, and Admin Dashboard functionality. This helped ensure that the application behaved consistently during normal usage.

9. REST API

Testing The REST APIs connecting the React frontend and Node.js backend were tested systematically. The major API groups included: User APIs Menu APIs Order APIs Table APIs Order Item APIs Kitchen APIs Admin APIs. The requests and responses were checked to ensure that the correct data was transferred between the frontend, backend, and MySQL database.

10. Testing

Documentation Test cases and testing results were documented during the testing phase. Each test case included the operation being tested, input conditions, expected result, actual result, and testing status. The documented test cases covered important workflows such as registration, login, menu browsing, cart operations, order placement, table selection, kitchen status updates, and admin management. Overall, the combination of Postman, Git/GitHub, browser Developer Tools, MySQL, Visual Studio Code, React Developer Tools, and browser testing provided the necessary tools for validating the Smart Restaurant Ordering and Kitchen Display System. These tools helped verify API communication, frontend behaviour, database consistency, role-based functionality, and the complete order workflow from customer placement to kitchen processing and status tracking.

8.4 Test Cases and Results

The testing phase of the Smart Restaurant Ordering and Kitchen Display System was carried out in a systematic and highly structured manner to verify that every functional and non-functional aspect of the application performed according to specification and user expectations. A comprehensive suite of test cases was prepared to examine the correctness, reliability, and security of all major modules—Customer, Kitchen Display System, Admin, Menu, Cart, Table Management, and Order Management—together with supporting components such as authentication, REST API integration, and database operations. Each test case was documented with clear pre-conditions, execution steps, expected outcomes, and pass criteria, and these were executed repeatedly using manual testing and API-based testing methods. Automated API verification was handled through Postman, while frontend workflows were validated through browser-based testing to ensure that the React.js interface responded accurately to data received from the Node.js and Express.js backend.

Functional testing confirmed that user registration, login, menu browsing, cart operations, table selection, order placement, and order tracking operations behaved as defined, returning consistent responses and writing correct data to the underlying MySQL database. API testing through Postman verified that the REST endpoints correctly handled requests related to users, menu items, orders, tables, order items, kitchen operations, and administrative functions. Different HTTP methods such as GET, POST, PUT, and DELETE were tested to confirm proper communication between the frontend and backend. Integration testing verified the complete flow between the React.js frontend, Node.js and Express.js backend, REST APIs, and MySQL database.

When a customer selected food items and placed an order, the system was tested to ensure that the order details were correctly stored in the database and made available to the Kitchen Display System. The kitchen workflow was tested through the status sequence New → Accepted → Preparing → Ready → Served, confirming that every status update was correctly processed and displayed. Non-functional evaluations were also performed to verify usability, browser compatibility, and application reliability.

The system interface was tested on commonly used browsers such as Chrome, Edge, and Firefox to verify that navigation, menu display, cart operations, checkout, order tracking, Kitchen Display System, and Admin Dashboard worked correctly. Browser Developer Tools and React Developer Tools were used to identify frontend errors, network issues, and component rendering problems. Security testing focused on authentication and role-based authorization. Customer, kitchen, and admin accounts were tested separately to verify that users could access only the modules permitted for their respective roles. Invalid login credentials and unauthorized access attempts were also tested to confirm that restricted modules were protected. Password handling through the backend authentication system was also verified during testing.

During integration testing, a small number of issues related to API communication, incorrect data mapping, order information display, and frontend validation were identified and resolved. After applying the corrections, the affected test cases were executed again to verify that the issues had been successfully fixed and that existing functionality remained unaffected. Regression testing was performed after major changes to ensure that previously working modules continued to function correctly.

Database testing was carried out using MySQL to verify the accuracy and consistency of records created during application operations. User registrations, menu items, restaurant tables, orders, and order items were checked after different transactions. The relationships between customers, orders, tables, and menu items were also verified to ensure that the correct records were associated with each order.

The final consolidated test results showed successful execution of the major functional test cases covering customer registration and login, menu browsing, cart management, order placement, table selection, kitchen processing, order status tracking, admin management, API communication, and database operations. The test results confirmed stable interoperability between the frontend, backend, REST APIs, and database layers. Overall, the results validated that the Smart Restaurant Ordering and Kitchen Display System delivers a dependable, organized, and efficient environment for customers to place food orders, kitchen staff to manage and process orders, and administrators to manage restaurant operations through a single unified system, fulfilling the major objectives of the project and providing a stable foundation for practical restaurant use.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

The deployment of the Smart Restaurant Ordering and Kitchen Display System represents the final stage of the development lifecycle, transforming the developed application into a usable web-based restaurant management system. Deployment is an important phase within the software engineering process, requiring careful coordination of the frontend, backend, database, configuration, and post-deployment verification. The following section elaborates, in detail, the multi-stage deployment procedure used to ensure stability, maintainability, and proper functioning of the system. The initial step in the deployment process involves environment preparation. The development team begins by ensuring that the project source code is updated and maintained in the Git repository hosted on GitHub. The project consists of the React.js frontend, Node.js and Express.js backend, and MySQL database. Before deployment, the source code is checked for errors and the major application modules are tested to confirm that the Customer, Kitchen, and Admin functionalities are working correctly. Required dependencies are installed using the respective package configuration files. Following environment preparation, the frontend build process is initiated.

The frontend is developed using React.js and Vite. The production build is generated using the appropriate npm build command, which compiles the React components, optimises the JavaScript and CSS files, and prepares the static assets required for deployment. The generated build files contain the customer interface, Kitchen Display System, Admin Dashboard, menu pages, cart, checkout, and order tracking interfaces. The next step involves preparing the backend environment. The backend is developed using Node.js and Express.js and provides REST APIs for users, menu items, orders, tables, order items, kitchen operations, and admin operations. The required Node.js packages are installed using npm, and the backend configuration is checked before starting the server. The backend is configured to communicate with the MySQL database and provide API responses to the React frontend. After the frontend and backend are prepared, the MySQL database configuration is completed. The smart_restaurant database is created and the required tables are configured, including users, menu_items, restaurant_tables, orders, and order_items.

The database is populated with the required restaurant data such as menu items and restaurant tables. The database connection configuration is then verified to ensure that the Node.js backend can successfully connect to MySQL and perform the required CRUD operations. The next phase involves local system deployment and integration. During the current development and testing stage, the React frontend runs using the Vite development server, while the Node.js and Express.js backend runs separately. The frontend is accessed through the local development address, while the backend provides the REST APIs through its configured local server. The frontend communicates with the backend using HTTP requests, and the backend communicates with the MySQL database for storing and retrieving application data. Once the local deployment has been configured, post-deployment testing is performed. The team verifies important workflows such as user registration, login, menu browsing, adding food items to the cart, placing orders, selecting tables, and viewing order status.

The Kitchen Display System is tested to confirm that newly placed orders appear correctly and that kitchen staff can update the status through New → Accepted → Preparing → Ready → Served. The Admin Dashboard is also tested to verify menu, table, user, and order management functions. The deployment process also includes configuration and security verification.

Database credentials and other configuration information are maintained separately from the main application code wherever applicable. Authentication and role-based authorization are tested to ensure that customers, kitchen staff, and administrators can access only their permitted modules. Passwords are securely handled using the backend authentication mechanism, and unnecessary sensitive information is not exposed through API responses. Version control is maintained throughout the deployment process using Git and GitHub. Changes made during development and deployment are committed to the repository so that different versions of the project can be tracked. Before using a stable version of the application, the latest source code is checked and tested to ensure that recent modifications have not introduced errors into existing functionality. Once the deployment configuration is completed, end-to-end verification is performed. This includes testing the complete workflow from customer login and menu selection to cart operations, order placement, table selection, kitchen processing, order status updates, and customer order tracking.

Administrative operations are also verified to ensure that the system provides consistent data across all modules. Finally, deployment documentation and version information are maintained for future reference. The project configuration, installation steps, database setup, frontend setup, backend setup, and required dependencies are documented so that the system can be reproduced on another development machine. Backup copies of the database and source code can also be maintained to support recovery in case of unexpected errors. In summary, the deployment of the Smart Restaurant Ordering and Kitchen Display System is a structured process involving environment preparation, frontend and backend setup, MySQL configuration, local integration, security verification, testing, and documentation. By following these deployment steps, the system provides a stable and maintainable environment connecting customers, kitchen staff, and administrators through a unified digital restaurant platform.

9.2 Environment Configuration

Environment configuration represents one of the most crucial aspects of software engineering practice, serving as the technical foundation that ensures the seamless interoperability of all system components. For the Smart Restaurant Ordering and Kitchen Display System, a carefully structured environment configuration is essential to maintain uniformity, predictability, and reliability throughout the development and testing lifecycle. This section elaborates, in detail, the environmental setup process, covering the configuration of software and hardware environments, dependency management, environment variables, version control integration, database configuration, and system optimisation practices. The environment configuration process commences with the establishment of the development and testing environments. The development environment supports coding, debugging, API development, database configuration, and frontend development, while the testing environment is used to verify the integrated behaviour of the application before final deployment. The environments are configured to use the same major technologies and dependencies so that the application behaves consistently during development and testing. The software environment begins with selecting a suitable operating system such as Windows or Ubuntu Linux for development. The front-end environment is powered by Node.js and Vite, with npm used for dependency management. The frontend is developed using React.js, HTML, CSS, and JavaScript. The back-end environment uses Node.js with Express.js, which provides the REST API layer and handles communication between the frontend and database. For database connectivity, MySQL is used as the relational database management system for storing users, menu items, restaurant tables, orders, and order items. Dependency management is an important part of maintaining a stable environment. The project dependencies are declared within the respective package.json files for the frontend and backend.

The package-lock.json files maintain the installed package versions and help provide consistent dependency installation across systems. npm commands are used to install the required packages and run common development tasks such as starting the frontend and backend servers. A significant part of environment configuration involves defining environment-specific configuration values. Database connection details, backend configuration, and API base URLs are configured according to the development environment. Sensitive information such as database passwords should not be directly exposed within publicly shared source code. Configuration files containing sensitive information are excluded from version control through the .gitignore file wherever applicable. The React frontend communicates with the Node.js backend through the configured backend API address. The frontend environment configuration focuses on setting up the React.js application and its required dependencies. Vite is used as the development and build tool for the React application. The frontend contains components and pages for customer login, registration, menu browsing, cart, checkout, orders, Kitchen Display System, Admin Dashboard, menu management, and table management. The development server allows the frontend to be tested locally before production deployment.

The backend environment configuration consists of Node.js, Express.js, REST API routes, controllers, and database connectivity. The backend provides separate API routes for users, menu items, orders, tables, order items, kitchen operations, and administrative operations. The backend server is configured to accept requests from the React frontend and communicate with MySQL for storing and retrieving application data. The database configuration is central to the environment's integrity. MySQL is configured with the smart_restaurant database containing tables such as users, menu_items, restaurant_tables, orders, and order_items.

The database connection is established through the backend configuration. Foreign-key relationships are maintained between related tables to preserve data consistency. Database operations are tested by creating, retrieving, updating, and deleting appropriate records through the backend APIs. Testing environments are configured to verify the system without affecting unrelated application data. Sample users, menu items, tables, and orders are used during development and testing. API requests are tested using Postman, while frontend behaviour is checked through web browsers and browser Developer Tools. The testing environment allows the team to identify integration, database, authentication, and interface-related issues before final deployment. Version control forms another important part of the environment configuration. Git and GitHub are used to maintain the project source code and track changes. The frontend and backend source files are stored in the project repository, while sensitive configuration files and unnecessary generated files are excluded through `.gitignore`. This allows team members to maintain a consistent project structure and recover previous working versions when required. Network configuration is also required for communication between the different application layers. During local development, the React frontend runs on the Vite development server, while the Node.js and Express.js backend runs on its configured server port. The frontend sends HTTP requests to the backend REST APIs, and the backend communicates with the MySQL database.

Correct API URLs, ports, and CORS configuration are verified to ensure successful communication between the application components. Security configuration is maintained through authentication and role-based authorization. The system supports customer, kitchen, and admin roles. Customer users are provided access to customer-related functionality, kitchen users access the Kitchen Display System, and administrators access management features. Passwords are handled through the backend authentication mechanism, and sensitive configuration values are not intended to be stored directly in publicly accessible source files.

Documentation and version control complete the configuration process. A project README and configuration information can be maintained to describe the required software, dependencies, database setup, commands for running the frontend and backend, and basic installation steps. Configuration files are version-controlled except for sensitive information, while the `.gitignore` file prevents unnecessary or confidential files from being committed to the repository. In conclusion, the environment configuration of the Smart Restaurant Ordering and Kitchen Display System follows the principles of standardisation, consistency, security, and maintainability. Through React.js and Vite for the frontend, Node.js and Express.js for the backend, MySQL for database management, npm for dependency management, and Git/GitHub for version control, the project establishes a structured development environment. This configuration ensures consistent communication between the customer, kitchen, and admin modules and provides a reliable foundation for testing, maintenance, and future deployment.

9.3 Hosting of Frontend and Backend

Hosting represents the final and perhaps most visible stage in the software lifecycle, wherein the system transitions from a controlled development environment into an accessible, reliable, and maintainable application environment. For the Smart Restaurant Ordering and Kitchen Display System, the hosting configuration has been designed to ensure proper availability, communication between frontend and backend, database connectivity, and a smooth user experience for customers, kitchen staff, and administrators.

This section describes the hosting architecture and operational processes governing both the front-end and back-end components of the system. It also discusses network configuration, application access, version control, database connectivity, and post-deployment verification. The current implementation is primarily configured for local hosting and development, providing a controlled environment for testing and demonstration. The hosting strategy is based on a local application architecture, where the React.js frontend, Node.js and Express.js backend, and MySQL database operate within the development environment. This approach provides direct control over the application during development and testing and allows the team to verify the complete system before considering deployment to a public cloud or production server.

The frontend communicates with the backend through REST APIs, while the backend communicates with the MySQL database for storing and retrieving restaurant information.

Frontend Hosting The frontend of the system, built using React.js and Vite, is hosted locally using the Vite development server. During development, the application is started using the appropriate npm command and is made available through the local browser. The frontend contains the customer interface, including login, registration, menu browsing, cart, checkout, table selection, and order tracking. It also contains the Kitchen Display System and Admin Dashboard interfaces. During the development build process, Vite compiles the React components and prepares the required JavaScript, CSS, and static assets. The frontend communicates with the Node.js backend through the configured REST API address. Requests related to users, menu items, orders, tables, kitchen operations, and administrative functions are sent from the React application to the backend. The frontend hosting environment allows developers to test changes immediately within the browser.

Browser Developer Tools are used to inspect console messages, network requests, API responses, and frontend rendering behaviour. The application can be accessed from the local development machine through the configured frontend port, allowing the complete restaurant workflow to be demonstrated without requiring an external hosting provider.

Backend Hosting The backend, developed using Node.js and Express.js, is hosted locally as an independent server. The backend provides the REST API layer responsible for handling requests from the React frontend and communicating with the MySQL database. Separate routes are provided for users, menu items, orders, tables, order items, kitchen operations, and administrative operations. The backend server is configured to run on the local machine using its designated port.

It processes incoming HTTP requests, performs authentication and validation, executes the required database operations, and returns JSON responses to the frontend. This architecture keeps the frontend and backend logically separated while allowing them to communicate through REST APIs. The backend communicates with the MySQL database through the configured database connection. Whenever customers register, browse menu information, place orders, select tables, or track order status, the backend processes the request and interacts with the appropriate database tables. Similarly, kitchen and admin operations are processed through the backend APIs.

Database Hosting The MySQL database is hosted locally as part of the development environment. The database used by the project is smart_restaurant, which contains the primary tables required for system operation, including users, menu_items, restaurant_tables, orders, and order_items. The database stores customer information, menu details, restaurant table information, order records, and individual order items. The backend establishes a connection with MySQL and performs the required CRUD operations. Foreign-key relationships between the tables help maintain consistency between users, orders, menu items, tables, and order items. Database connectivity is verified before running the

complete application. Test records are used during development to confirm that information can be inserted, retrieved, updated, and deleted correctly. Database records are also checked during testing to verify that frontend operations are correctly reflected in the database. **Application Communication** The communication between the frontend, backend, and database follows a structured flow. The React frontend sends requests to the Node.js and Express.js REST APIs. The backend receives these requests, performs the required business and database operations, and returns JSON responses to the frontend.

For example, when a customer places an order, the React frontend sends the order information to the backend. The backend validates the request and stores the order and its associated items in MySQL. The Kitchen Display System then retrieves the order through the kitchen API and displays it to the kitchen staff. When the kitchen staff changes the order status, the updated status is stored through the backend and becomes available for order tracking. **Version Control and Deployment** The project source code is maintained using Git and GitHub. The repository contains the React frontend, Node.js and Express.js backend, configuration files, and other required project resources. Version control allows the development team to track changes and maintain previous working versions of the system. Before running a stable version of the application, the latest changes are reviewed and tested. The frontend and backend are started separately, and the database connection is verified. This process helps ensure that the different application components remain compatible with each other.

At the current stage, the project is hosted locally for development, testing, demonstration, and academic evaluation. Cloud hosting, domain configuration, and automated production deployment are not part of the current implementation and can be considered as future deployment enhancements. **Network and Access Configuration** The frontend and backend communicate through local network ports configured during development. The React/Vite frontend operates through its local development server, while the Node.js and Express.js backend operates through its configured server port. API requests are directed from the frontend to the backend using the configured API base address. CORS configuration is used where required to allow communication between the frontend and backend development servers.

Browser Developer Tools and Postman are used to verify that requests reach the correct backend endpoints and that appropriate responses are returned. **Scaling, Backup, and Future Hosting** The current implementation is designed for local development and does not include cloud-based auto-scaling or load-balancing infrastructure. For future production deployment, the frontend could be hosted on a web hosting platform and the Node.js backend could be deployed on a dedicated application server or cloud service. The MySQL database could similarly be moved to a managed database service. Regular database backups can be maintained to protect restaurant information such as users, menu items, tables, and orders. As the number of customers and orders increases, the system can be extended with scalable hosting, load balancing, monitoring, and automated deployment mechanisms. **Post-Deployment Verification and Maintenance** Following each local deployment or major application update, a post-deployment verification phase is performed to validate the system's stability.

The frontend and backend servers are started and their connectivity is checked. Critical workflows such as user registration, login, menu browsing, cart operations, order placement, table selection, kitchen order processing, and order tracking are tested. The Admin Dashboard is also verified to ensure that menu items, tables, users, and orders are displayed correctly. API endpoints are checked through Postman, while database records are verified in MySQL to ensure that data is stored and retrieved correctly. Maintenance activities include updating npm dependencies when required, correcting application errors, maintaining database records, and keeping the GitHub repository updated. Any changes made to the system are tested before being integrated into the working version.

In summary, the hosting of the Smart Restaurant Ordering and Kitchen Display System follows a structured architecture consisting of a React.js frontend, Node.js and Express.js backend, and MySQL database hosted within the local development environment. This arrangement provides reliable communication between customers, kitchen staff, administrators, and the database while keeping the system easy to develop, test, demonstrate, and maintain. The current local hosting setup also provides a foundation for future migration to cloud-based hosting and production deployment when required.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

The development of the Smart Restaurant Ordering and Kitchen Display System presented a wide range of technical, operational, and coordination challenges that tested the problem-solving and adaptability of the project team. As the system integrates customer ordering, menu management, table selection, kitchen operations, order tracking, authentication, and administration within a single web application, the issues encountered covered different stages of the software development lifecycle—from frontend and backend implementation to database integration, testing, and deployment. One of the earliest and most persistent challenges involved integration between the React.js frontend and the Node.js and Express.js backend. The frontend and backend were developed as separate components and had to communicate through REST APIs. During development, issues such as incorrect API URLs, server connection errors, CORS problems, and unexpected API responses were encountered. These issues sometimes prevented menu data, order information, or user details from being displayed correctly in the frontend.

Debugging the API requests and verifying the corresponding backend responses helped resolve these communication problems. Another major issue involved database integration and data consistency. The system uses MySQL to manage users, menu items, restaurant tables, orders, and order items. During development, differences between frontend food item identifiers and database identifiers created incorrect item mappings in the cart and order workflow. For example, an item selected from the frontend could correspond to a different database record when the identifiers were not aligned correctly.

This required careful checking of database records, API responses, and frontend data structures to ensure that the correct food item, quantity, and price were associated with each order. The menu and cart implementation also presented challenges. The restaurant application contains a large number of food items, and maintaining consistent information such as item IDs, names, prices, categories, ratings, images, and vegetarian status required careful data management. During testing, incorrect or outdated cart data stored in the browser could cause unexpected items to appear. The cart implementation therefore required validation of local storage data and proper synchronization between the selected food items and the displayed cart contents. The frontend development phase encountered several challenges related to React components, state management, and dynamic data rendering. Components such as the menu, cart, checkout, orders, Kitchen Display System, and Admin Dashboard depend on changing application data. Incorrect state updates or API responses could result in outdated information being displayed.

Additional debugging was required to ensure that login information, cart contents, order details, and order status were updated correctly when users performed different operations. The Kitchen Display System introduced another important development challenge. The system needed to receive newly placed customer orders and display the correct customer name, table number, food items, quantities, and instructions.

It also needed to support the complete order status sequence New → Accepted → Preparing → Ready → Served. Ensuring that status changes made by kitchen staff were correctly stored in the database and reflected in the relevant interfaces required repeated API and integration testing. Authentication and role-based authorization also presented challenges during implementation. The system supports three main roles—customer, kitchen, and admin. Each role requires access to different parts of the application. During development, route protection and role verification had to be carefully implemented to prevent users from accessing modules that were not intended for their role. Login

validation, password handling, role information, and frontend navigation were tested repeatedly to ensure correct access behaviour. The Admin Dashboard also required debugging during development because information retrieved from the backend had to be mapped correctly to the frontend dashboard. Statistics such as total users, total orders, active orders, revenue, menu items, and tables depend on backend database queries. Differences between the backend response structure and the frontend expectations initially resulted in incorrect dashboard values. The response mapping and API handling were therefore reviewed and corrected. Another challenge was maintaining correct table and order relationships. The system allows customers to select a restaurant table as part of the ordering process. The selected table must be associated with the correct order in the database.

During development, table information was tested together with order creation to ensure that the correct table was connected to the corresponding customer order and displayed properly in the Kitchen Display System. Testing and debugging represented another significant area of difficulty. Individual components could be tested separately, but integration testing became more complex because the customer, kitchen, admin, backend, and database modules depended on one another. Postman was used to test APIs independently, while browser testing was used to verify the complete frontend workflow. Different invalid inputs and API failures were also tested to identify problems in error handling. The development environment also introduced dependency and configuration issues. The React frontend and Node.js backend require separate packages and configurations.

Problems related to npm dependencies, server ports, API configuration, and frontend-backend connectivity had to be resolved during development. Maintaining the correct project structure and ensuring that both frontend and backend servers were running with compatible configurations was necessary for successful integration. Version control and collaborative development presented additional challenges. Since multiple project components were developed and modified during the project, changes to shared files could result in conflicts or inconsistencies. Git and GitHub were used to maintain the source code and track changes. Careful organisation of commits and regular synchronisation of project files helped reduce these problems. The deployment and hosting stage also introduced challenges related to local server configuration.

The frontend and backend operate as separate services and must run on their respective ports. Database connectivity, API URLs, CORS configuration, and server availability had to be checked whenever the complete application was started. These issues were addressed by verifying the server configuration and testing communication between all three layers. Finally, maintaining consistency between the Customer, Kitchen, and Admin modules was an important challenge throughout development. A change made to one part of the system could affect another module because all modules share the same backend APIs and database. For example, an order created by a customer must appear correctly in the kitchen interface and also be reflected in the admin order information. Repeated integration testing was therefore required to ensure that changes remained consistent across the complete application. In summary, the development of the Smart Restaurant Ordering and Kitchen Display System was an iterative learning process that involved several technical and coordination challenges.

Issues related to frontend-backend communication, database integration, cart data, authentication, role-based access, order processing, KDS status updates, admin dashboard data, and local deployment were identified and resolved through continuous testing and debugging. Each challenge provided practical experience in React.js, Node.js, Express.js, MySQL, REST APIs, authentication, database management, and software integration, contributing to the development of a more stable and functional restaurant management system.

10.2 Solutions Applied

The resolution of the numerous challenges faced during the development of the Smart Restaurant Ordering and Kitchen Display System required a systematic and practical approach, combining software development principles with continuous testing and debugging. Each solution applied reflected an incremental development process, as many of the issues were interconnected across the frontend, backend, database, and user-interface layers. The following section presents a detailed discussion of the strategies, corrective actions, and implementation approaches used to address the technical, operational, and coordination problems identified during the project lifecycle. To address the frontend and backend integration inconsistencies, the development team adopted a structured REST API architecture using Node.js and Express.js. Separate API routes were created for users, menu items, orders, tables, order items, kitchen operations, and administrative operations.

The React.js frontend was configured to communicate with these APIs through HTTP requests and JSON responses. API requests were tested using Postman to verify that the required data was being sent and received correctly. This approach helped resolve issues related to incorrect API endpoints, request formats, and unexpected backend responses. For the data consistency and synchronisation challenges, the relationship between frontend data, backend APIs, and MySQL records was carefully verified. The menu item identifiers used by the frontend and database were checked to ensure that the correct food items were associated with cart and order records. The cart implementation was also reviewed to prevent outdated browser-stored information from interfering with newly selected items. These corrections ensured that the selected food item, quantity, price, and corresponding order information remained consistent throughout the ordering process.

In terms of database performance and schema organisation, the database structure was maintained using MySQL with separate tables for users, menu items, restaurant tables, orders, and order items. Primary keys and foreign-key relationships were used to maintain proper relationships between records. The orders and order_items tables were connected so that each order could contain multiple food items with their respective quantities and prices. Database queries and API responses were checked during testing to ensure that records were correctly inserted, retrieved, updated, and deleted. To resolve frontend rendering and state management issues, the React application was divided into reusable components and pages for different system functions. Components were structured for features such as menu browsing, cart management, checkout, order tracking, Kitchen Display System, and Admin Dashboard. React state and browser storage were used where required to maintain information during user interactions. Additional debugging was performed when incorrect data or outdated information appeared in the interface. Browser Developer Tools and React Developer Tools helped identify rendering and state-related issues.

In tackling the Kitchen Display System integration challenges, the kitchen workflow was connected directly with the order APIs. When a customer placed an order, the backend stored the order information in MySQL and made it available through the kitchen API. The Kitchen Display System retrieved the order and displayed the customer name, table number, food items, quantities, and instructions.

The order status workflow was implemented as New → Accepted → Preparing → Ready → Served, with every status change being stored through the backend. Authentication and role-based authorization problems were addressed by separating the application into customer, kitchen, and admin roles.

Passwords were securely handled using bcrypt, while the backend checked the role associated with each user

account. Frontend route protection was also implemented to prevent users from directly accessing restricted pages. Customers were provided access to customer functions, kitchen users to the Kitchen Display System, and administrators to management features. This approach helped ensure that each user interacted only with the functions relevant to their role. The Admin Dashboard data issues were resolved by aligning the backend response structure with the frontend data mapping. Backend dashboard queries were checked to provide values such as total users, menu items, orders, active orders, revenue, and tables. The frontend was then updated to correctly read these values from the returned dashboard object. This ensured that the administrative statistics displayed in the dashboard matched the actual database information.

To address table and order relationship problems, table information was integrated directly with the order creation process. Customers select a table as part of the ordering workflow, and the selected table ID is associated with the corresponding order record in MySQL. The Kitchen Display System retrieves this information so that kitchen staff can identify the table associated with each order. Database relationships and API responses were repeatedly tested to ensure that the correct table was connected to each order. Addressing API testing and debugging challenges required a structured testing process using Postman. Individual API endpoints were tested with valid and invalid inputs before being integrated with the frontend. Successful and unsuccessful responses were checked using HTTP status codes and JSON response data. API testing was performed for user authentication, menu operations, table retrieval, order creation, order item operations, kitchen operations, and admin functionality.

This helped identify backend errors before they affected the complete application. For dependency and configuration issues, the frontend and backend package configurations were maintained separately using their respective `package.json` and `package-lock.json` files. Required npm dependencies were installed and verified before running the application. The frontend and backend servers were configured to operate on their designated local ports, while the database configuration was checked to ensure successful MySQL connectivity. This helped maintain a consistent development environment. The local deployment and server configuration issues were resolved by running the frontend and backend as separate services and verifying communication between them. The React/Vite development server was used to provide the frontend, while Node.js and Express.js provided the backend REST APIs. CORS configuration was checked where required so that the frontend could communicate with the backend during local development.

API connectivity was verified using both browser Developer Tools and Postman. Testing and debugging were improved through a multi-layer testing approach. Individual frontend components and backend APIs were tested separately before performing integration testing. Postman was used for API testing, browser Developer Tools were used for frontend debugging, and MySQL was used for database verification. Complete workflows such as registration, login, menu selection, cart management, order placement, table selection, kitchen processing, and order tracking were then tested together. The collaboration and code management difficulties were addressed through Git and GitHub. The project source code was maintained in a shared repository, allowing changes to be tracked and previous versions to be recovered when required. Regular commits and organised project files helped reduce conflicts and maintain consistency between the frontend and backend components.

Documentation and configuration information were also maintained alongside the project source code. Beyond the technical fixes, the team also implemented process-level improvements such as repeated integration testing, API verification before frontend integration, database validation after important operations, and re-testing after bug fixes. These practices helped ensure that changes made to one module did not negatively affect other modules. The

complete customer-to-kitchen-to-admin workflow was repeatedly verified during development. In conclusion, the solutions applied throughout the project were both corrective and preventive in nature, reflecting a commitment to developing a stable and maintainable restaurant management system. Each solution addressed an immediate development problem while also improving the overall reliability of the application. The successful resolution of frontend-backend communication, database consistency, authentication, order processing, Kitchen Display System integration, table management, and administrative functionality helped transform the initial implementation into a more reliable and functional Smart Restaurant Ordering and Kitchen Display System.

10.3 Current Limitations or Known Bugs

Despite the comprehensive development and extensive refinement of the Smart Restaurant Ordering and Kitchen Display System, certain limitations and known bugs remain that are primarily due to constraints in time, resources, and the current development environment. While the system has been tested and validated under multiple scenarios, it continues to face minor challenges that are typical of integrated web applications involving customer ordering, kitchen operations, administration, and database management. This section provides a detailed and academically structured examination of the existing issues, categorising them based on their origin, nature, and impact on the system's functionality and user experience. One of the significant limitations lies in the system's dependency on the local development environment for frontend, backend, and database operation. The React.js frontend, Node.js and Express.js backend, and MySQL database currently operate within the local development setup. Therefore, the application is not yet configured as a fully public production system accessible from different locations. Any interruption to the local frontend server, backend server, or MySQL service can temporarily make the application unavailable. This limitation is particularly relevant when demonstrating the complete system because all three layers must be running correctly for the application to function.

Another limitation concerns real-time order updates between the customer interface and the Kitchen Display System. The current implementation retrieves and refreshes order information through API communication rather than using a dedicated real-time communication mechanism such as WebSockets. As a result, there may be a short delay before a newly placed order or an updated kitchen status becomes visible on another interface. Although the order information remains stored correctly in the database, instantaneous synchronisation between customer and kitchen screens is not currently implemented. The menu management system also has certain limitations related to data consistency. The application contains a large number of food items with information such as names, categories, prices, ratings, descriptions, and images. Maintaining the same identifiers and information between the frontend food data and MySQL database requires careful synchronisation. During development, differences between frontend and database identifiers occasionally resulted in incorrect item mapping in the cart or order workflow. These issues were corrected through data validation, but maintaining consistency remains an important consideration when new menu items are added.

The cart functionality has a minor limitation related to browser-based storage. Cart information is maintained on the client side during the ordering process. If the browser storage is cleared, modified, or becomes inconsistent with the current menu data, previously selected items may no longer appear correctly. Similarly, using an outdated cart from an earlier session can result in unexpected items being displayed. Although validation and cart clearing mechanisms help reduce this problem, a fully server-side persistent cart system is not currently implemented. From a database perspective, the system currently uses MySQL as a central relational database, which provides structured storage for

users, menu items, tables, orders, and order items. However, the current implementation is primarily intended for project development and demonstration rather than large-scale restaurant deployment. Very high numbers of simultaneous customers, orders, and database requests have not been tested under a large production workload. Therefore, additional database optimisation, indexing, connection management, and scalability measures may be required if the system is deployed across multiple restaurant branches. Another noteworthy limitation concerns table availability management. The system provides restaurant table information and allows customers to select tables during the ordering process.

However, advanced table reservation and real-time table locking mechanisms are not currently implemented. In a situation where multiple customers attempt to select the same table simultaneously, additional server-side validation would be required to prevent conflicts. Future versions could introduce real-time table availability updates and temporary table locking during order processing. The authentication and authorization system, although implemented with role-based access and password hashing, currently provides the essential login and access-control functionality required for the project. Advanced authentication mechanisms such as two-factor authentication, password recovery through email, refresh-token management, and social login are not included in the current implementation. These features could be incorporated in future versions to provide additional security and account-management capabilities. A recurring frontend limitation is browser and screen-size consistency. Although the React interface is designed to be responsive, certain components may display slight spacing, alignment, or sizing differences depending on the browser and screen resolution. Large menu sections, food images, tables, Kitchen Display cards, and Admin Dashboard components may require additional responsive optimisation for smaller mobile screens.

These limitations do not prevent the core functionality but indicate areas where further interface refinement could improve the user experience. The Kitchen Display System also has some limitations related to advanced kitchen workflow management. The current system supports the status progression from New → Accepted → Preparing → Ready → Served, but it does not currently provide advanced features such as priority-based order queues, estimated preparation times, separate kitchen stations, automatic sound notifications, or workload distribution between kitchen staff. These features could be considered for future improvements as the system becomes more complex. From a testing and maintenance standpoint, one current limitation involves test coverage across all possible scenarios. The major functionalities such as authentication, menu operations, cart management, order creation, table selection, kitchen processing, and administration have been tested, but exhaustive automated testing for every possible user interaction and failure condition has not been implemented. Additional automated regression and end-to-end test cases would improve confidence when future changes are introduced. The API and error-handling system also contains areas that can be improved. The REST APIs provide communication between the React frontend and Node.js backend, but unexpected server failures, database connection interruptions, or malformed requests may require additional detailed error handling. Improving centralised logging and monitoring would make it easier to identify the exact source of failures during future deployment and maintenance.

The deployment configuration presents another limitation because the current system is primarily designed for local development and demonstration. Cloud hosting, automated deployment pipelines, load balancing, auto-scaling, and production monitoring have not been implemented as part of the current project. Moving the application to a production environment would require additional configuration for secure hosting, database management, environment variables, HTTPS, monitoring, and backup procedures. From a user experience perspective, a few minor issues may occur when network connectivity is slow or unavailable. Since the frontend depends on the

backend REST APIs for menu, order, table, and user information, interruption of the backend or network connection can prevent new information from being retrieved. The system does not currently provide a complete offline ordering mechanism or automatic request synchronisation after connectivity is restored. Lastly, there exists a limitation in the system's scalability for large restaurant operations and multiple branches. The current architecture is designed around a single restaurant environment with a central set of users, menu items, tables, and orders. Supporting multiple restaurant branches would require additional entities and management features such as branch identification, branch-specific menus, separate table configurations, staff assignment, branch-level reporting, and central administration. In conclusion, although the Smart Restaurant Ordering and Kitchen Display System has achieved a high degree of functionality and stability, these current limitations highlight the iterative nature of software engineering. None of the identified issues prevent the core system from performing its primary functions, but they represent areas for improvement as the application evolves. Addressing these limitations will require continued testing, user feedback, database optimisation, improved real-time communication, stronger deployment infrastructure, and additional feature development. By identifying and documenting these limitations, the development team establishes a clear foundation for future enhancements while maintaining the system's core objective of providing an efficient digital environment for customers, kitchen staff, and administrators.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

The Smart Restaurant Ordering and Kitchen Display System has been designed with scalability and extensibility in mind, allowing future enhancements to be integrated without disrupting the existing system. While the current system provides core functionalities such as digital menu browsing, food ordering, table selection, kitchen order management, order status tracking, authentication, and administration, the project's long-term vision encompasses several planned features aimed at enhancing customer experience, increasing automation, improving restaurant operations, and expanding the system's capabilities. These planned developments are conceived not merely as optional add-ons, but as strategic components intended to transform the system into a more intelligent, efficient, and complete restaurant management solution. One of the most important planned features is the integration of online payment facilities. The current system focuses primarily on food selection, order placement, table selection, and order management. Future updates can integrate secure payment gateways that allow customers to complete their transactions digitally using UPI, debit cards, credit cards, or digital wallets. Payment confirmation can automatically update the corresponding order record and provide customers with a digital payment receipt. This will reduce the dependency on manual payment handling and provide a more convenient checkout experience. Another major enhancement planned for the system is real-time order notifications.

The current Kitchen Display System allows kitchen staff to update order statuses such as New, Accepted, Preparing, Ready, and Served. Future versions can introduce real-time notifications so that customers immediately receive updates whenever the status of their order changes. For example, when an order becomes Ready, the customer can receive a notification informing them that their food is ready. This feature can improve communication between customers and kitchen staff and reduce the need for customers to repeatedly check their order status. The integration of an AI-based food recommendation system is another important component of the future roadmap. The system can analyse customer preferences, previous orders, food categories, ratings, and frequently selected items to provide personalised recommendations.

For example, if a customer frequently orders vegetarian dishes or a particular category of food, the system can recommend similar items. AI-based recommendations can also be used to identify popular combinations and suggest suitable food items during the ordering process, improving customer engagement and helping restaurants promote relevant menu items. A dedicated mobile application is also among the major planned developments. While the current system is implemented as a web-based application using React.js, future versions can provide dedicated Android and iOS applications. The mobile application can provide customers with faster access to menus, orders, tables, and notifications. Features such as push notifications, mobile-friendly ordering, saved preferences, and order history can further improve accessibility and convenience for customers. To improve restaurant operations, a more advanced real-time Kitchen Display System is planned.

The future KDS can include priority-based orders, estimated preparation times, kitchen station allocation, sound notifications, and visual alerts for delayed orders. Orders could be automatically categorised based on preparation requirements, allowing different kitchen staff or stations to process them efficiently. This enhancement would provide better coordination during busy restaurant periods. Another planned feature is advanced table management

and reservation. The current system supports restaurant table information and table selection during ordering. Future versions can provide a visual restaurant floor layout showing available, occupied, reserved, and temporarily held tables. Customers could reserve tables before arriving at the restaurant, while restaurant staff could manage reservations and table availability through the Admin Dashboard. Real-time table locking can also prevent multiple customers from selecting the same table simultaneously. The project also plans to introduce an advanced customer profile and order history module. Customers will be able to view previous orders, frequently ordered food items, favourite dishes, and saved preferences. This information can be used to make repeat ordering faster and more convenient. Customers could also save preferred tables or food combinations for future visits, subject to the restaurant's operational requirements. On the administrative side, a more comprehensive business analytics dashboard is planned. The dashboard can provide detailed information about daily and monthly orders, revenue, popular menu items, customer activity, table utilisation, and kitchen performance. Graphs and charts can help administrators understand restaurant trends and identify frequently ordered products. Predictive analytics can also be introduced in future versions to help restaurants estimate demand and manage menu inventory more effectively. Another enhancement planned for the system is inventory and stock management. The current system focuses mainly on menu and order management, while future versions can connect menu items with ingredient availability. When an order is placed, the corresponding ingredient quantities can be updated automatically. If an ingredient reaches a defined threshold, the system can notify administrators to restock it.

Menu items can also be automatically marked unavailable when the required ingredients are out of stock. To improve communication between customers and restaurant staff, the system can include a digital feedback and rating module. Customers will be able to rate food items and provide feedback after completing their orders. Administrators can review customer feedback and identify highly rated or frequently criticised menu items. This information can support menu improvements and help restaurants understand customer preferences. The system can also be extended to support multiple restaurant branches. The current implementation is designed around a single restaurant environment, but future versions can introduce branch management functionality. Each branch could maintain its own menu, tables, kitchen orders, staff, and operating information while allowing administrators to manage multiple branches through a central dashboard. This would make the system suitable for restaurant chains and larger food-service businesses. In terms of backend development, future versions can introduce real-time communication technologies such as WebSockets to improve communication between customers, kitchen staff, and administrators. Instead of relying only on regular API requests for updated information, changes in order status, table availability, and kitchen activity could be transmitted instantly to connected users. This would improve the responsiveness of the complete restaurant system.

The project also envisions improvements in security and authentication. Future releases can include features such as two-factor authentication, password recovery, stronger session management, account verification, and detailed access-control policies. These improvements would provide additional protection for customer accounts and administrative operations as the system grows. Another future enhancement is the integration of digital receipts and billing management. Once an order is completed, the system can automatically generate a detailed digital bill containing food items, quantities, prices, taxes, discounts, payment information, and total amount. Customers could view or download their receipts, while administrators could maintain billing records for accounting and reporting purposes. The system can further support loyalty programmes and promotional offers.

Customers could earn reward points based on completed orders and redeem them for discounts or selected offers. Administrators could create promotional campaigns, discount codes, combo offers, and time-based deals. These features can help restaurants improve customer engagement and encourage repeat visits. In addition, future development can focus on automated order optimisation and kitchen workload management. The system could analyse incoming orders and provide kitchen staff with an organised preparation queue based on order time, preparation duration, table requirements, and priority. This can help reduce preparation delays and improve coordination between different kitchen activities. In conclusion, the planned features of the Smart Restaurant Ordering and Kitchen Display System reflect a forward-looking vision focused on automation, convenience, intelligent restaurant management, and improved customer experience.

Each enhancement—whether online payment, real-time notifications, AI recommendations, mobile applications, advanced KDS capabilities, inventory management, analytics, or multi-branch support—builds upon the existing system architecture and expands its practical capabilities. These planned developments ensure that the project can evolve from a digital ordering and kitchen management system into a more comprehensive and intelligent restaurant management platform.

11.2 Possible Integrations or Optimisations

As the Smart Restaurant Ordering and Kitchen Display System continues to evolve into a more sophisticated and intelligent restaurant management ecosystem, the need for advanced integrations and systemic optimisations becomes increasingly essential. The existing framework provides a strong foundation for digital food ordering, table management, kitchen order processing, and administrative management, but further refinements can significantly enhance scalability, performance, automation, and user satisfaction. This section provides a detailed academic exploration of potential integrations and optimisations that can be introduced to strengthen the platform's functional efficiency, interoperability, and long-term sustainability within the digital restaurant environment. Pasted markdown

One of the foremost integration priorities is the incorporation of online payment gateways such as Razorpay, Stripe, and UPI-based payment services.

Currently, the system primarily focuses on menu browsing, cart management, table selection, and order processing. By integrating secure payment services, customers could complete transactions directly through the application using UPI, debit cards, credit cards, and digital wallets. This integration would simplify the checkout process, reduce manual payment handling, and allow successful payment confirmations to be automatically linked with the corresponding order records. A parallel optimisation opportunity lies in introducing real-time communication mechanisms between customers, kitchen staff, and administrators. The present system communicates through REST APIs, while future development could incorporate WebSocket-based communication to provide instant updates. By adopting real-time event-driven communication, the platform could immediately notify customers when an order changes from New to Accepted, Preparing, Ready, or Served.

This would reduce the need for repeated API requests and improve the responsiveness of the Kitchen Display System. Another key optimisation involves the introduction of advanced API management and caching mechanisms. Frequently accessed information such as menu items, food categories, table availability, and popular dishes could be temporarily cached to reduce repeated database queries. API request throttling and structured error handling could also be implemented to prevent excessive requests from affecting system performance.

These optimisations would become increasingly important as the number of customers and restaurant orders grows. From a database perspective, one significant enhancement would be the introduction of advanced indexing, query optimisation, and database backup mechanisms. As the number of users, menu items, and orders increases, MySQL queries may require additional optimisation to maintain fast response times. Appropriate indexes can be applied to frequently searched fields such as user IDs, order IDs, table IDs, and menu item IDs. Automated database backups and recovery procedures could also be introduced to protect important restaurant data and improve system reliability. In the realm of user experience, Progressive Web Application (PWA) technology offers another promising integration opportunity.

PWA capabilities could allow customers to install the restaurant application directly on their devices and access selected features more conveniently. Cached menu information and service-worker functionality could improve loading performance and provide limited functionality during temporary connectivity problems. This enhancement would make the system more accessible across different devices while maintaining the benefits of a web-based application. A promising area for integration lies in Artificial Intelligence and Machine Learning-based food recommendations. By analysing previous orders, customer preferences, ratings, food categories, and frequently selected combinations, the system could generate personalised recommendations. Machine learning models could identify patterns in customer behaviour and suggest suitable dishes, meal combinations, or popular items. This would improve the ordering experience while helping restaurants promote relevant menu items. From an infrastructural standpoint, the platform can benefit substantially from containerisation and cloud deployment. Although the current system operates in a local development environment, future versions could use Docker containers to package the React frontend, Node.js backend, and MySQL-related services consistently.

Cloud hosting could then be introduced to make the application accessible remotely. Automated deployment and monitoring mechanisms could further improve maintainability and reduce configuration-related issues. In terms of payment and customer services, integration with digital receipt and notification services stands out as a useful enhancement. After a successful order and payment, the system could generate a digital receipt and send confirmation through email, SMS, or application notifications. Customers could receive notifications for order confirmation, preparation progress, food readiness, and order completion. This would improve transparency and communication throughout the ordering process. The next layer of optimisation involves advanced table management and reservation capabilities. The current system provides table information and allows customers to select tables during the ordering process. Future versions could introduce real-time table availability, table reservation, temporary table locking, and visual restaurant floor layouts. These features would reduce table-selection conflicts and provide administrators with better control over restaurant seating capacity.

A vital planned integration is inventory and stock management. Future versions could connect menu items with their required ingredients and automatically update stock quantities when orders are placed. When an ingredient reaches a minimum threshold, the system could notify administrators to restock it. Menu items could also be automatically marked unavailable when required ingredients are out of stock, improving the accuracy of the digital menu and reducing unsuccessful orders. Another prospective optimisation involves multi-branch restaurant management. The current system is designed primarily for a single restaurant environment, but the architecture can be extended to support multiple branches. Each branch could maintain its own menu items, tables, staff, kitchen orders, and operational information while administrators could monitor multiple branches through a central dashboard. This would make the system suitable for restaurant chains and larger food-service organisations.

From a security perspective, the platform can be strengthened through advanced authentication and security mechanisms. Future versions could incorporate two-factor authentication, account verification, password recovery, stronger session management, and detailed role-based access controls. These enhancements would provide additional protection for customer, kitchen, and administrator accounts as the system expands. From a performance perspective, load balancing and scalable server infrastructure can be introduced when the system is deployed for large-scale restaurant operations. Multiple backend instances could distribute incoming requests, while monitoring systems could track CPU usage, memory consumption, database performance, request latency, and server availability. Such mechanisms would help maintain stable performance during peak restaurant hours. Another useful integration is advanced analytics and business intelligence. The Admin Dashboard could be expanded to analyse daily and monthly sales, popular food items, customer ordering patterns, table utilisation, peak ordering periods, and kitchen performance. Visual charts and reports could help restaurant administrators make informed operational decisions. Predictive analytics could additionally be used to estimate future demand and support inventory planning. In terms of customer engagement, the system could integrate loyalty programmes, discounts, and promotional services. Customers could earn points based on completed orders and redeem them for discounts or selected offers. Administrators could create promotional campaigns, combo offers, discount codes, and time-based promotions. These features could increase customer engagement and encourage repeat visits. The platform could also integrate voice-based ordering and conversational assistance in future versions. Customers could interact with the system through natural-language commands to search for food, ask about menu items, or receive recommendations. A conversational assistant could also answer common questions regarding tables, orders, preparation status, and restaurant services, improving accessibility and convenience.

A further optimisation involves improving the Kitchen Display System through intelligent order prioritisation. The system could analyse preparation times, order timestamps, table requirements, and kitchen workload to organise incoming orders more efficiently. Visual alerts could identify delayed or high-priority orders, while different kitchen stations could receive only the orders relevant to their assigned preparation tasks. This could improve kitchen coordination during busy periods. Finally, the project can benefit from integration with external restaurant and food-service platforms where appropriate. Future APIs could provide services such as delivery tracking, customer communication, location-based restaurant information, and promotional integrations. Such integrations would expand the capabilities of the system while maintaining the central role of the existing ordering, kitchen, and administrative modules.

In conclusion, the potential integrations and optimisations for the Smart Restaurant Ordering and Kitchen Display System represent a holistic evolution from a functional digital ordering application into a smart, scalable, and highly integrated restaurant management ecosystem. Each proposed enhancement—whether payment integration, real-time communication, AI recommendations, inventory management, analytics, mobile access, or advanced kitchen operations—serves to improve the system’s resilience, responsiveness, and practical value. With strategic planning and phased implementation, these integrations can transform the platform into a future-ready restaurant technology solution aligned with emerging digital technologies and evolving customer expectations.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The Smart Restaurant Ordering and Kitchen Display System is a comprehensive web-based solution designed to simplify the process of food ordering and improve coordination between customers, kitchen staff, and restaurant administrators. The project aims to bring essential restaurant operations—including menu browsing, food selection, cart management, table selection, order processing, kitchen management, and order tracking—under a single integrated platform powered by modern web technologies and database management. Its primary purpose is to address the limitations of traditional restaurant ordering systems by providing a unified digital environment with an intuitive user interface. This section provides a detailed academic summary of the project, covering its conceptual foundation, technological design, development methodology, testing process, deployment approach, and overall outcome achieved during implementation. The genesis of this project lies in recognising the inefficiencies associated with traditional restaurant ordering methods. Customers often depend on manual communication with waiters, while kitchen staff receive handwritten or verbally communicated orders. Such processes can result in incorrect orders, delays, communication gaps, and difficulties in tracking order progress.

To overcome these challenges, the project introduces a centralised restaurant ordering ecosystem that connects customers, kitchen staff, and administrators through one unified web application. The conceptual model emphasises customer convenience, accurate data handling, efficient kitchen coordination, and improved order visibility. The project's scope extends beyond basic food ordering by incorporating table management, kitchen order processing, administrative management, and status tracking. From a technical perspective, the architecture of the platform follows a modular client-server architecture that separates the frontend, backend, and database layers. The frontend is developed using React.js with Vite, HTML, CSS, and JavaScript, providing a dynamic and responsive user interface for different system modules. The backend is developed using Node.js and Express.js, which handles server-side operations and RESTful API communication. MySQL is used as the relational database management system for storing structured information. The database contains entities such as users, menu items, restaurant tables, orders, and order items. This architecture provides organised data management and allows the different components of the system to communicate efficiently. The system integrates multiple internal REST API modules for users, menu items, orders, tables, order items, kitchen operations, and administrative operations.

Through these APIs, customers can register and log in, browse food items, add products to the cart, place orders, select tables, and view order status. Kitchen staff can retrieve incoming orders and update their status through the Kitchen Display System. Administrators can manage menu items, tables, users, and orders through the Admin Dashboard. The backend ensures that information submitted through the frontend is processed and stored correctly in the MySQL database. Authentication and role-based access are implemented to separate customer, kitchen, and administrator functionality. An essential highlight of the project is the Kitchen Display System (KDS), which provides kitchen staff with a centralised view of customer orders. Instead of depending on manual communication, kitchen staff can view order information including customer details, table information, food items, quantities, and instructions through a dedicated interface.

The order workflow follows the sequence New → Accepted → Preparing → Ready → Served. This provides clear visibility into the current state of every order and improves coordination between customer ordering and kitchen operations. The development process involved continuous implementation, integration, testing, and debugging of the different system modules. The project was developed incrementally, beginning with the frontend structure and customer interface, followed by backend APIs, MySQL database integration, authentication, Kitchen Display System, and Admin Dashboard. Git and GitHub were used for version control and source-code management. Postman was used extensively to test REST API endpoints and verify communication between the frontend and backend. Database records were also checked to ensure that operations such as user registration, menu retrieval, order creation, table selection, and order status updates were correctly reflected in MySQL. During the implementation phase, several integration and debugging issues were identified and resolved. These included frontend-backend communication problems, incorrect API responses, menu item and database identifier mismatches, cart data inconsistencies, authentication issues, and incorrect dashboard data mapping. The development team addressed these issues through repeated API testing, database verification, frontend debugging, and integration testing. The complete workflow from customer login and menu selection to order placement, kitchen processing, and order tracking was tested to ensure that the major components worked together correctly. The deployment process currently focuses primarily on a local development environment, where the React/Vite frontend, Node.js and Express.js backend, and MySQL database operate as separate components. The frontend is accessed through the Vite development server, while the backend provides REST APIs through the Node.js server. Database connectivity is configured through the backend MySQL connection. Environment configuration and dependency management are maintained through the respective project configuration files. This setup provides a controlled environment for development, testing, debugging, and project demonstration. From a user experience perspective, the interface was designed with simplicity and accessibility in mind. The customer interface allows users to move logically from menu browsing to cart management, checkout, table selection, and order tracking. The Kitchen Display System provides kitchen staff with a focused view of active orders and their current status. The Admin Dashboard provides administrators with centralised management capabilities. Responsive layouts and clear navigation are used across the application to make the major functions easier to access and understand. The project's testing and optimisation phases focused on improving system reliability, data consistency, and communication between the application layers. API testing was performed using Postman, while frontend behaviour was verified through browser testing and developer tools. Database records were cross-checked after important operations to confirm that the correct information was stored. Integration testing verified communication between the React frontend, Node.js backend, and MySQL database.

Functional workflows including registration, login, menu browsing, cart operations, order placement, table selection, kitchen processing, and administration were tested under different conditions. The final version of the system represents a balance between functionality, usability, database management, and practical restaurant operations. It provides customers with a single environment to browse food, place orders, select tables, and track their orders. Kitchen staff receive a dedicated interface for processing incoming orders, while administrators can manage important restaurant information through the Admin Dashboard.

From a development standpoint, the project demonstrates the integration of frontend technologies, backend services, REST APIs, authentication, and relational database management in a real-world application context.

In academic terms, this project demonstrates the practical application of knowledge acquired in web development, database management, software engineering, API integration, authentication, and user interface design. It incorporates important software engineering principles including modular development, database relationships, client-server communication, testing, debugging, version control, and documentation.

The systematic approach to designing, implementing, testing, and refining the application reflects the practical stages of the software development lifecycle. The success of the Smart Restaurant Ordering and Kitchen Display System lies not only in its technical implementation but also in its practical value for restaurant operations. It reduces dependence on manual ordering, improves communication between customers and kitchen staff, provides better order visibility, and centralises restaurant management activities. The modular structure also provides a foundation for future enhancements such as online payments, real-time notifications, AI-based food recommendations, mobile applications, inventory management, advanced analytics, and multi-branch restaurant support. In conclusion, the project represents the journey from conceptualisation to implementation of a functional digital restaurant management platform that combines convenience, automation, and efficient order coordination. It demonstrates proficiency in modern full-stack web development, database integration, REST API communication, authentication, and real-world problem-solving. The Smart Restaurant Ordering and Kitchen Display System stands as both an academic achievement and a practical prototype for modern restaurant management, providing a unified digital environment that connects customers, kitchen staff, and administrators while creating a strong foundation for future technological enhancements.

12.2 What Was Achieved

The Smart Restaurant Ordering and Kitchen Display System stands as a major milestone in the development of a digital restaurant management solution, representing the culmination of planning, design, coding, database integration, API development, and testing efforts. Through this project, we successfully developed a unified web-based platform capable of handling important restaurant operations within a single interface—an achievement that simplifies the ordering process and demonstrates the practical application of full-stack web development principles. The outcomes of this project can be broadly categorised into technical achievements, functional deliverables, user experience enhancements, and academic learnings, each of which reflects both theoretical understanding and practical implementation. The most immediate and visible achievement is the successful integration of multiple restaurant modules into one cohesive web application. The system incorporates three major user areas—Customer, Kitchen, and Admin—which are traditionally handled through separate manual processes. By bringing these operations into a unified ecosystem, the system reduces the need for manual communication and provides a centralised platform for restaurant activities.

Each module performs its specific responsibilities while communicating through the backend APIs and shared database. This integration ensures that when a customer places an order, the information can be processed by the backend and displayed to the kitchen staff for further processing. From a technical perspective, the project achieved functionality across the major layers of the technology stack. On the frontend, a responsive and interactive interface was developed using React.js, Vite, HTML, CSS, and JavaScript. These technologies enabled the creation of dynamic interfaces for customers, kitchen staff, and administrators. The frontend follows a component-based design approach, allowing individual components such as the Navbar, Menu, Cart, Checkout, Orders, Kitchen Dashboard, and Admin Dashboard to be developed and maintained independently.

Features such as menu browsing, food selection, cart management, table selection, order placement, and status tracking were implemented with attention to usability and navigation. On the backend, significant progress was achieved by developing a structured server environment using Node.js and Express.js. The backend is responsible for handling authentication, menu operations, table management, order processing, kitchen operations, administrative functions, and database communication. RESTful APIs were developed for users, menu items, orders, tables, order items, kitchen operations, and administration. The successful implementation of these APIs allows information to move between the React frontend and MySQL database in a structured manner. Backend routes and controllers were organised according to their respective responsibilities, improving the maintainability of the application. One of the important accomplishments of the project is the successful implementation of the MySQL relational database. The database contains structured tables for users, menu items, restaurant tables, orders, and order items. Relationships between these entities allow customer information, selected food items, order details, and table information to be maintained consistently.

The implementation of primary keys and foreign-key relationships provides structured data management and helps maintain relationships between different restaurant operations. This database integration represents an important academic achievement in applying relational database concepts to a real-world application. Another critical achievement lies in the integration between the frontend, backend, and database layers. The project successfully establishes communication through REST APIs using JSON-based requests and responses. Customer actions such as registration, login, menu retrieval, order creation, table selection, and order tracking are processed through the backend and connected to the MySQL database. Similarly, kitchen and administrative operations retrieve the required information through backend endpoints. Postman was used to test API requests and responses, helping verify that the different system components communicated correctly. In terms of user experience and interface design, the project achieved a functional and organised restaurant interface. The customer module provides a clear pathway from menu browsing to food selection, cart management, checkout, table selection, and order tracking. The Kitchen Display System presents incoming orders in a structured format so that kitchen staff can easily identify customer information, table details, food items, quantities, and instructions. The Admin Dashboard provides a centralised interface for managing restaurant information. These interfaces were developed to make the major functions accessible without unnecessary complexity. From the authentication and authorisation perspective, the project successfully implemented login and registration functionality for different types of users. Passwords are protected using bcrypt hashing, while the system uses role information to distinguish between customers, kitchen staff, and administrators. Role-based access ensures that users are directed towards the appropriate functionality. Customers access ordering-related features, kitchen staff access the Kitchen Display System, and administrators access management functions. This separation improves the organisation and security of the application. One of the most meaningful accomplishments of this project is the successful implementation of the Kitchen Display System. The KDS provides kitchen staff with a centralised view of active customer orders instead of relying on handwritten notes or verbal communication. Orders can be viewed along with their food items, quantities, table information, customer details, and instructions.

The system supports the complete order status workflow of New → Accepted → Preparing → Ready → Served. This provides better visibility of order progress and establishes a clear connection between customer orders and kitchen operations. Another significant achievement is the implementation of digital table selection and management. Customers can select an available restaurant table during the ordering process, while the system stores the selected table information with the corresponding order.

The Admin module also provides table management functionality. This integration connects customer orders with restaurant seating information and reduces the need for separate manual table records. Furthermore, the project successfully implemented important administrative functionality. The Admin Dashboard provides administrators with information related to users, menu items, orders, active orders, revenue, and tables. Separate administrative interfaces support menu and table management. This gives administrators a centralised view of important restaurant operations rather than requiring them to manage each activity independently. Extensive testing and quality assurance activities were also carried out during development.

Postman was used for API testing and validation, while browser-based testing was used to verify frontend functionality. Integration testing confirmed communication between React, Node.js, Express.js, and MySQL. Important workflows including registration, login, menu retrieval, cart operations, order placement, table selection, kitchen processing, and order status updates were tested. Database records were also checked to ensure that operations performed through the application were correctly stored and retrieved. The project also provided valuable experience in identifying and resolving real development issues. Problems involving frontend-backend communication, incorrect API responses, database identifier mismatches, cart data inconsistencies, authentication, and dashboard data mapping were encountered during implementation. These issues were resolved through debugging, API testing, database verification, and repeated integration testing.

The process strengthened the understanding of how different software components interact within a complete web application. From a development and project-management perspective, Git and GitHub were used for version control and source-code management. This allowed the project files and changes to be maintained systematically and supported collaborative development. The use of version control also helped maintain different stages of the application during development and debugging. From an academic standpoint, this project provided an important opportunity to apply theoretical principles of Software Engineering, Database Management, Web Development, REST API Integration, Authentication, and User Interface Design in a practical environment.

It strengthened understanding of real-world development activities including requirement analysis, database design, frontend development, backend development, API testing, debugging, integration testing, version control, and documentation. It also improved practical problem-solving and collaborative development skills. In summary, the Smart Restaurant Ordering and Kitchen Display System achieved its primary goal of simplifying restaurant ordering and improving communication between customers, kitchen staff, and administrators through a single integrated platform. It delivered a functional, organised, and database-driven web solution with customer ordering, kitchen order processing, table management, order tracking, and administrative management.

The project demonstrates practical knowledge of full-stack development and system integration while providing a strong foundation for future improvements such as online payment integration, real-time notifications, mobile applications, AI-based food recommendations, inventory management, and advanced restaurant analytics. Overall, the project successfully transformed the initial concept into a working digital restaurant management solution. It demonstrates how modern web technologies can be combined with relational database management and REST APIs to solve practical problems in restaurant operations. The completed system represents both an academic achievement and a functional prototype that can be further enhanced into a more advanced restaurant automation platform.

12.3 Skills Learned During Development

The development of the Smart Restaurant Ordering and Kitchen Display System provided an excellent opportunity to translate theoretical knowledge into a practical, real-world full-stack web development project. The experience offered not only technical proficiency but also growth in analytical thinking, project planning, teamwork, debugging, and academic discipline.

This section summarises the extensive range of skills—technical, analytical, managerial, and interpersonal—gained throughout the project lifecycle. The culmination of these competencies has enhanced our understanding of how complete web-based systems are designed, developed, integrated, tested, and maintained within practical and academic environments. One of the foremost technical skills developed during this project was full-stack web development, covering both frontend and backend environments. On the frontend side, proficiency in React.js was significantly strengthened. The project involved designing dynamic, modular, and reusable interfaces for customers, kitchen staff, and administrators. Through developing components such as the menu, cart, checkout, orders, Kitchen Display System, and Admin Dashboard, we gained practical experience in React components, hooks, state handling, event management, routing, and responsive interface development.

Working with HTML, CSS, and JavaScript alongside React.js further improved our understanding of modern frontend development practices. From the backend perspective, the project strengthened our abilities in Node.js and Express.js, teaching us the principles of server-side programming, routing, middleware management, and REST API development. We learned how to create backend routes for users, menu items, orders, tables, order items, kitchen operations, and administrative operations. Developing RESTful APIs helped us understand how frontend and backend layers communicate through HTTP requests and JSON responses. Error handling, input validation, asynchronous programming, and database connectivity became important parts of the learning process. This exposure provided a strong understanding of backend architecture and data flow within a web application. A major learning milestone was achieved in database design and management. Implementing the MySQL database required understanding relational database concepts, table structures, primary keys, foreign keys, relationships, and CRUD operations. We developed practical experience in designing tables for users, menu items, restaurant tables, orders, and order items. Creating relationships between these entities helped us understand how real-world restaurant information can be organised within a relational database. Querying, inserting, updating, and retrieving records through the backend strengthened our knowledge of SQL and database connectivity.

A critical area of growth was in API integration and data communication. The project required connecting the React frontend with the Node.js and Express.js backend through RESTful APIs. Through this process, we developed a solid understanding of HTTP methods such as GET, POST, PUT, and DELETE, request parameters, JSON data, response handling, and API endpoints. We learned how customer actions such as login, menu retrieval, order placement, table selection, and order tracking are converted into API requests and processed by the backend. Testing these communication flows using Postman provided practical knowledge of API development and integration. Another key area of development was authentication and role-based authorisation. We learned how to implement registration and login functionality and protect user passwords using bcrypt hashing. The project introduced us to the importance of assigning different roles to customers, kitchen staff, and administrators.

We learned how role information can be used to control access to different parts of the application. Implementing protected routes and role-specific functionality improved our understanding of application security and access control. From the project management perspective, this endeavour provided practical experience in iterative software development. The project was developed in stages, beginning with the frontend structure and gradually adding backend services, database integration, authentication, ordering functionality, Kitchen Display System, and Admin Dashboard. This incremental approach helped us divide the project into manageable tasks and identify problems at earlier stages. Regular testing and debugging allowed changes to be introduced without affecting the complete application unnecessarily. Proficiency in version control systems such as Git and GitHub was also an important achievement. The project required maintaining source code and tracking changes throughout development. Through Git, we learned how to create commits, manage project versions, work with branches, and maintain a structured development history.

GitHub provided a central location for storing and managing the project code. This experience strengthened both technical discipline and collaboration skills and provided practical exposure to commonly used software development workflows. The project also provided practical experience in environment and dependency management. Working with React/Vite, Node.js, Express.js, and MySQL required configuring different development tools and dependencies correctly. We learned how package configuration files such as `package.json` and `package-lock.json` help manage project dependencies. Setting up the frontend and backend environments separately also improved our understanding of how different application layers operate together within a local development environment. A particularly significant learning experience was the use of testing tools and methodologies. Employing Postman for API testing enabled us to validate backend endpoints and identify incorrect responses or integration problems. Browser-based testing was used to verify frontend functionality and user interactions.

We learned to perform functional and integration testing for features such as registration, login, menu browsing, cart operations, order placement, table selection, kitchen processing, and order tracking. Debugging through browser developer tools, backend console output, and database verification improved our problem-solving abilities and demonstrated the importance of systematic quality assurance. The Kitchen Display System provided another important learning experience. Developing the KDS required understanding how customer orders are retrieved, displayed, and updated by kitchen staff.

We learned how order status changes can be represented within an application using the sequence New → Accepted → Preparing → Ready → Served. Working on this module improved our understanding of real-time-style operational workflows, data retrieval, status management, and user-specific interfaces. The UI/UX design process also contributed significantly to our skill set. Designing separate interfaces for customers, kitchen staff, and administrators required consideration of usability, navigation, information organisation, and visual clarity. The customer interface was structured around the natural ordering process, while the Kitchen Display System focused on quick access to order information. The Admin Dashboard was organised around restaurant management functions. This experience strengthened our understanding of how interface design should reflect the requirements and responsibilities of different users. Beyond technical proficiencies, the project cultivated essential analytical and problem-solving skills. Each phase of development involved identifying technical problems, analysing possible causes, evaluating solutions, and implementing suitable fixes. Issues such as incorrect API responses, database mismatches, cart inconsistencies, authentication problems, and dashboard data mapping required systematic debugging. This process improved our logical thinking, troubleshooting ability, patience, and adaptability.

Equally important were the communication and teamwork skills developed through collaborative coding, documentation, testing, and presentation of the project. Working as a team taught us how to divide responsibilities, discuss implementation decisions, communicate technical issues, and coordinate different development activities. The collaborative workflow provided practical experience of how software development projects require both individual responsibility and effective team communication. Academically, this project served as a bridge between theory and practical implementation. It allowed us to apply classroom concepts such as Software Development Life Cycle (SDLC), database management, relational database design, client-server architecture, REST APIs, authentication, software testing, and version control. We also learned to prepare structured technical documentation, maintain project records, and present the system in a formal academic format. This systematic approach strengthened our research capabilities, attention to detail, and technical writing skills. Moreover, the project cultivated a sense of innovation and practical thinking. Designing a system that connects customer ordering, table selection, kitchen processing, order tracking, and administrative management required us to think beyond individual features and understand how different restaurant operations work together. The development of the Kitchen Display System particularly demonstrated how software can replace manual communication processes with a structured digital workflow. This experience reinforced the importance of designing software around real-world problems and user requirements.

Overall, the development of the Smart Restaurant Ordering and Kitchen Display System provided a comprehensive learning experience covering frontend development, backend development, database management, REST API integration, authentication, testing, debugging, version control, UI/UX design, documentation, and teamwork. The project strengthened our technical knowledge while also improving our analytical, communication, and problem-solving abilities. These skills provide a strong foundation for developing more advanced full-stack applications and for applying software engineering concepts in future academic and professional projects.

CHAPTER -13 REFERENCES

Books

Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson Education. Duckett, J. (2014). *HTML & CSS: Design and Build Websites*. Wiley.

Duckett, J. (2015). *JavaScript and JQuery: Interactive Front-End Web Development*. Wiley.

Tutorials and Learning Resources

W3Schools. (2025). *React.js, HTML, CSS, JavaScript, and Web Development Tutorials*.

Retrieved from <https://www.w3schools.com>

MDN Web Docs. (2025). *Web Technologies Documentation*. Retrieved from <https://developer.mozilla.org>

React Documentation. (2025). *React.js Documentation*. Retrieved from <https://react.dev>

Node.js Documentation. (2025). *Node.js Documentation*. Retrieved from <https://nodejs.org/docs/latest/api/>

Express.js Documentation. (2025). *Express.js Web Framework Documentation*. Retrieved from <https://expressjs.com>

GeeksforGeeks. (2025). *React.js, Node.js, Express.js, and Database Tutorials*. Retrieved from <https://www.geeksforgeeks.org>

APIs and Tools Used

MySQL. (2025). *MySQL Documentation*. Retrieved from <https://dev.mysql.com/doc/>

Postman. (2025). *API Platform and Documentation*. Retrieved from <https://www.postman.com>

Vite. (2025). *Vite Documentation*. Retrieved from <https://vite.dev>

GitHub. (2025). *GitHub Documentation*. Retrieved from <https://docs.github.com> Git. (2025).

Git Documentation. Retrieved from <https://git-scm.com/doc>

Documentation and Research References

MySQL. (2025). *MySQL Reference Manual and Database Documentation*. Retrieved from <https://dev.mysql.com/doc/refman/>

React. (2025). *React Documentation and Learning Resources*. Retrieved from <https://react.dev/learn Express.js> (2025).

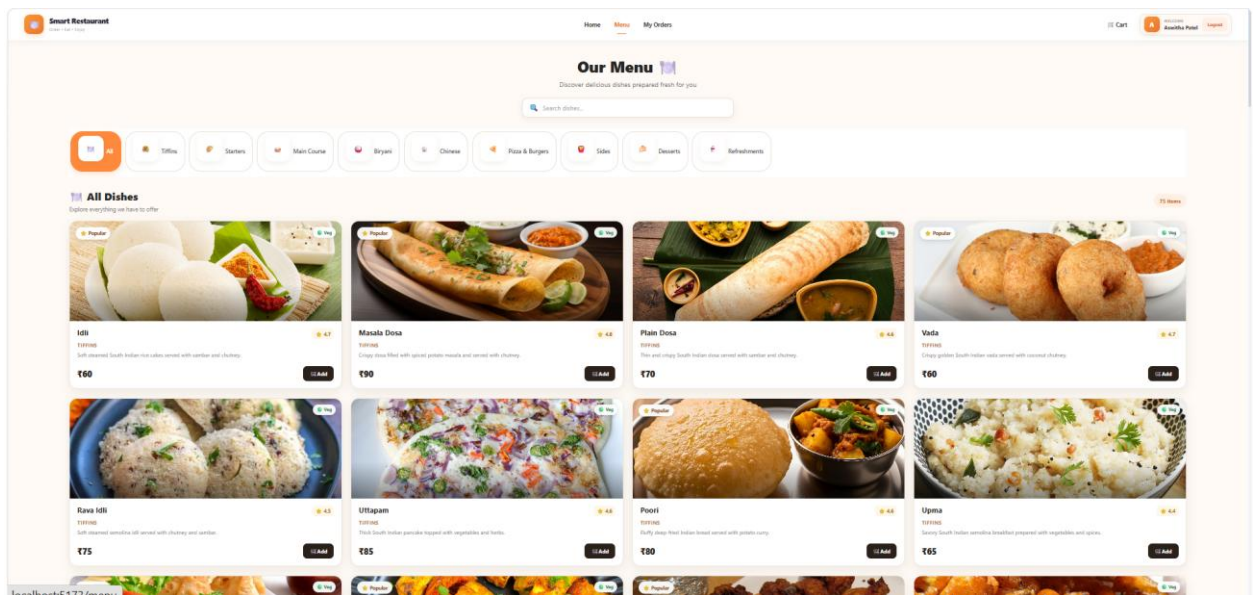
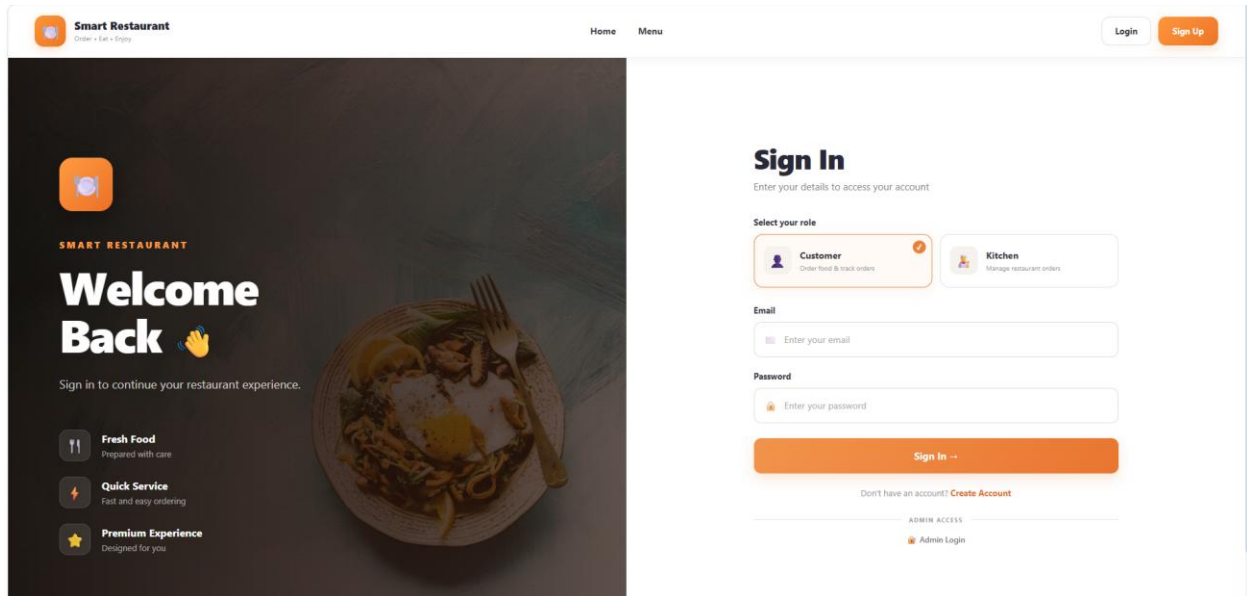
Express.js Guide and API Reference. Retrieved from <https://expressjs.com/en/guide/>

Node.js. (2025). *Node.js Guides and API Documentation*. Retrieved from <https://nodejs.org/en/docs>

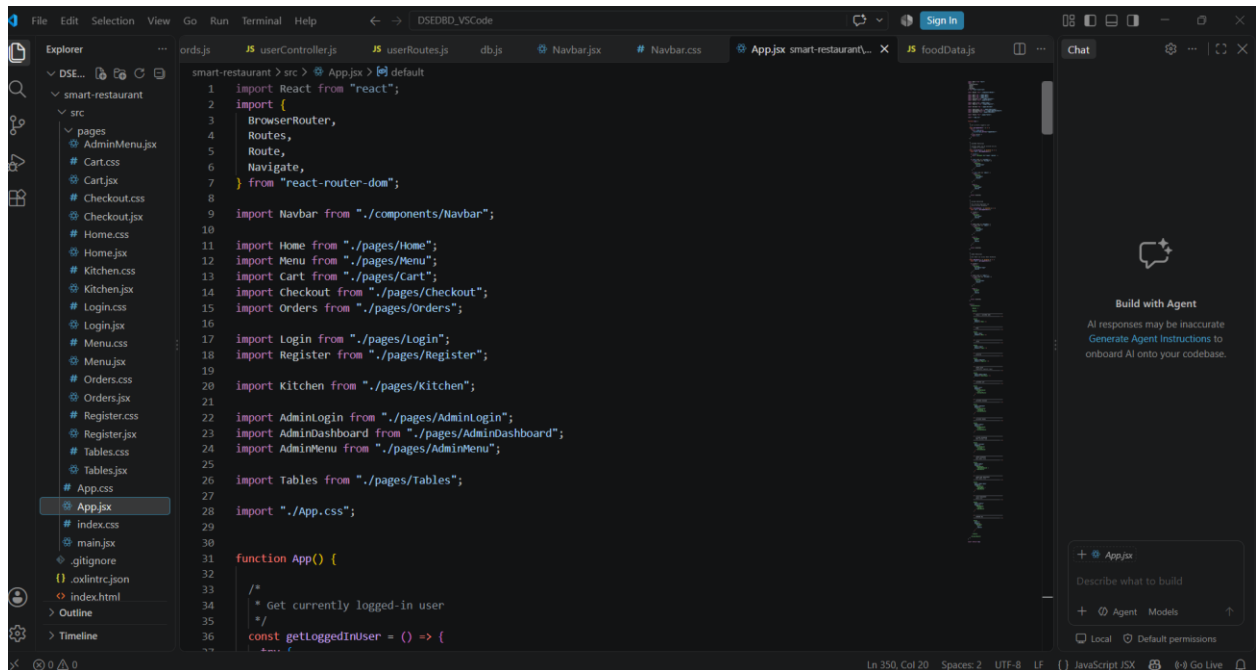
GitHub. (2025). *GitHub Documentation and Developer Resources*. Retrieved from <https://docs.github.com>

CHAPTER -14 APPENDICES

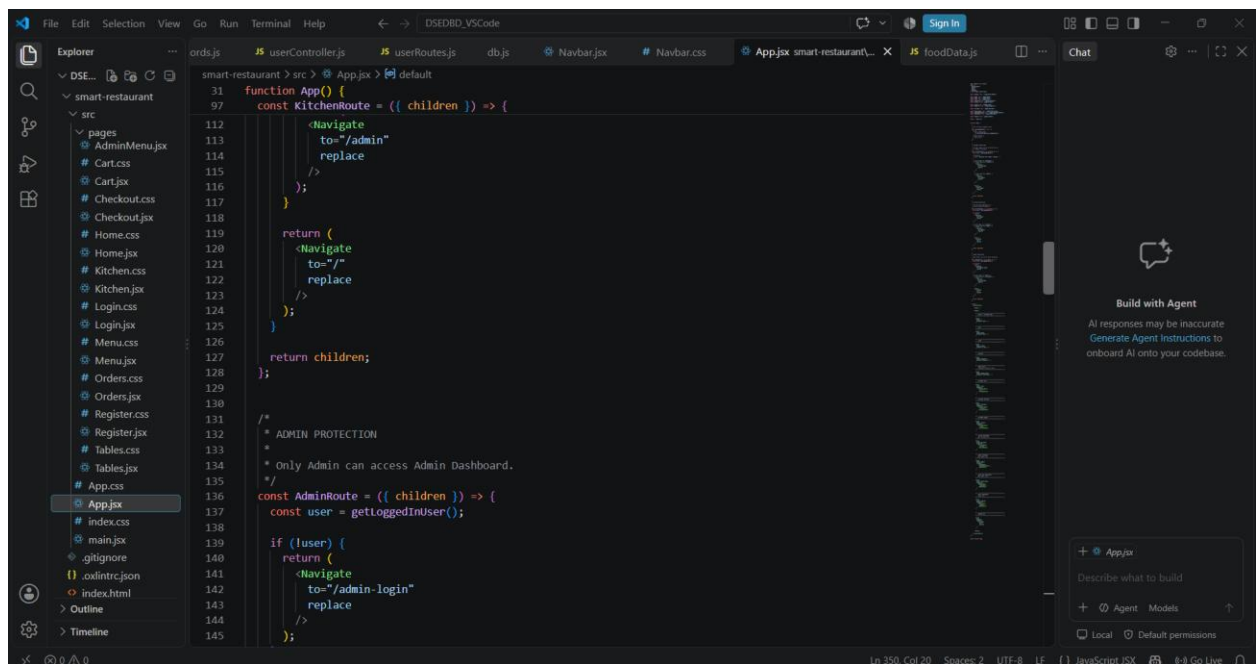
Screenshots of the app



Sample code snippets



```
1 import React from "react";
2 import {
3   BrowserRouter,
4   Routes,
5   Route,
6   Navigate,
7 } from "react-router-dom";
8
9 import Navbar from "../components/Navbar";
10
11 import Home from "../pages/Home";
12 import Menu from "../pages/Menu";
13 import Cart from "../pages/Cart";
14 import Checkout from "../pages/Checkout";
15 import Orders from "../pages/Orders";
16
17 import Login from "../pages/Login";
18 import Register from "../pages/Register";
19
20 import Kitchen from "../pages/Kitchen";
21
22 import AdminLogin from "../pages/AdminLogin";
23 import AdminDashboard from "../pages/AdminDashboard";
24 import AdminMenu from "../pages/AdminMenu";
25
26 import Tables from "../pages/Tables";
27
28 import "../App.css";
29
30
31 function App() {
32
33   /*
34    * Get currently logged-in user
35    */
36   const getLoggedInUser = () => {
```



```
31 function App() {
32   const KitchenRoute = ({ children }) => {
33     return (
34       <Navigate
35         to="/admin"
36         replace
37       />
38     );
39   };
40
41   return (
42     <Navigate
43       to="/"
44       replace
45     />
46   );
47
48   return children;
49 );
50
51 /*
52 * ADMIN PROTECTION
53 *
54 * Only Admin can access Admin Dashboard.
55 */
56 const AdminRoute = ({ children }) => {
57   const user = getLoggedInUser();
58
59   if (!user) {
60     return (
61       <Navigate
62         to="/admin-login"
63         replace
64       />
65     );
66   };
67
68   return children;
69 );
70 }
```

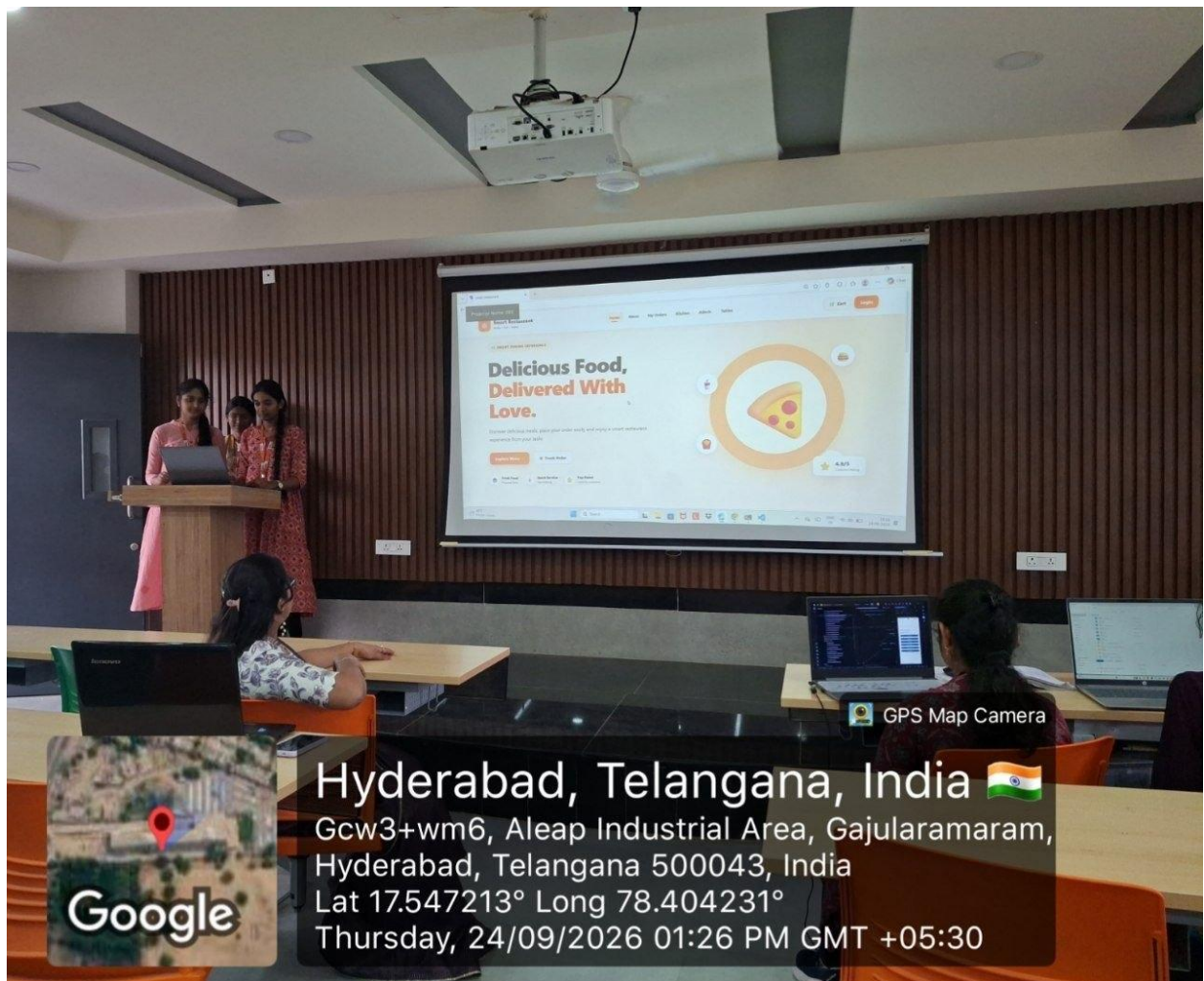
Installation/setup instructions

The Smart Restaurant Ordering and Kitchen Display System is installed and configured using React.js, Node.js, Express.js, and MySQL. The setup requires the frontend, backend, and database components to be configured correctly. The setup can be completed in a few steps. First, download or copy the complete Smart Restaurant project folder to the local computer. If the project is shared as a ZIP file, extract it using any standard extraction tool. Make sure the main project folder contains the React frontend files and the backend folder containing the Node.js and Express.js server files. Next, open the project folder in Visual Studio Code. Open the terminal and install the required frontend dependencies using the npm install command. Similarly, navigate to the backend folder and install the backend dependencies using npm install. Make sure that Node.js and npm are installed on the system before performing these steps. The MySQL server should also be installed and running on the local computer. After setting up MySQL, create the smart_restaurant database and execute the required SQL commands to create the tables such as users, menu_items, restaurant_tables, orders, and order_items.

Configure the backend database connection according to the local MySQL setup. Once the database is connected, start the Express backend server. The backend runs locally on <http://localhost:5000>. The frontend can then be started using the Vite development server. From the main project folder, run the appropriate npm command to start the React application. The frontend is available locally at <http://localhost:5173>. Open this address in a modern browser such as Google Chrome, Microsoft Edge, or Mozilla Firefox. The frontend communicates with the Node.js and Express backend through REST APIs, while the backend communicates with the MySQL database. After starting all three components, the complete restaurant system can be used for demonstration and testing. User Manual or Guide When the Smart Restaurant Ordering and Kitchen Display System is opened, the homepage provides navigation to the main restaurant functions. Customers can begin by selecting the Login or Register option. After successful login, customers can browse the digital menu, view food items, select items, and add them to the cart. The cart displays the selected food items and quantities before proceeding to checkout. During the ordering process, customers can place their order and select an available restaurant table. After the order is submitted, customers can view their order and track its current status.

The order status progresses through New → Accepted → Preparing → Ready → Served as it is processed by the kitchen. Kitchen staff can log in through the kitchen interface and access the Kitchen Display System (KDS). The KDS displays incoming orders along with customer details, table information, food items, quantities, and instructions. Kitchen staff can update the status of each order according to its preparation stage. Administrators can access the Admin Dashboard to manage important restaurant information. The admin interface provides functionality for managing menu items, tables, users, and orders. The dashboard also provides an overview of restaurant activities such as total orders, active orders, customers, revenue, and available tables. The system provides separate functionality for customers, kitchen staff, and administrators through role-based access. The complete application demonstrates how digital ordering, kitchen management, table management, and order tracking can be integrated into a single restaurant management platform.

Geo Tag photos with guide





Review forms with guide signatures

